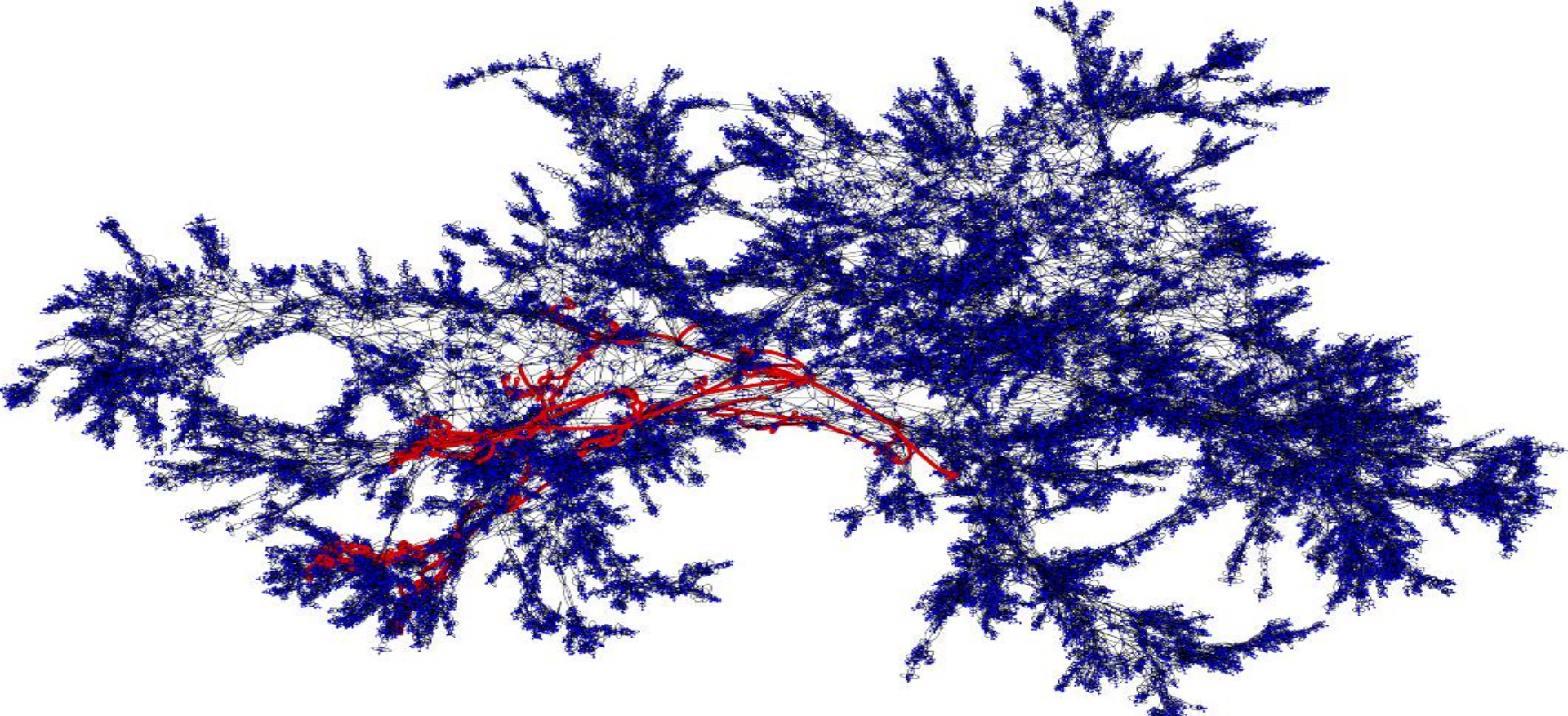




Data Structures

CPT102 – Lecture 10
Erick Purwanto



Xi'an Jiaotong-Liverpool University

西交利物浦大學

CPT102 Data Structures

Lecture 10

Shortest Paths and Minimum Spanning Trees

Welcome!

- Welcome to Lecture 10 !
- In this lecture we are going to
 - learn about shortest paths:
 - shortest path trees
 - single-source shortest paths with Dijkstra's algorithm
 - learn about minimum spanning trees:
 - minimum spanning trees and cut property
 - Prim's algorithm
 - Kruskal's algorithm

Part 1: Shortest Paths

- Recall that in Week 9 (last week) we have learned to find a shortest path in an unweighted graph using BFS
- Can we use BFS to compute a shortest route from location s to t in a map application?

Breadth First Search in Map Applications

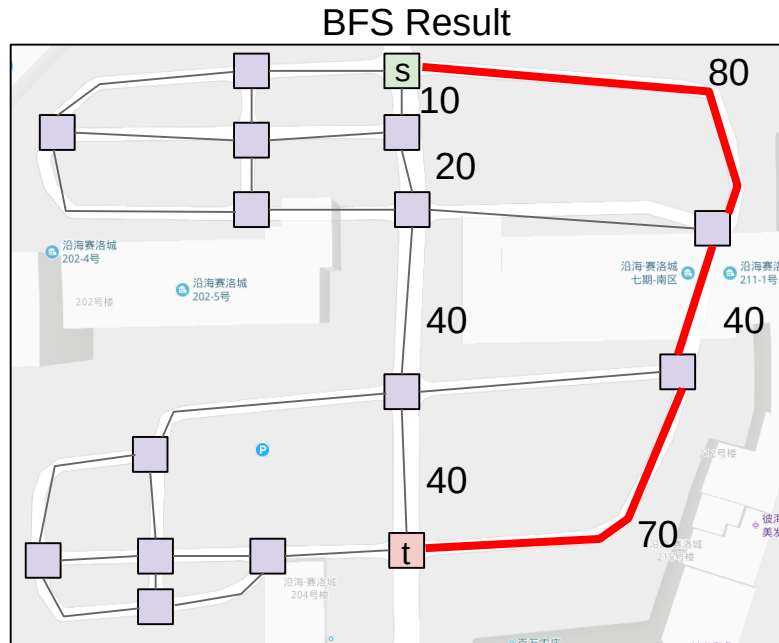
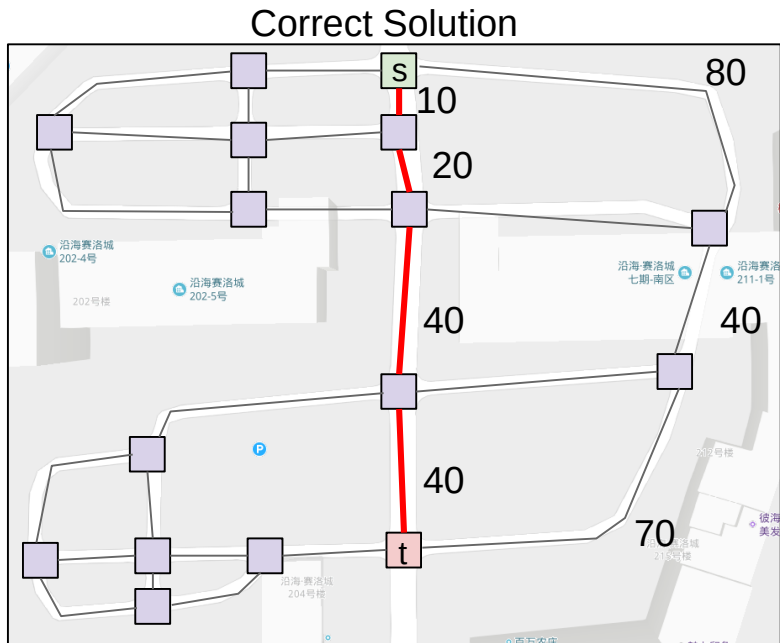
Suppose we're trying to get from s to t



Breadth First Search for Mapping Applications

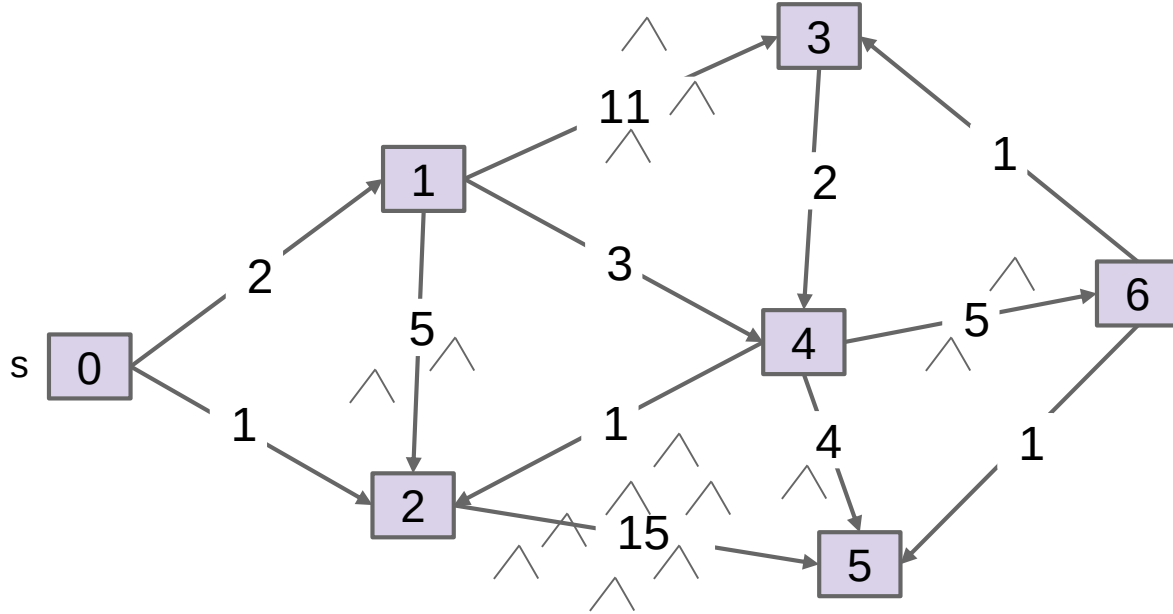
BFS yields the wrong route from s to t

- BFS yields a route of length 190 instead of 110
- need an algorithm that takes into account edge distances (edge weights)



Single Source Single Target Shortest Paths

Goal: Find the shortest paths from source vertex s to some target vertex t

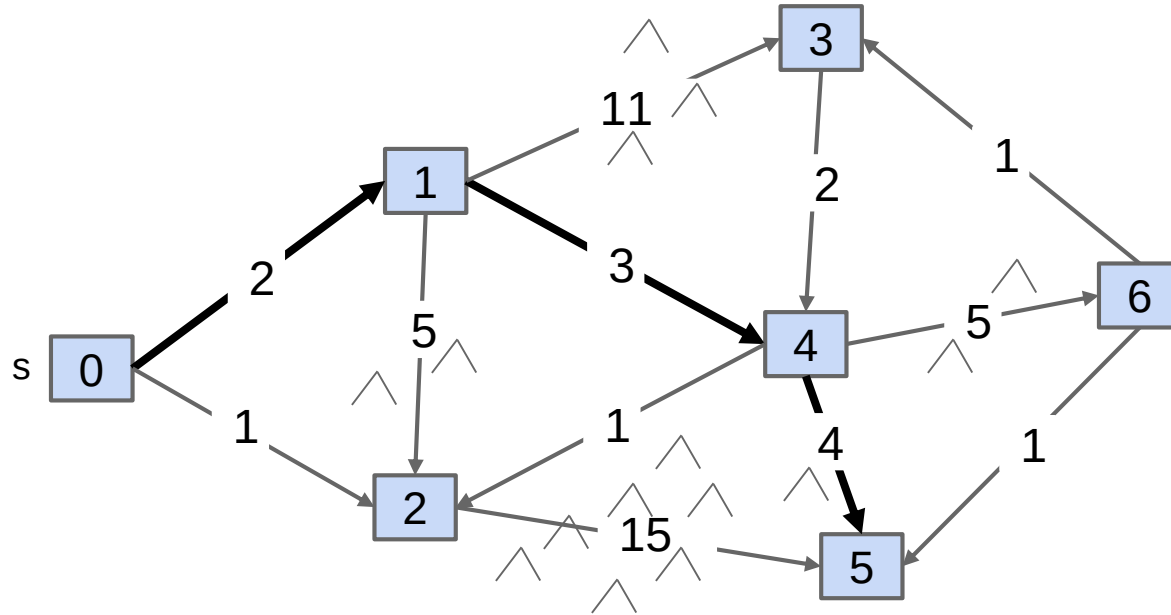


Challenge: Try to find the shortest path from town 0 to town 5

- Each edge has a number representing the length of that road in miles

Single Source Single Target Shortest Paths

Goal: Find the shortest paths from source vertex s to some target vertex t



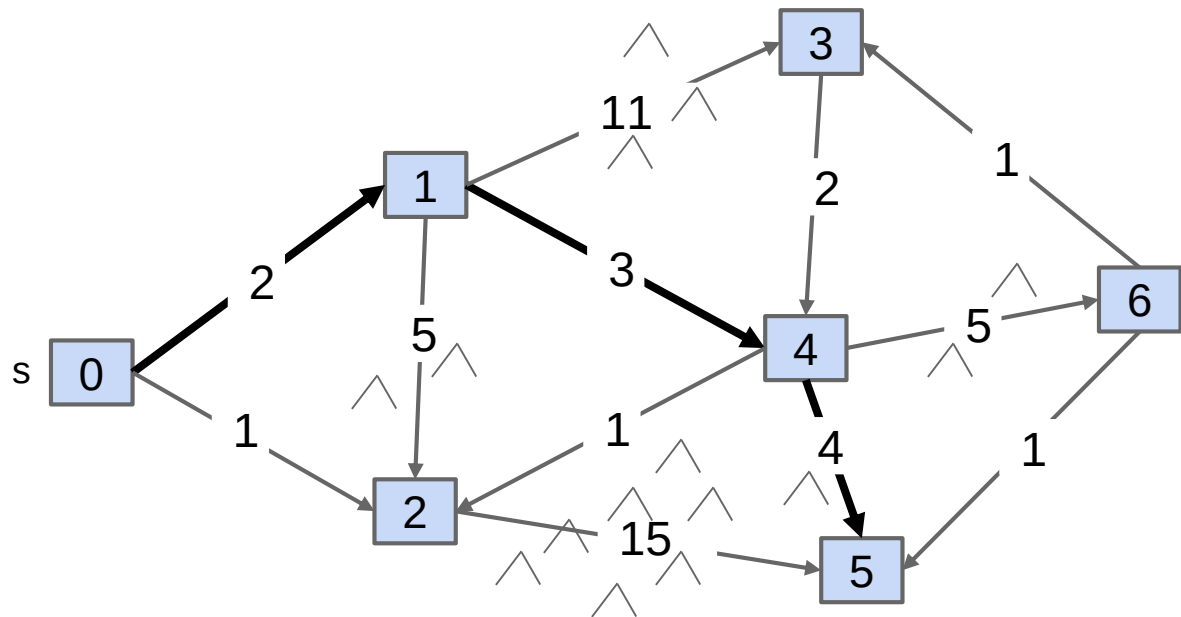
Best path from 0 to 5 is

- 0 -> 2 -> 4 -> 5
- Total length is 9 miles

The path 0 -> 2 -> 5 only involves three towns, but total length is 16 miles

Single Source Single Target Shortest Paths

Goal: Find the shortest paths from source vertex s to some target vertex t .



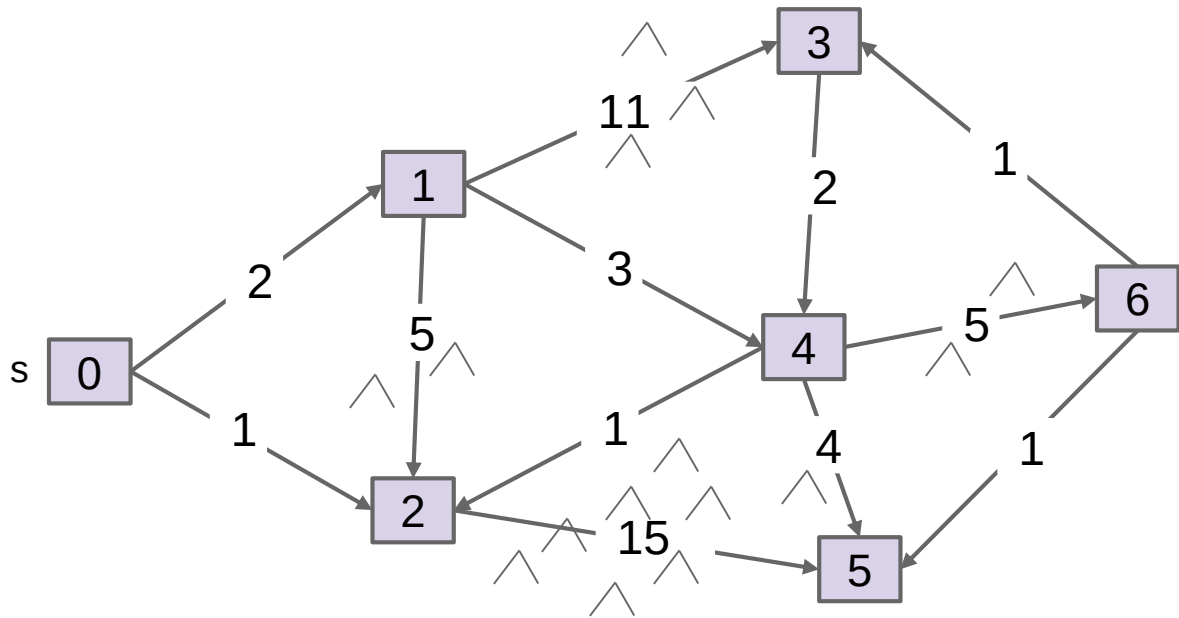
v	distTo[]	edgeTo[]
0	0.0	-
1	2.0	0→1
2	-	-
3	-	-
4	5.0	1→4
5	9.0	4→5
6	-	-

Shortest path from $s=0$ to $t=5$

Observation: Solution will always be a **path with no cycles**
(assuming non-negative weights)

Single Source Shortest Paths

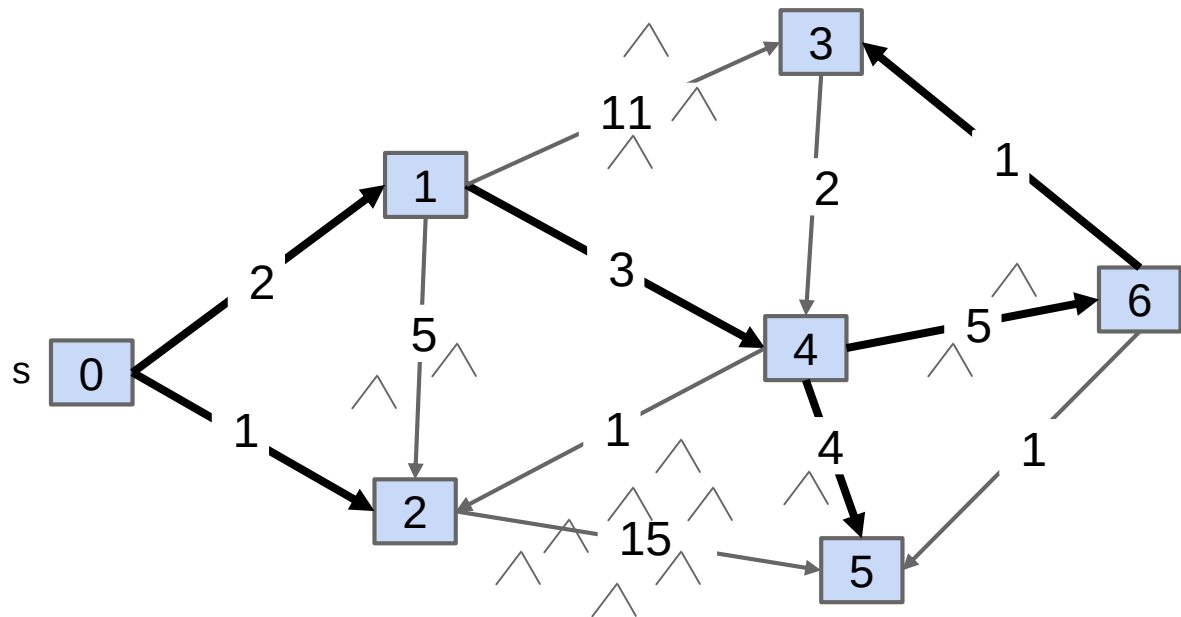
Goal: Find the shortest paths from source vertex s to every other vertex



Challenge: Try to write out the solution for this graph

Single Source Shortest Paths

Goal: Find the shortest paths from source vertex s to every other vertex



v	distTo[]	edgeTo[]
0	0.0	-
1	2.0	0→1
2	1.0	0→2
3	11.0	6→3
4	5.0	1→4
5	9.0	4→5
6	10.0	4→6

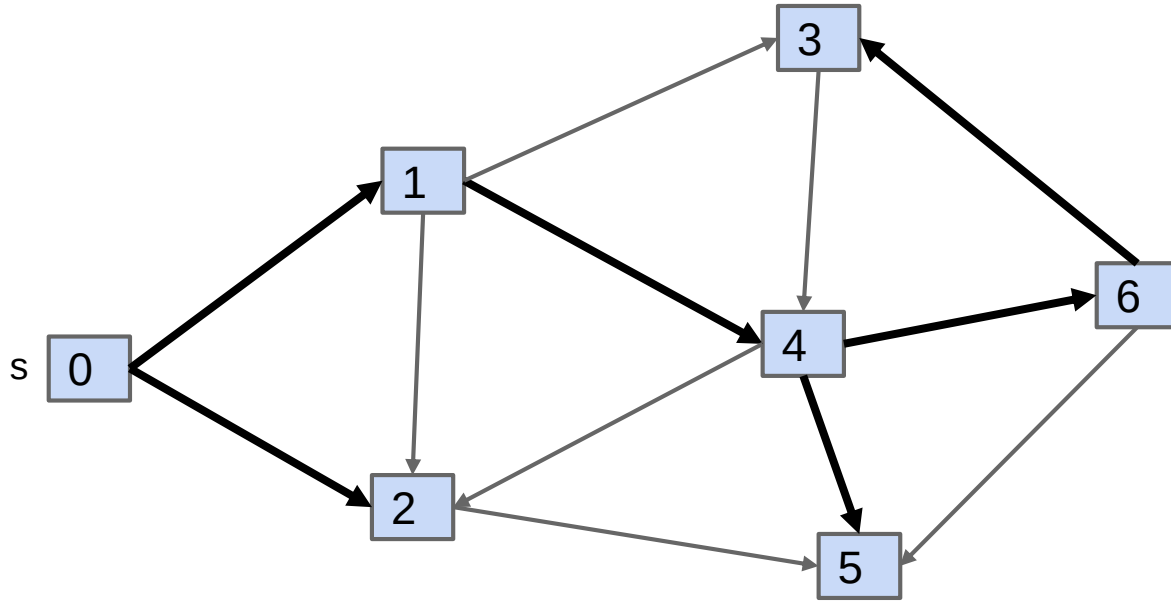
Shortest paths from $s=0$

Observation: Solution will always be a **tree**

- Can think of as the union of the shortest paths to all vertices

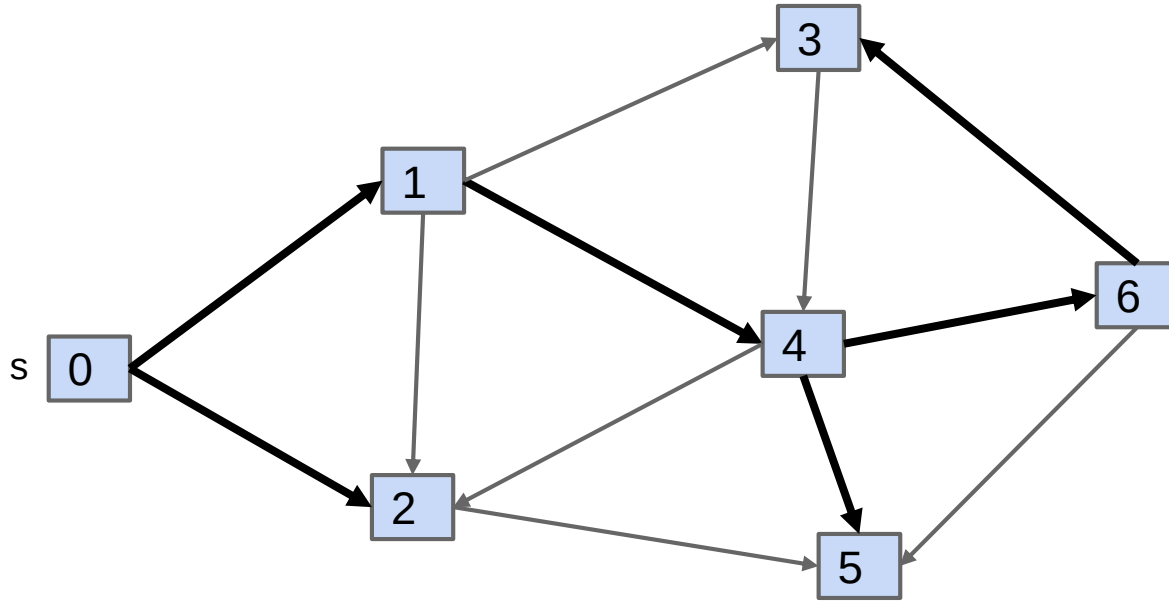
SPT Edge Count

If G is a connected edge-weighted graph with V vertices and E edges, how many edges are in the **Shortest Paths Tree** (SPT) of G ? [assume every vertex is reachable]



SPT Edge Count

If G is a connected edge-weighted graph with V vertices and E edges, how many edges are in the **Shortest Paths Tree** (SPT) of G ? [assume every vertex is reachable]



$V: 7$

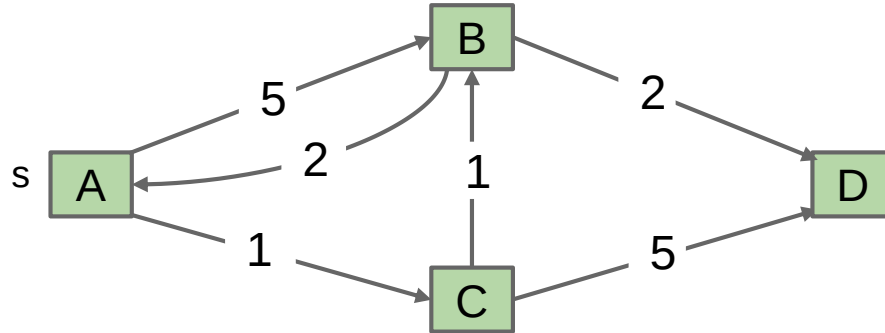
Number of edges in SPT
is 6

Always $V-1$:

- For each vertex, there is exactly one input edge (except source)

Finding a Shortest Paths Tree

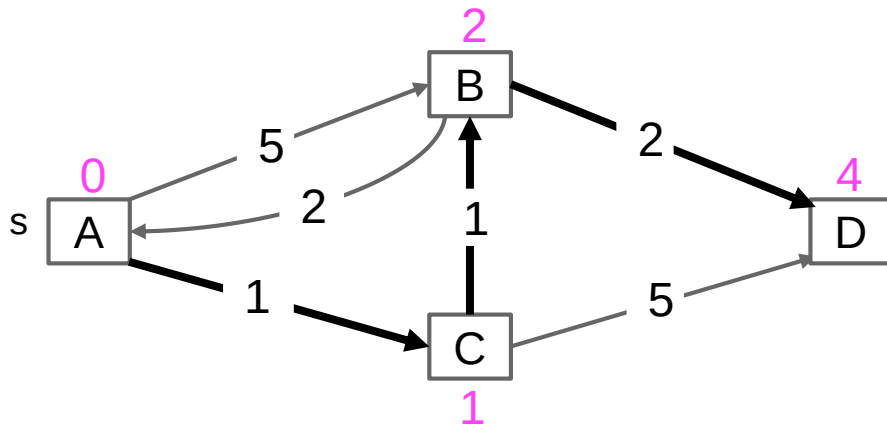
What is the shortest paths tree for the graph below?



Finding a Shortest Paths Tree

What is the shortest paths tree for the graph below?

- annotation in magenta shows the **total distance** from the source

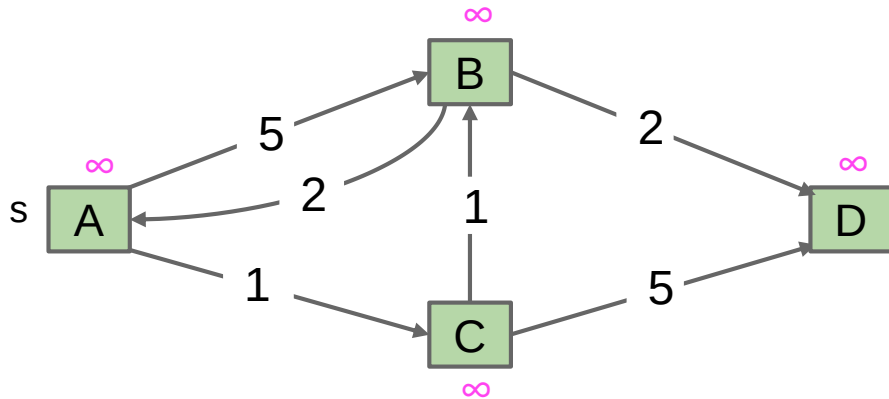


Creating an Algorithm

Let's create an algorithm for finding the shortest paths!

Will start with a buggy algorithm and then successively improve it

- algorithm begins in state below: all vertices unmarked; all distances infinite; no edges in the SPT



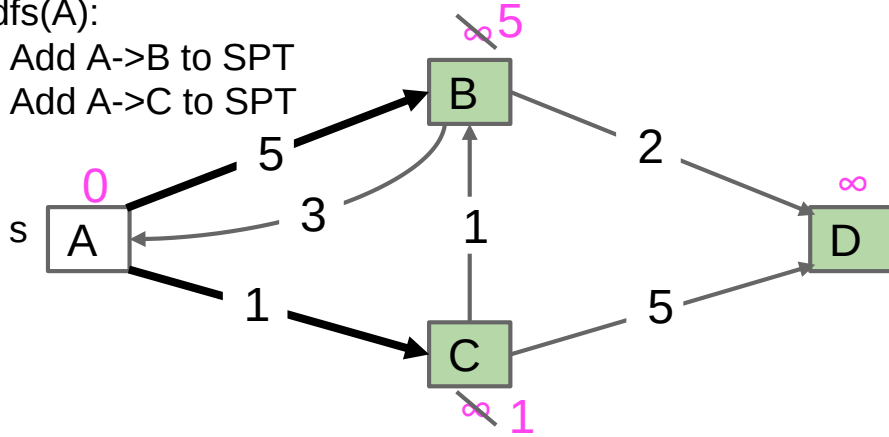
Finding a Shortest Paths Tree 1

Buggy algorithm #1: Perform a depth first search. When you visit v :

- For each edge from v to w , if w is not already part of SPT, add the edge

dfs(A):

Add A→B to SPT
Add A→C to SPT



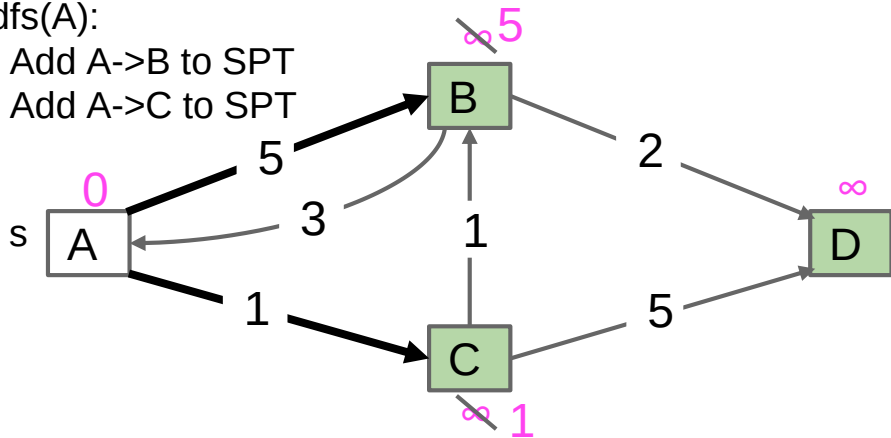
Finding a Shortest Paths Tree 1

Buggy algorithm #1: Perform a depth first search. When you visit v :

- For each edge from v to w , if w is not already part of SPT, add the edge

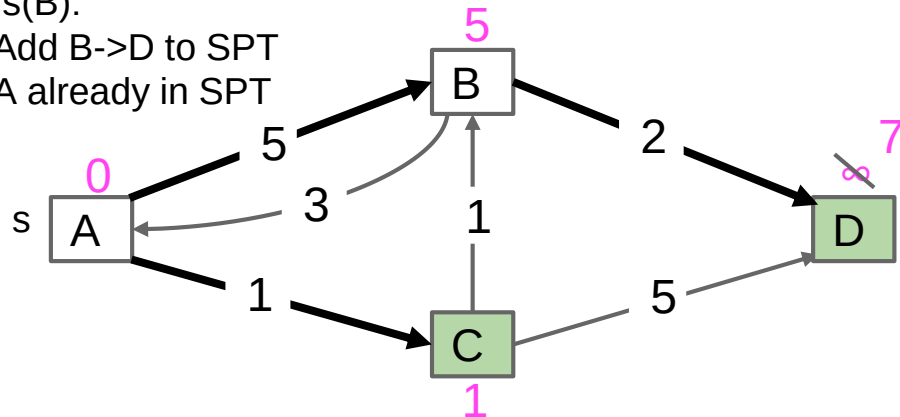
dfs(A):

Add A→B to SPT
Add A→C to SPT



dfs(B):

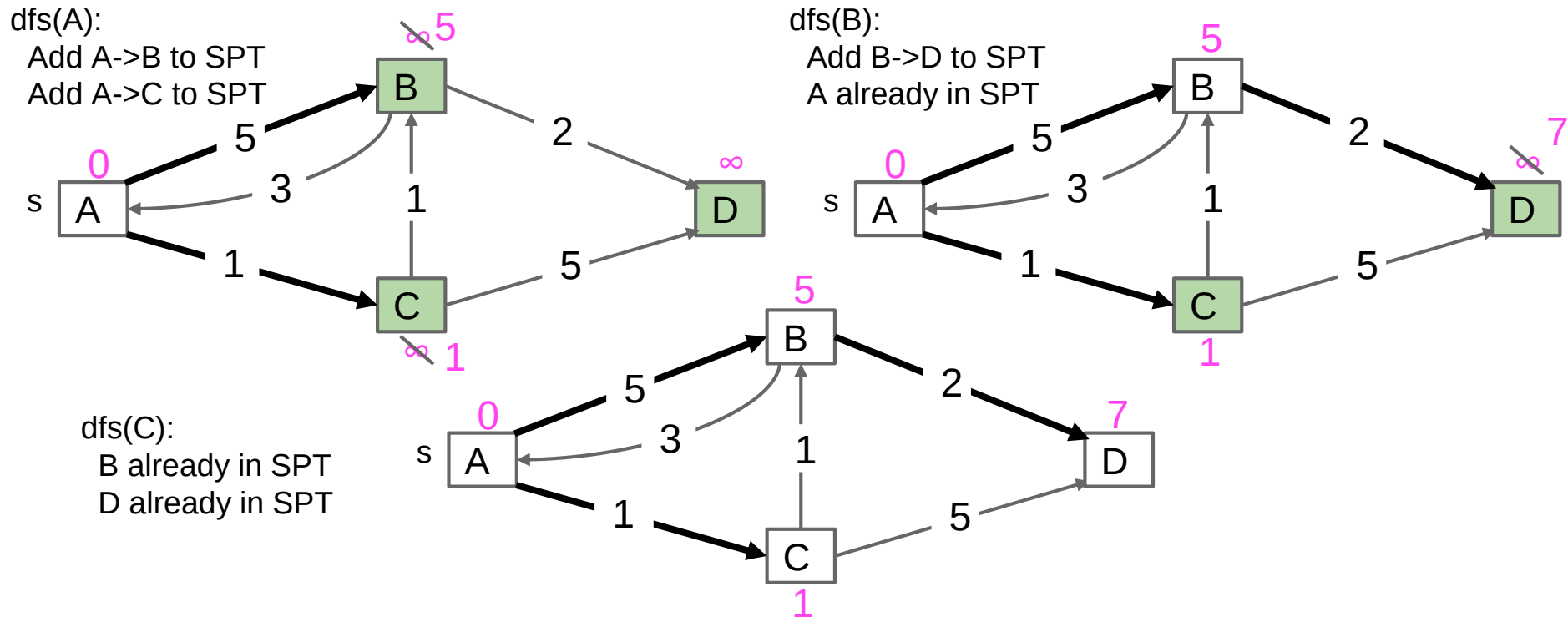
Add B→D to SPT
A already in SPT



Finding a Shortest Paths Tree 1

Buggy algorithm #1: Perform a depth first search. When you visit v :

- For each edge from v to w , if w is not already part of SPT, add the edge



Finding a Shortest Paths Tree 2

We'll call this process **edge relaxation**

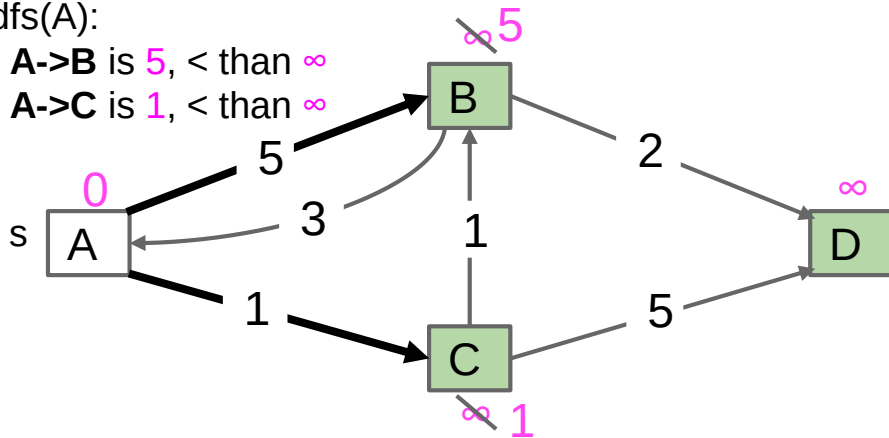
Buggy algorithm #2: Perform a depth first search. When you visit v :

- For each edge from v to w , add edge to the SPT ***only if that edge yields better distance***

dfs(A):

A $\rightarrow$ B is 5, < than ∞

A→C is 1, < than ∞



Finding a Shortest Paths Tree 2

Buggy algorithm #2: Perform a depth first search. When you visit v:

- For each edge from v to w, add edge to the SPT **only if that edge yields better distance**

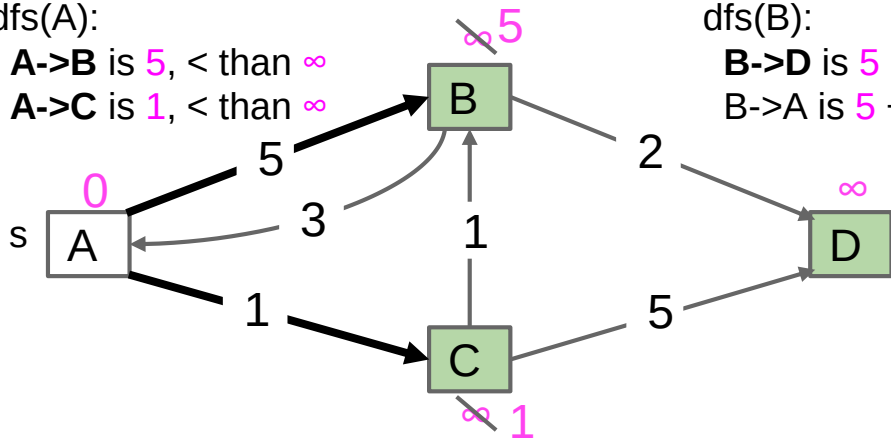
We'll call this process edge relaxation



dfs(A):

A → B is 5, < than ∞

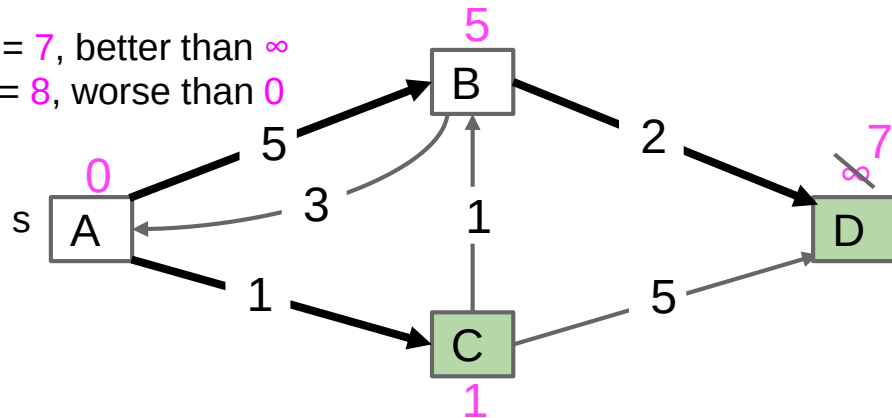
A → C is 1, < than ∞



dfs(B):

B → D is $5 + 2 = 7$, better than ∞

B → A is $5 + 3 = 8$, worse than 0



Finding a Shortest Paths Tree 2

We'll call this process edge relaxation

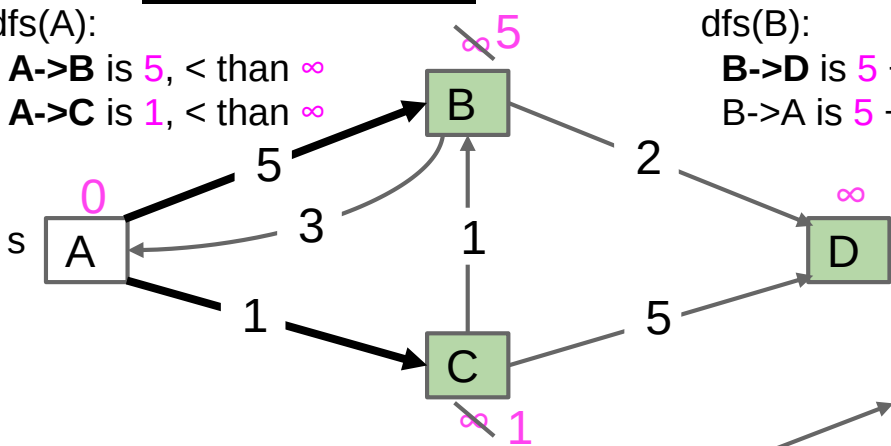
Buggy algorithm #2: Perform a depth first search. When you visit v:

- For each edge from v to w, add edge to the SPT **only if that edge yields better distance**

dfs(A):

A→B is 5, < than ∞

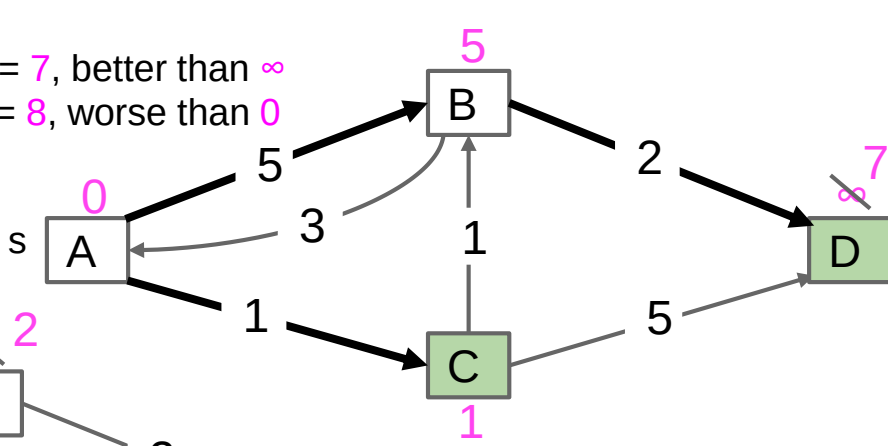
A→C is 1, < than ∞



dfs(B):

B→D is 5 + 2 = 7, better than ∞

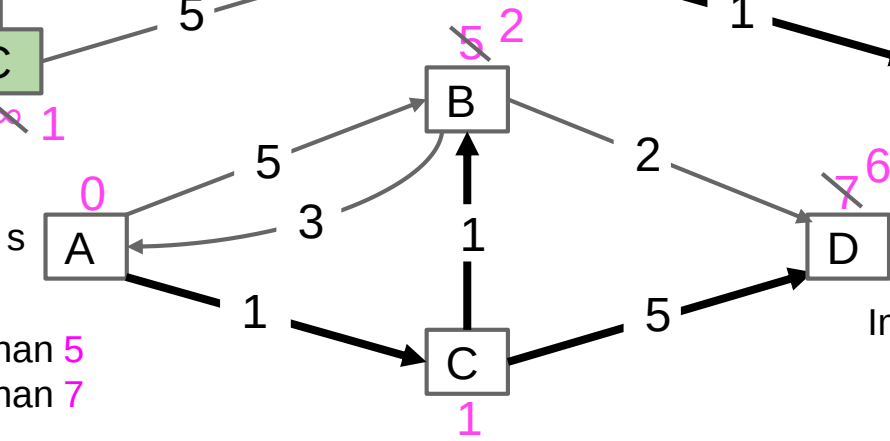
B→A is 5 + 3 = 8, worse than 0



dfs(C):

C→B is 1 + 1 = 2, better than 5

C→D is 1 + 5 = 6, better than 7



Improvements:

- use better edges if found

Finding a Shortest Paths Tree 3

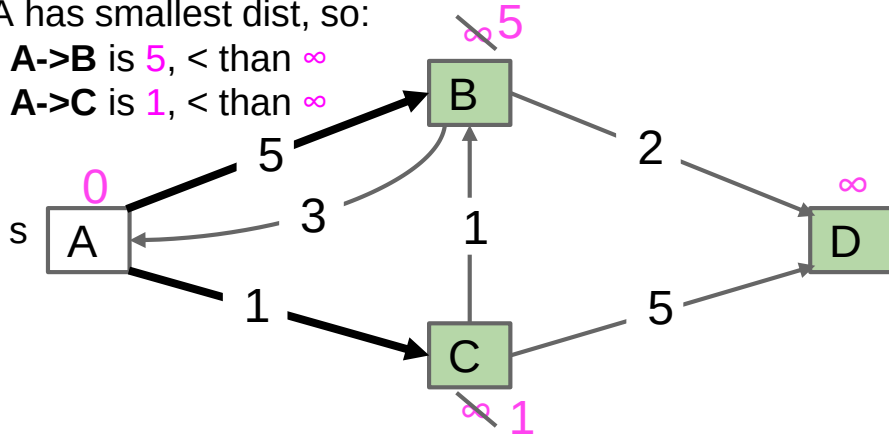
Dijkstra's Algorithm: Perform a **smallest distance first search**. When you visit v :

- For each from v to w , **relax that edge**

A has smallest dist, so:

A->**B** is 5, < than ∞

A->**C** is 1, < than ∞



Finding a Shortest Paths Tree 3

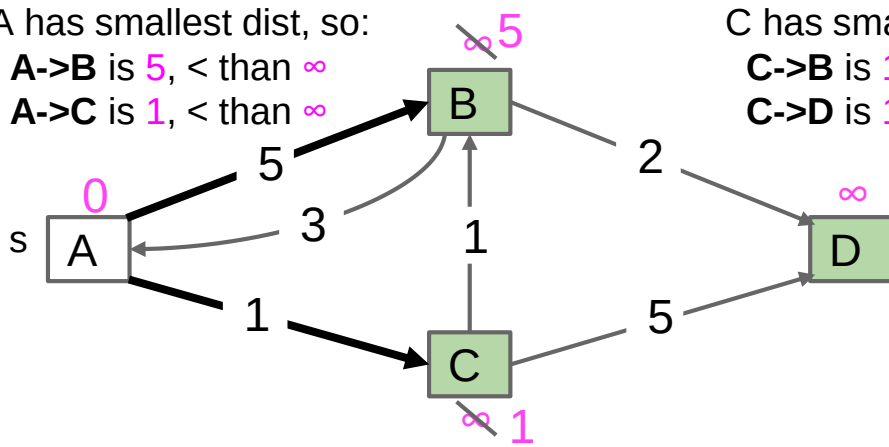
Dijkstra's Algorithm: Perform a **smallest distance first search**. When you visit v :

- For each from v to w , **relax that edge**

A has smallest dist, so:

$A \rightarrow B$ is 5, < than ∞

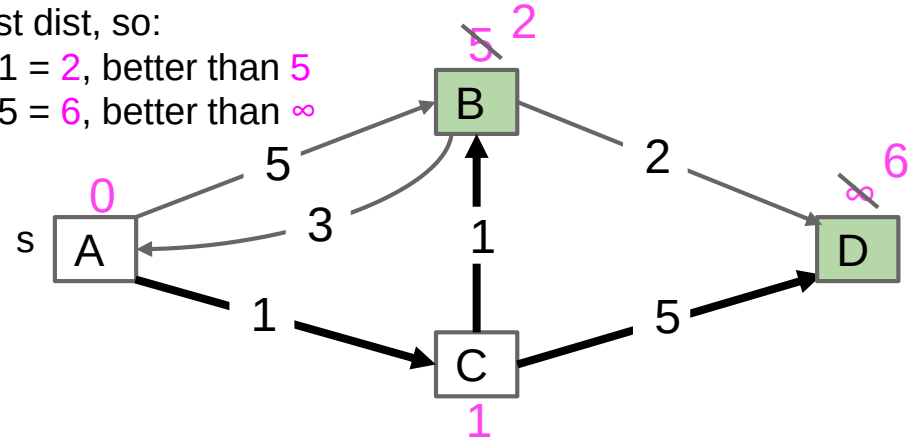
$A \rightarrow C$ is 1, < than ∞



C has smallest dist, so:

$C \rightarrow B$ is $1 + 1 = 2$, better than 5

$C \rightarrow D$ is $1 + 5 = 6$, better than ∞



Finding a Shortest Paths Tree 3

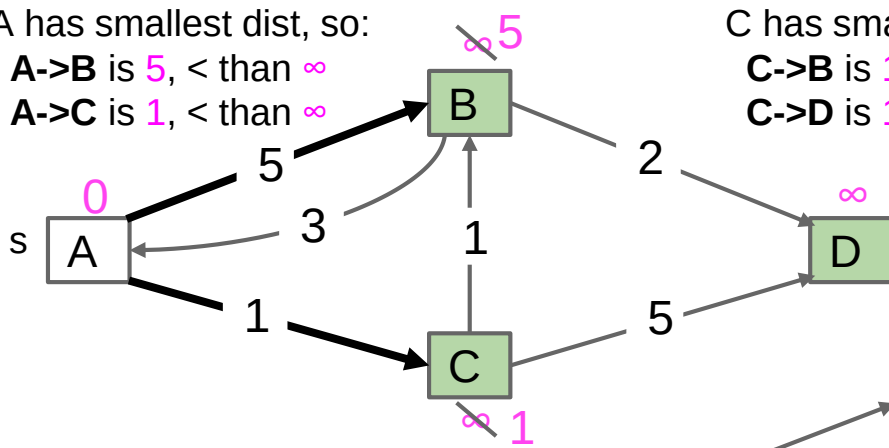
Dijkstra's Algorithm: Perform a **smallest distance first search**. When you visit v :

- For each from v to w , **relax that edge**

A has smallest dist, so:

$A \rightarrow B$ is 5, < than ∞

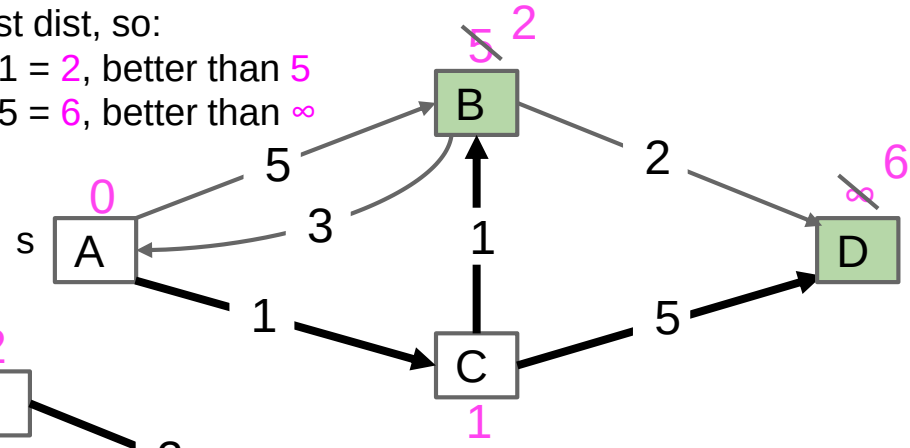
$A \rightarrow C$ is 1, < than ∞



C has smallest dist, so:

$C \rightarrow B$ is $1 + 1 = 2$, better than 5

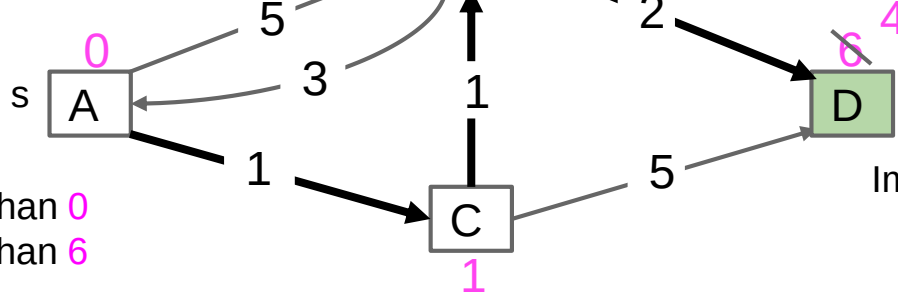
$C \rightarrow D$ is $1 + 5 = 6$, better than ∞



B has smallest dist, so:

$B \rightarrow A$ is $2 + 3 = 5$, worse than 0

$B \rightarrow D$ is $2 + 2 = 4$, better than 6



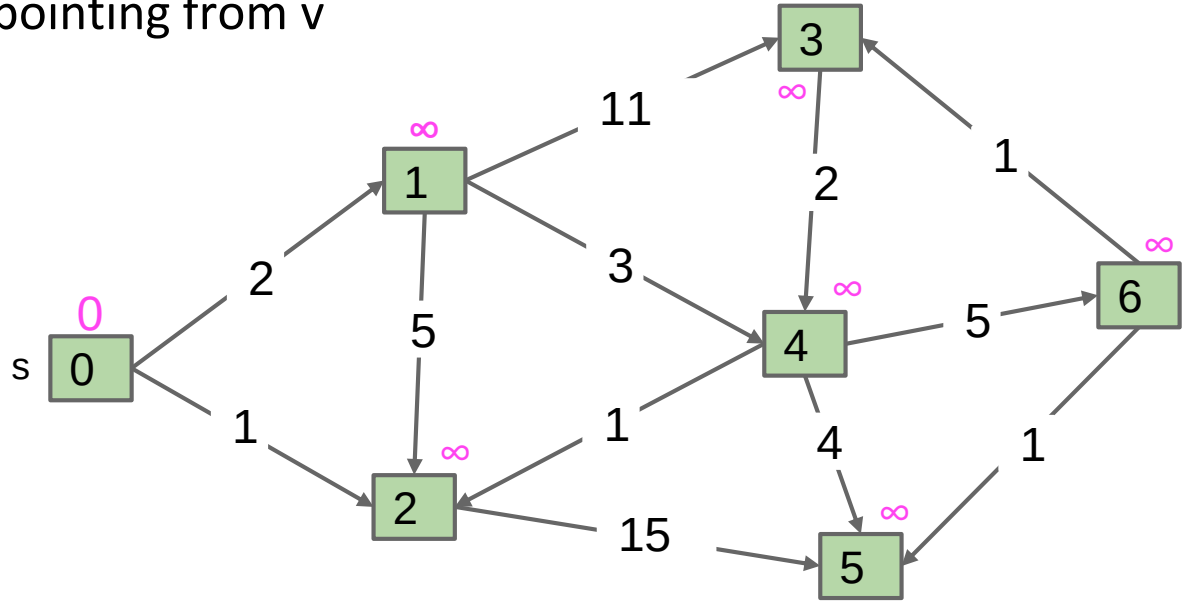
Improvements:

- use better edges if found
- traverse smallest dist first

Dijkstra's Algorithm

Insert all vertices into PQ, storing vertices in order of distance from source

Repeat: Remove (closest/smallest dist) vertex v from PQ
and relax all edges pointing from v

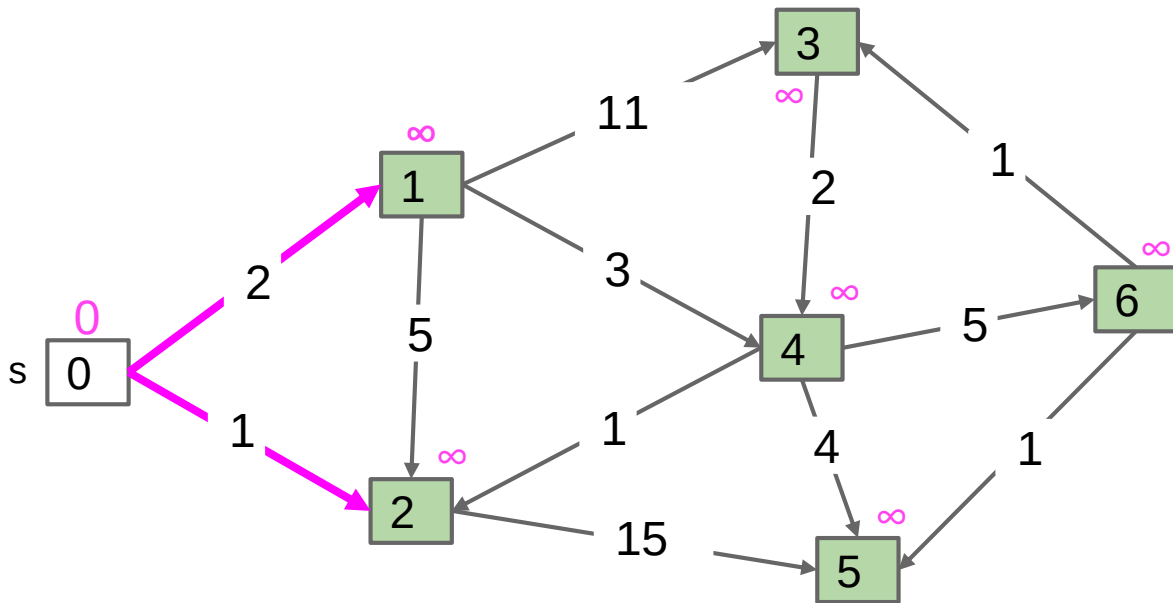


Dijkstra's Demo

Insert all vertices into PQ, storing vertices in order of distance from source

Repeat: Remove (closest) vertex v from PQ, and relax all edges pointing from v

#	distTo	edgeTo
0	0	-
1	∞	-
2	∞	-
3	∞	-
4	∞	-
5	∞	-
6	∞	-



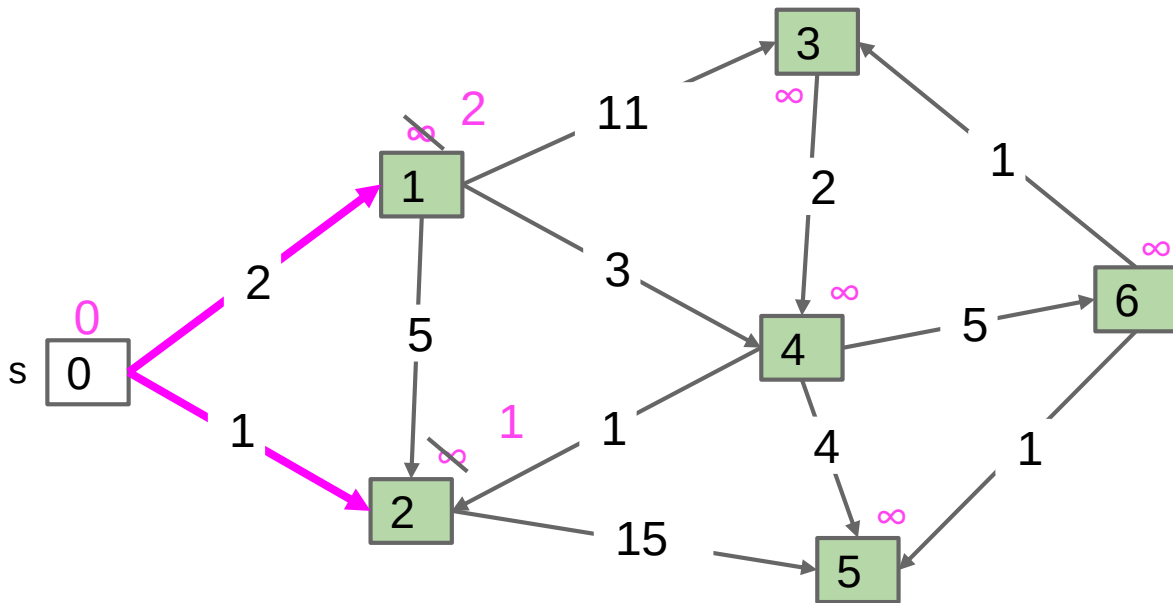
PQ: [(1: ∞), (2: ∞), (3: ∞), (4: ∞), (5: ∞), (6: ∞)]

Dijkstra's Demo

Insert all vertices into PQ, storing vertices in order of distance from source

Repeat: Remove (closest) vertex v from PQ, and relax all edges pointing from v

#	distTo	edgeTo
0	0	-
1	2	0
2	1	0
3	∞	-
4	∞	-
5	∞	-
6	∞	-



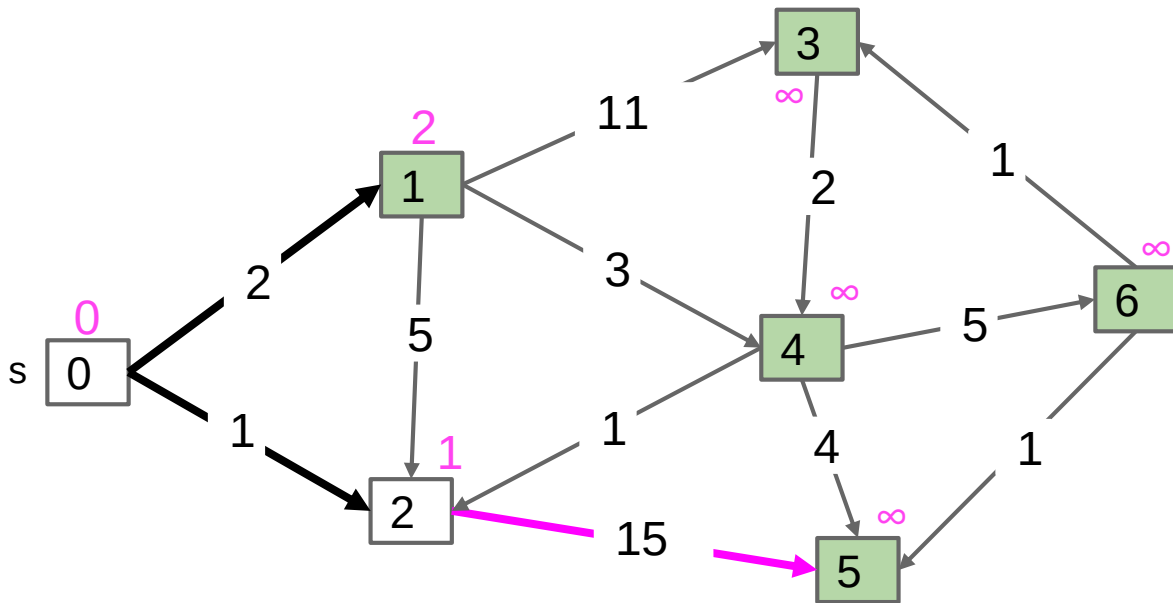
PQ: [(2: 1), (1: 2), (3: ∞), (4: ∞), (5: ∞), (6: ∞)]

Dijkstra's Demo

Insert all vertices into PQ, storing vertices in order of distance from source

Repeat: Remove (closest) vertex v from PQ, and relax all edges pointing from v

#	distTo	edgeTo
0	0	-
1	2	0
2	1	0
3	∞	-
4	∞	-
5	∞	-
6	∞	-



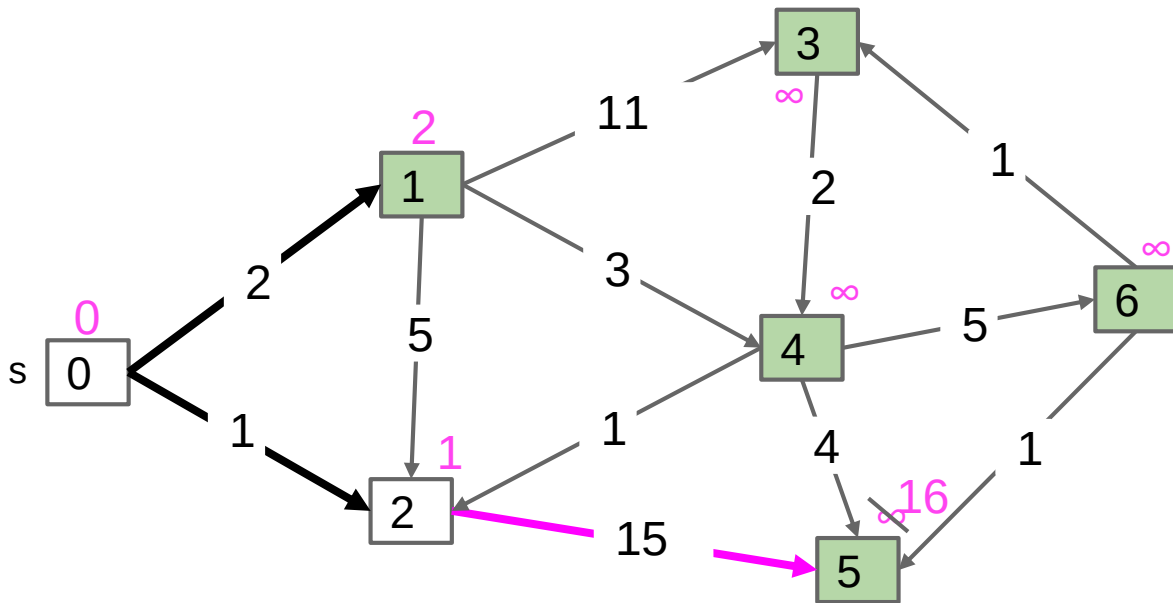
PQ: [(1: 2), (3: ∞), (4: ∞), (5: ∞), (6: ∞)]

Dijkstra's Demo

Insert all vertices into PQ, storing vertices in order of distance from source

Repeat: Remove (closest) vertex v from PQ, and relax all edges pointing from v

#	distTo	edgeTo
0	0	-
1	2	0
2	1	0
3	∞	-
4	∞	-
5	16	2
6	∞	-



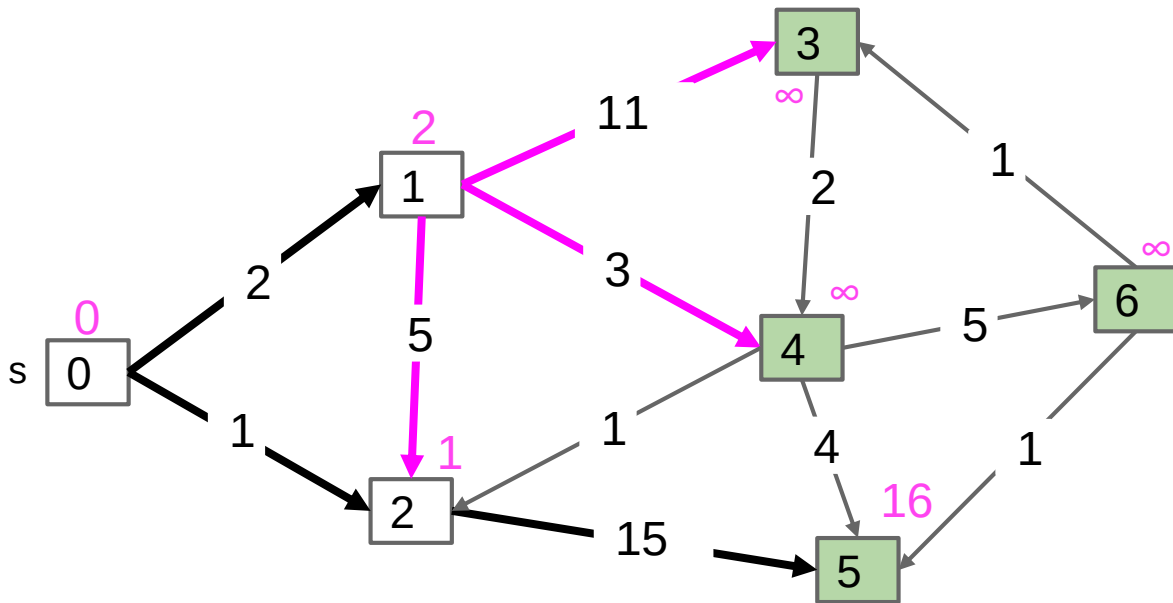
PQ: [(1: 2), (5: 16), (3: ∞), (4: ∞), (6: ∞)]

Dijkstra's Demo

Insert all vertices into PQ, storing vertices in order of distance from source

Repeat: Remove (closest) vertex v from PQ, and relax all edges pointing from v

#	distTo	edgeTo
0	0	-
1	2	0
2	1	0
3	∞	-
4	∞	-
5	16	2
6	∞	-



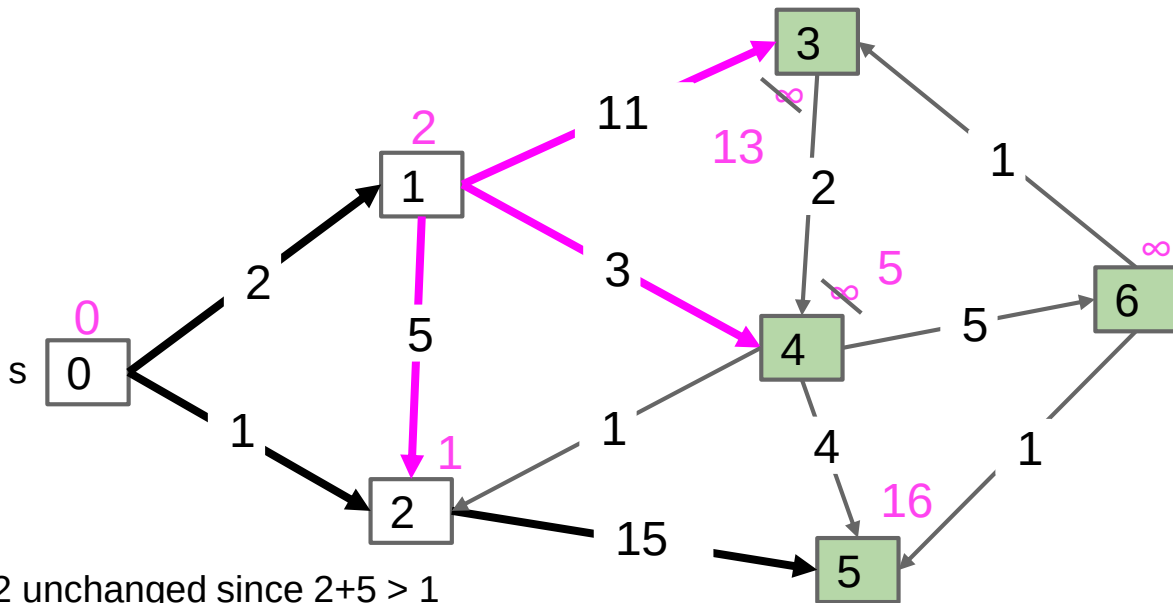
PQ: [(5: 16), (3: ∞), (4: ∞), (6: ∞)]

Dijkstra's Demo

Insert all vertices into PQ, storing vertices in order of distance from source

Repeat: Remove (closest) vertex v from PQ, and relax all edges pointing from v

#	distTo	edgeTo
0	0	-
1	2	0
2	1	0
3	13	1
4	5	1
5	16	2
6	∞	-



Vertex 2 unchanged since $2+5 > 1$

PQ: [(4: 5), (3: 13), (5: 16), (6: ∞)]

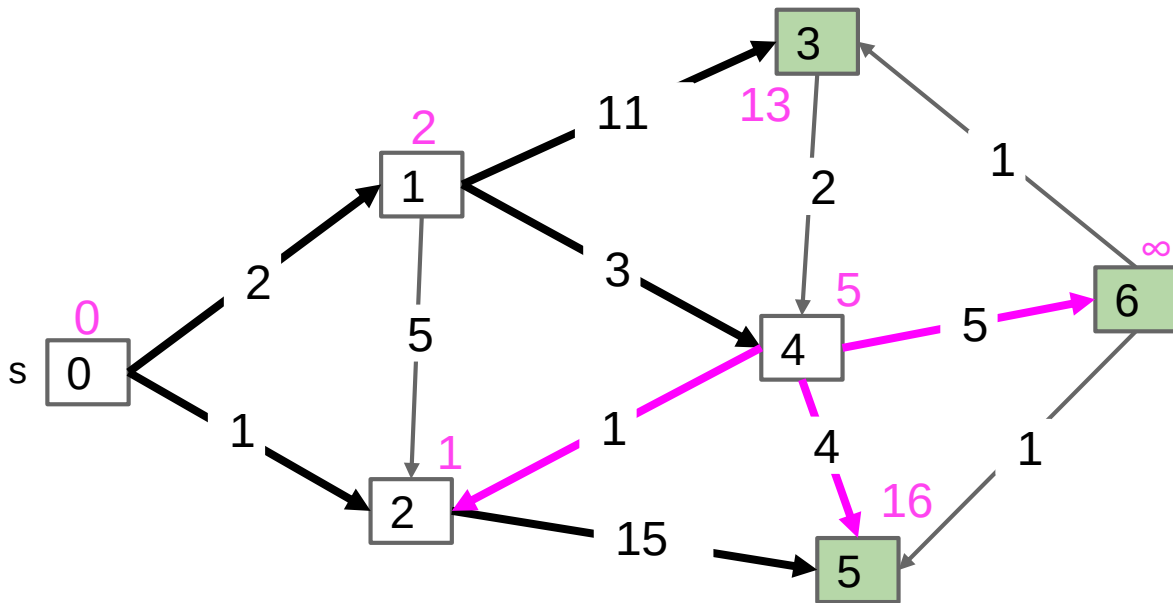
Which vertex is removed next?

Dijkstra's Demo

Insert all vertices into PQ, storing vertices in order of distance from source

Repeat: Remove (closest) vertex v from PQ, and relax all edges pointing from v

#	distTo	edgeTo
0	0	-
1	2	0
2	1	0
3	13	1
4	5	1
5	16	2
6	∞	-



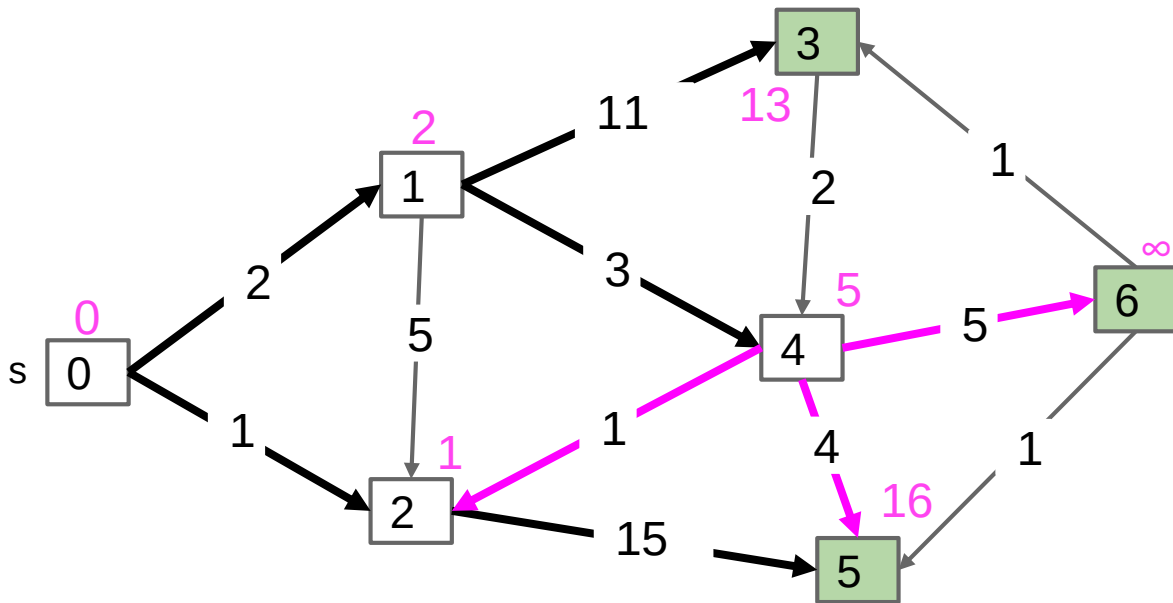
PQ: [(3: 13), (5: 16), (6: ∞)]

Dijkstra's Demo

Insert all vertices into PQ, storing vertices in order of distance from source

Repeat: Remove (closest) vertex v from PQ, and relax all edges pointing from v

#	distTo	edgeTo
0	0	-
1	2	0
2	1	0
3	13	1
4	5	1
5	16	2
6	∞	-



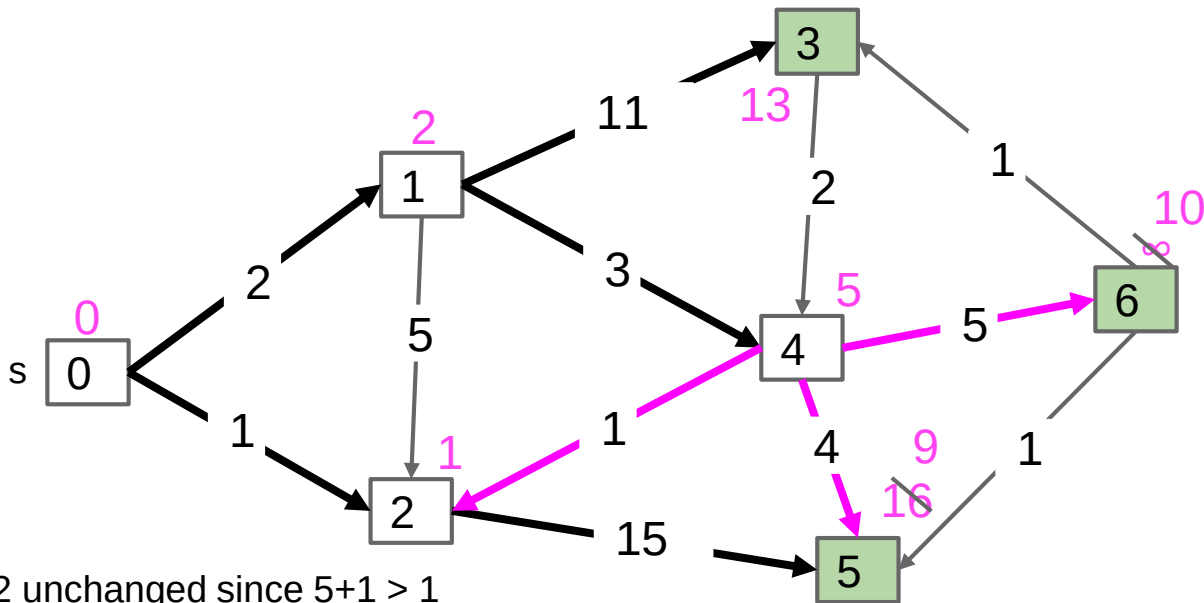
PQ: [(3: 13), (5: 16), (6: ∞)]

Dijkstra's Demo

Insert all vertices into PQ, storing vertices in order of distance from source

Repeat: Remove (closest) vertex v from PQ, and relax all edges pointing from v

#	distTo	edgeTo
0	0	-
1	2	0
2	1	0
3	13	1
4	5	1
5	9	4
6	10	4



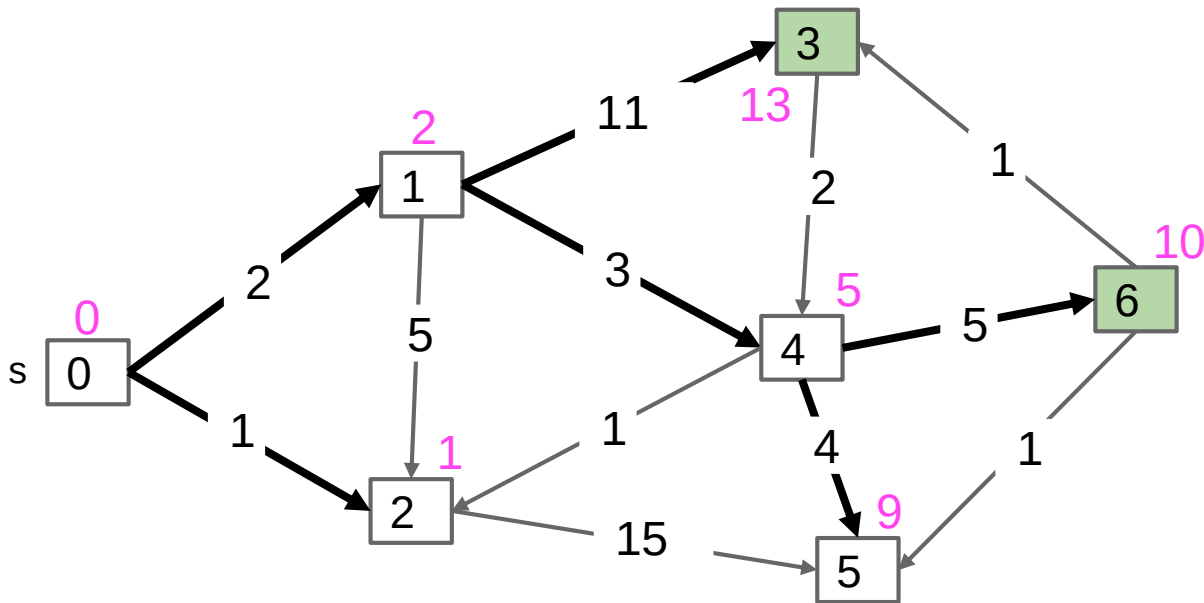
PQ: [(5: 9), (6: 10), (3: 13)]

Dijkstra's Demo

Insert all vertices into PQ, storing vertices in order of distance from source

Repeat: Remove (closest) vertex v from PQ, and relax all edges pointing from v

#	distTo	edgeTo
0	0	-
1	2	0
2	1	0
3	13	1
4	5	1
5	9	4
6	10	4



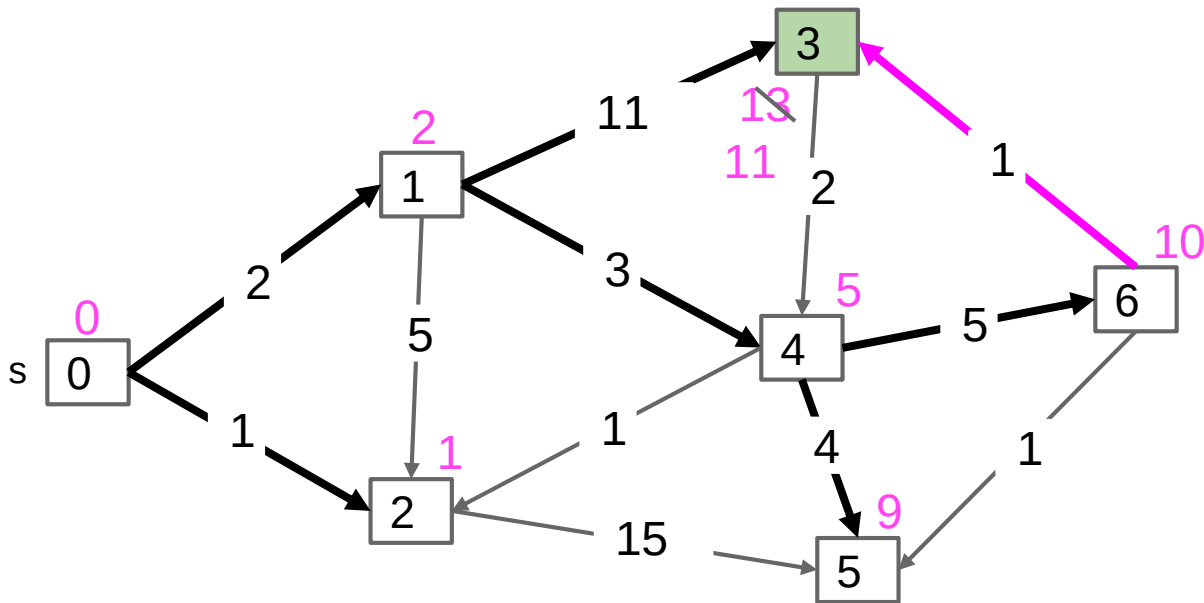
PQ: [(6: 10), (3: 13)]

Dijkstra's Demo

Insert all vertices into PQ, storing vertices in order of distance from source

Repeat: Remove (closest) vertex v from PQ, and relax all edges pointing from v

#	distTo	edgeTo
0	0	-
1	2	0
2	1	0
3	11	6
4	5	1
5	9	4
6	10	4



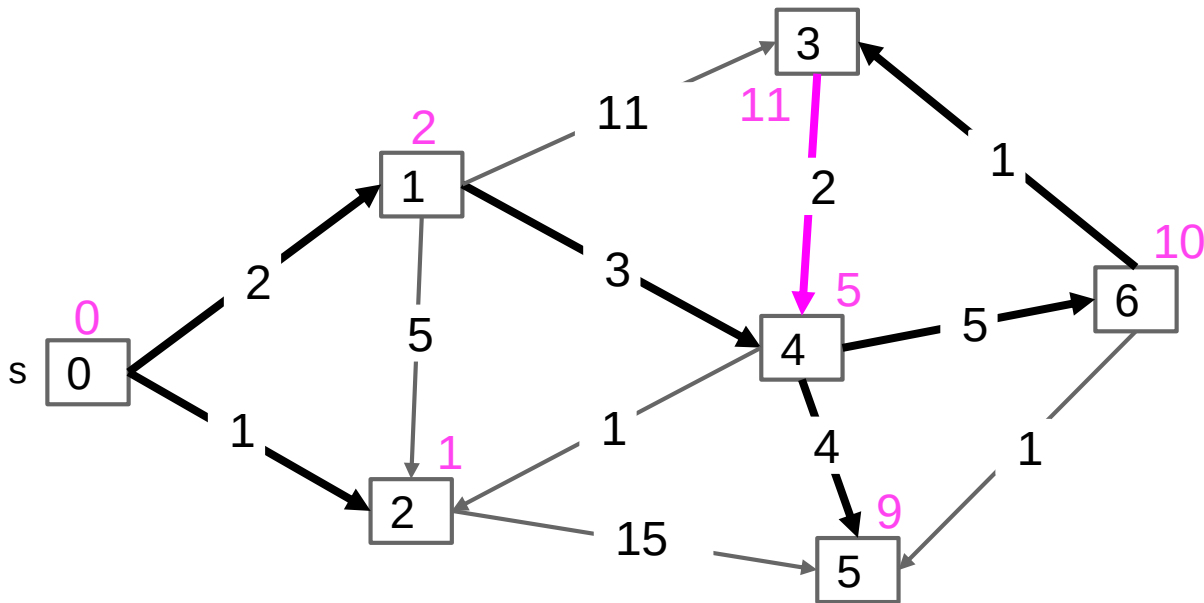
PQ: [(3: 11)]

Dijkstra's Demo

Insert all vertices into PQ, storing vertices in order of distance from source

Repeat: Remove (closest) vertex v from PQ, and relax all edges pointing from v

#	distTo	edgeTo
0	0	-
1	2	0
2	1	0
3	11	6
4	5	1
5	9	4
6	10	4



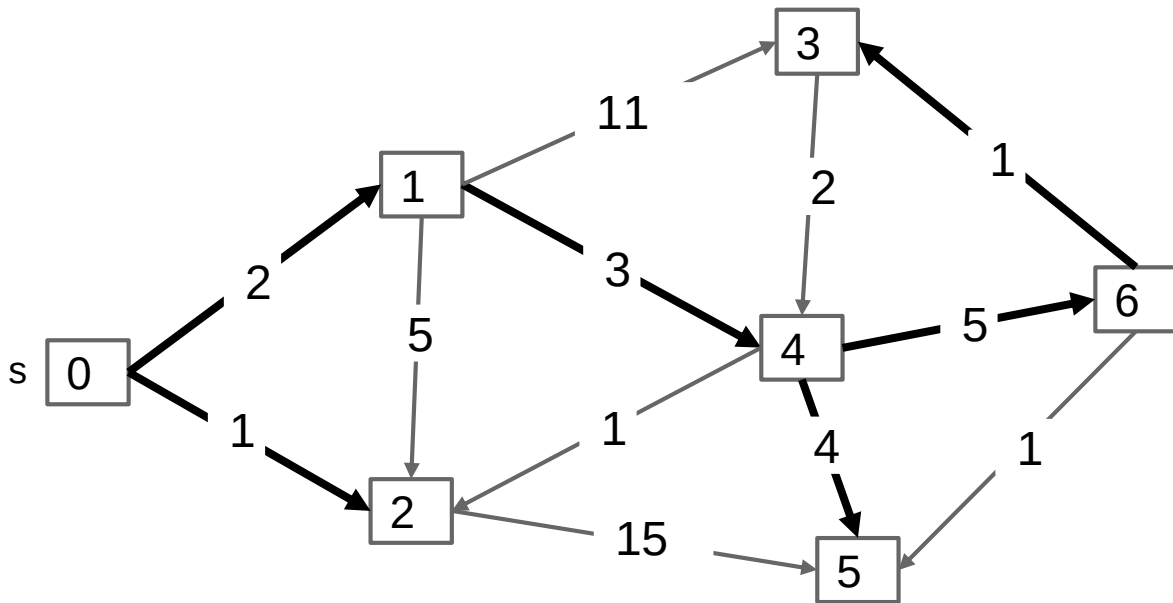
PQ: []

Dijkstra's Demo

Insert all vertices into PQ, storing vertices in order of distance from source

Repeat: Remove (closest) vertex v from PQ, and relax all edges pointing from v

#	distTo	edgeTo
0	0	-
1	2	0
2	1	0
3	11	6
4	5	1
5	9	4
6	10	4



PQ: []

Dijkstra's Algorithm Pseudocode

Dijkstra's:

- PQ.add(source, 0)
- For other vertices v , PQ.add(v , infinity)
- While PQ is not empty:
 - $p = \text{PQ.delMin}()$
 - Relax all edges from p

Relaxing an edge $p \rightarrow q$ with weight w :

- If $\text{distTo}[p] + w < \text{distTo}[q]$:
 - $\text{distTo}[q] = \text{distTo}[p] + w$
 - $\text{edgeTo}[q] = p$
 - PQ.decreaseKey(q , $\text{distTo}[q]$)

Dijkstra's Algorithm Pseudocode

Dijkstra's:

- `PQ.add(source, 0)`
- For other vertices v , `PQ.add(v, infinity)`
- While PQ is not empty:
 - `p = PQ.delMin()`
 - Relax all edges from p

Relaxing an edge $p \rightarrow q$ with weight w :

- If $\text{distTo}[p] + w < \text{distTo}[q]$:
 - $\text{distTo}[q] = \text{distTo}[p] + w$
 - $\text{edgeTo}[q] = p$
 - `PQ.decreaseKey(q, distTo[q])`

Key invariants:

- $\text{edgeTo}[v]$ is the best known predecessor of v
- $\text{distTo}[v]$ is the best known total distance from source to v
- PQ contains all unvisited vertices in order of distTo

Important properties:

- Always visits vertices in order of total distance from source
- Relaxation always fails on edges to visited (white) vertices

Guaranteed Optimality

Dijkstra's Algorithm:

- Visit vertices in **order of best-known distance** from source
- On visit, ***relax*** every edge from the visited vertex

Dijkstra's is guaranteed to return a correct result if all edges are ***non-negative***

Guaranteed Optimality

Dijkstra's is guaranteed to be optimal so long as there are **no negative edges**

- Proof relies on the property that relaxation always fails on edges to visited (white) vertices

Proof sketch: Assume all edges have non-negative weights

- At start, $\text{distTo}[\text{source}] = 0$, which is optimal
- After relaxing all edges from source, let vertex v_1 be the vertex with minimum weight, i.e. that is closest to the source

Claim: $\text{distTo}[v_1]$ is optimal, and thus future relaxations will fail. Why?

- $\text{distTo}[p] \geq \text{distTo}[v_1]$ for all p , therefore
- $\text{distTo}[p] + w \geq \text{distTo}[v_1]$
- Can use induction to prove that this holds for all vertices after dequeuing

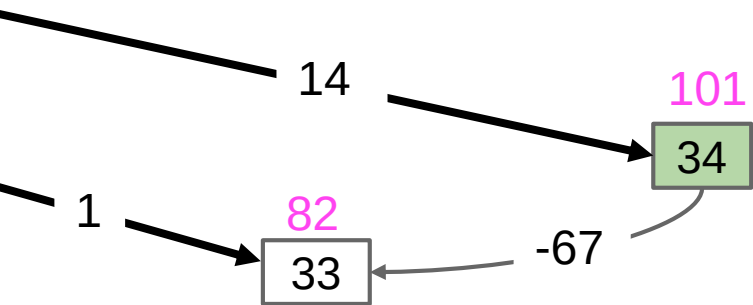
Negative Edges

Dijkstra's Algorithm:

- Visit vertices in **order of best-known distance** from source. On visit, *relax* every edge from the visited vertex.

Dijkstra's can fail if graph has negative weight edges. Why?

- Relaxation of already visited vertices can succeed



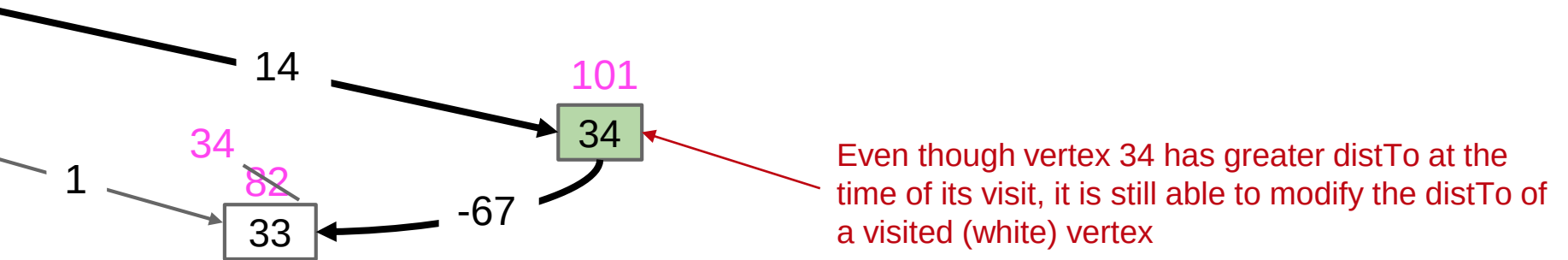
Negative Edges

Dijkstra's Algorithm:

- Visit vertices in **order of best-known distance** from source. On visit, *relax* every edge from the visited vertex.

Dijkstra's can fail if graph has negative weight edges. Why?

- Relaxation of already visited vertices can succeed



Need **Bellman-Ford algorithm** that runs in $O(V \cdot E)$

Dijkstra's Algorithm Runtime

Priority Queue operation count, assuming binary heap MinPQ (lecture 8):

- add: V , each costing $O(\log V)$ time
- delMin: V , each costing $O(\log V)$ time
- decreaseKey: E , each costing $O(\log V)$ time

Overall runtime: $O(V \cdot \log(V) + V \cdot \log(V) + E \cdot \log V)$

- Assuming $E > V$, this is just **$O(E \log V)$** for a connected graph

Operations	# Operations	Cost per operation	Total cost
PQ add	V	$O(\log V)$	$O(V \log V)$
PQ delMin	V	$O(\log V)$	$O(V \log V)$
PQ decreaseKey	E	$O(\log V)$	$O(E \log V)$

Binary Heap with Decrease Key Implementation

- To maintain the logarithmic running time operations for our binary heap MinPQ implementation (*lecture 8*), we can create an **indexed priority queue** implementation
 - Start with same code as MinPQ
 - Maintain parallel arrays `keys[ ]`, `pq[ ]`, and `qp[ ]` so that:
 - `keys[i]` is the priority of `i`
 - `pq[i]` is the index of the key in heap position `i`
 - `qp[i]` is the heap position of the key with index `i`
 - Use `swim(qp[i])` to implement **decreaseKey(i, key)**

Part 2: Minimum Spanning Trees

- In the second part of Lecture 10, we will discuss algorithms to find minimum spanning trees

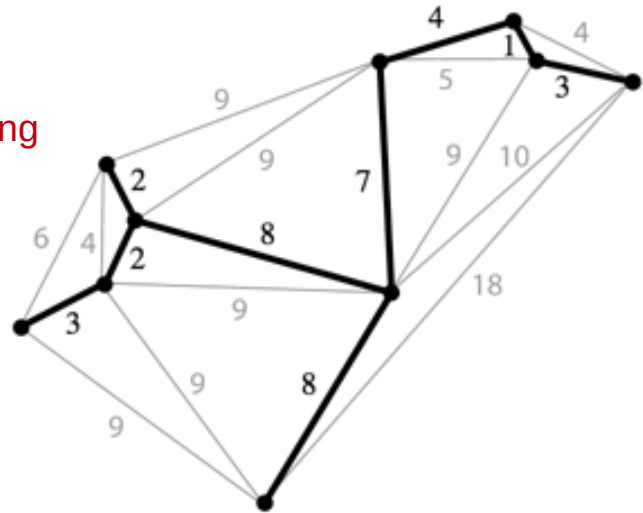
Spanning Trees

Given an **undirected** graph, a ***spanning tree*** T is a subgraph of G , where T :

- Is connected
 - Is acyclic
 - Includes all of the vertices
- These two properties make it a tree
- This makes it spanning

Example:

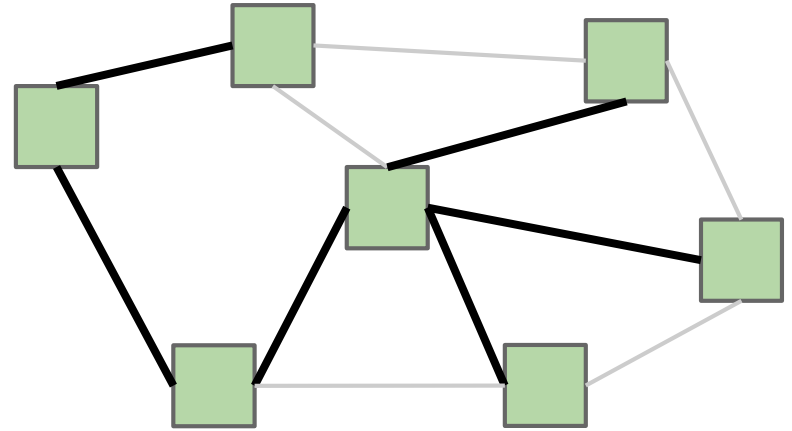
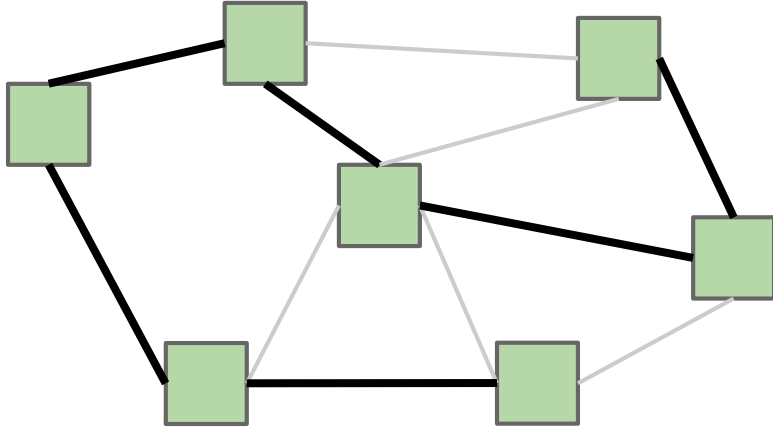
- Spanning tree is the black edges and vertices



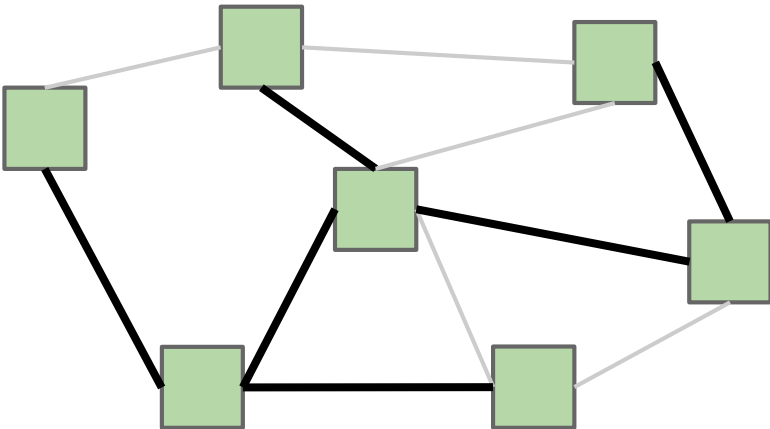
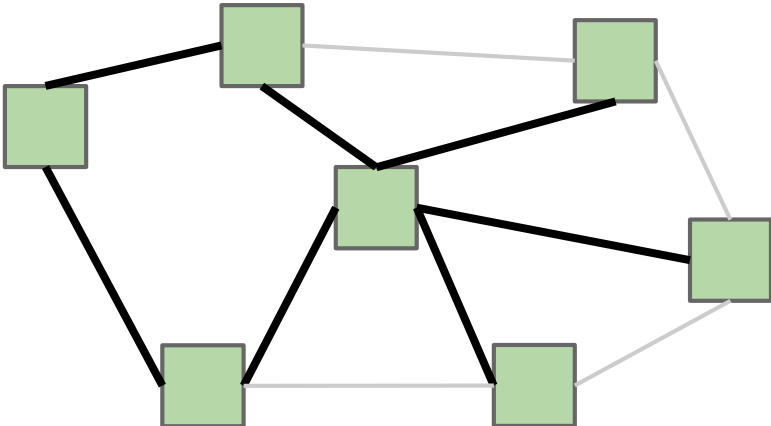
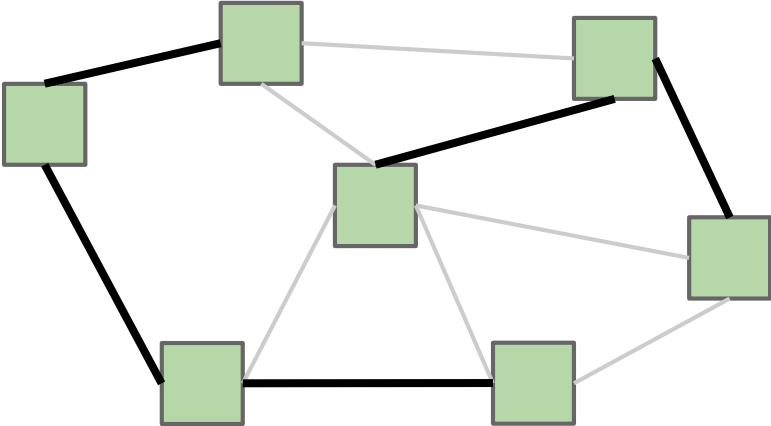
A ***minimum spanning tree*** is a spanning tree of minimum total weight

- example: directly connecting buildings by power lines

Spanning Trees Examples

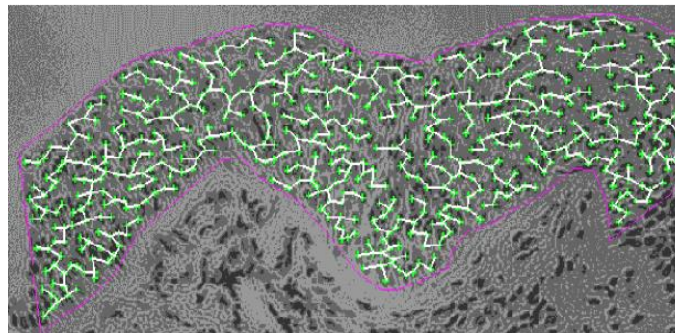
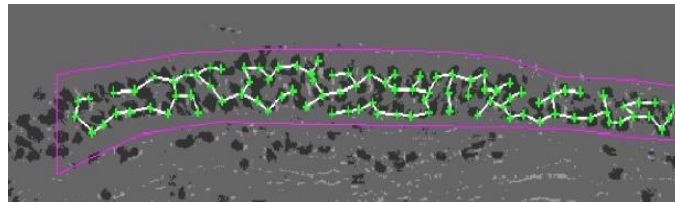
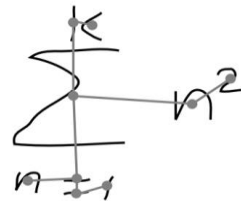


Which one(s) are Spanning Trees?



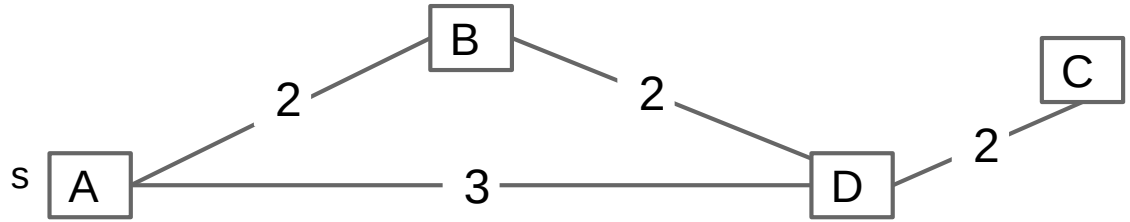
MST Applications

- Old school handwriting recognition
- Medical imaging (e.g. arrangement of nuclei in cancer cells)
- Max bottleneck paths
- Real-time face verification
- LDPC codes for error correction
- Image registration with Renyi entropy
- Find road networks in satellite and aerial imagery
- Reducing data storage in sequencing amino acids in a protein
- Model locality of particle interactions in turbulent fluid flows
- Autoconfig protocol for Ethernet bridging to avoid cycles in a network
- Approximation algorithms for NP-hard problems (e.g., TSP, Steiner tree)
- Network design (communication, electrical, hydraulic, computer, road)



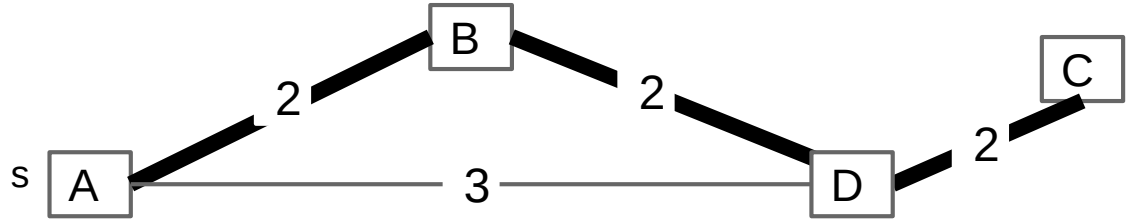
MST

Find the MST for the graph



MST

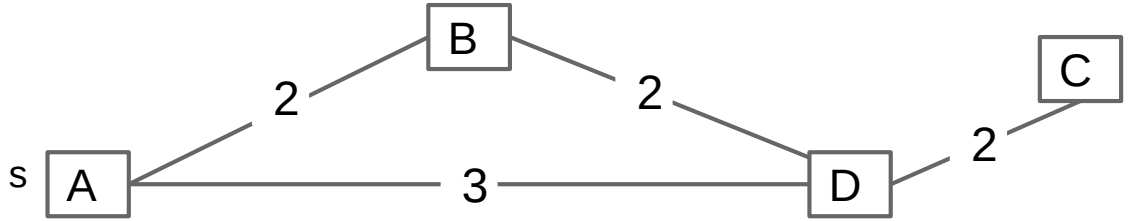
Find the MST for the graph.



MST vs. SPT

Is the MST for this graph also a shortest paths tree? If so, using which node as the starting node for this SPT?

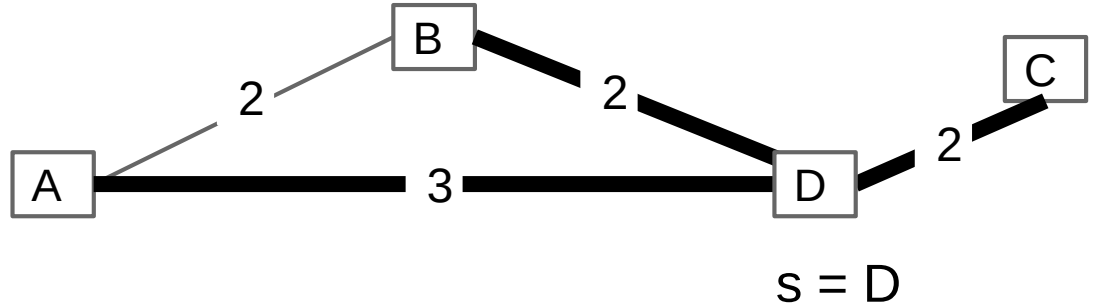
- A. A
- B. B
- C. C
- D. D
- E. No SPT is an MST



MST vs. SPT

Is the MST for this graph also a shortest paths tree? If so, using which node as the starting node for this SPT?

- A. A
- B. B
- C. C
- D. D
- E. No SPT is an MST



Doesn't work for $s = D$!

MST vs. SPT

Is the MST for this graph also a shortest paths tree? If so, using which node as the starting node for this SPT?

A. A

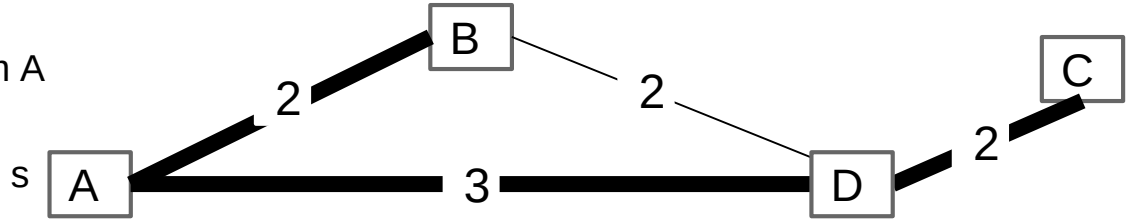
B. B

C. C

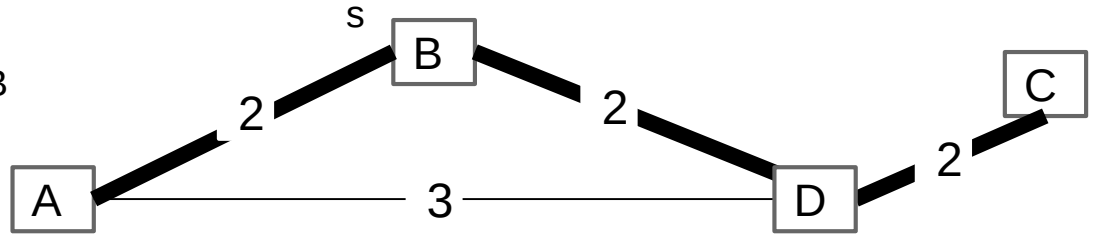
D. D

E. No SPT is an MST

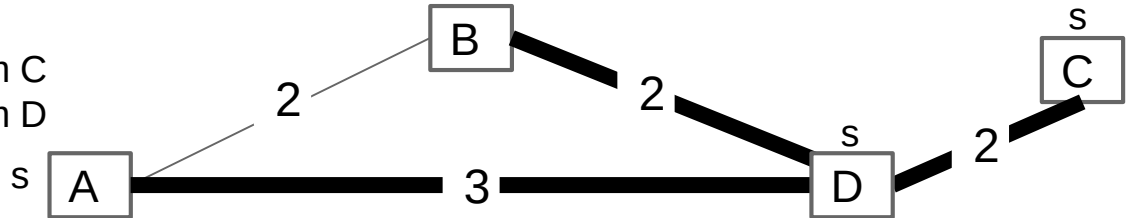
SPT from A



SPT from B



SPT from C
SPT from D



MST vs. SPT

A shortest paths tree depends on the start vertex:

- Because it tells you how to get from a source to EVERYTHING

There is ***no*** source for a MST

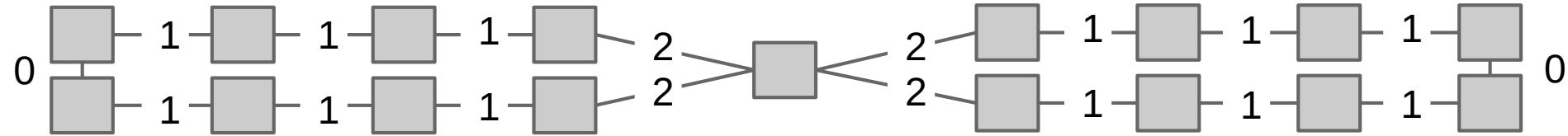
- It's a global property

Nonetheless, the MST sometimes happens to be an SPT for a specific vertex

MST vs. SPT 2

Give a valid MST for the graph below

- Is there a node whose SPT is also the MST?

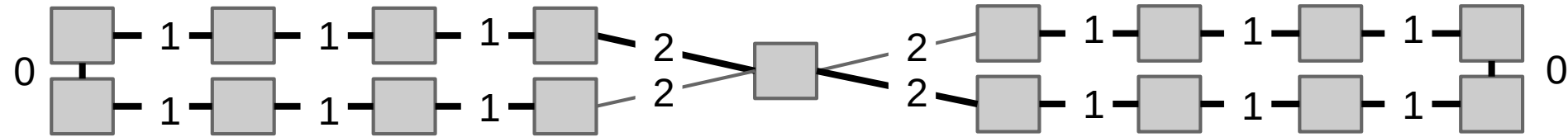


- A. Yes
- B. No

MST vs. SPT 2

Give a valid MST for the graph below

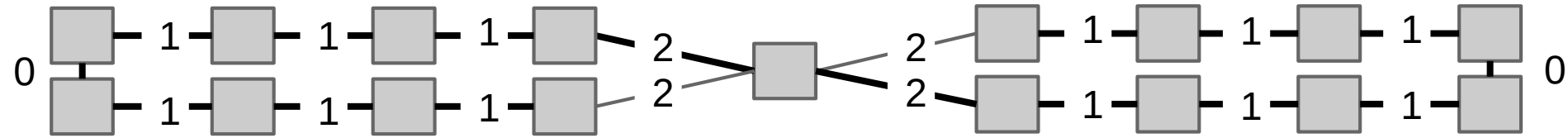
- Is there a node whose SPT is also the MST?
- Here is one of the MST (the other one is the mirror image)



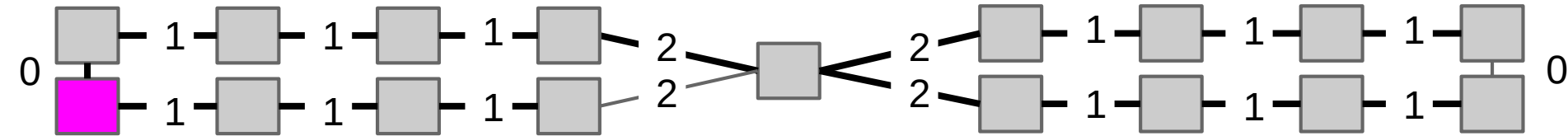
MST vs. SPT 2

Give a valid MST for the graph below.

- Is there a node whose SPT is also the MST?
- **No!** Minimum spanning tree must include only 2 of the 2 weight edges, but the SPT always includes at least 3 of the 2 weight edges.

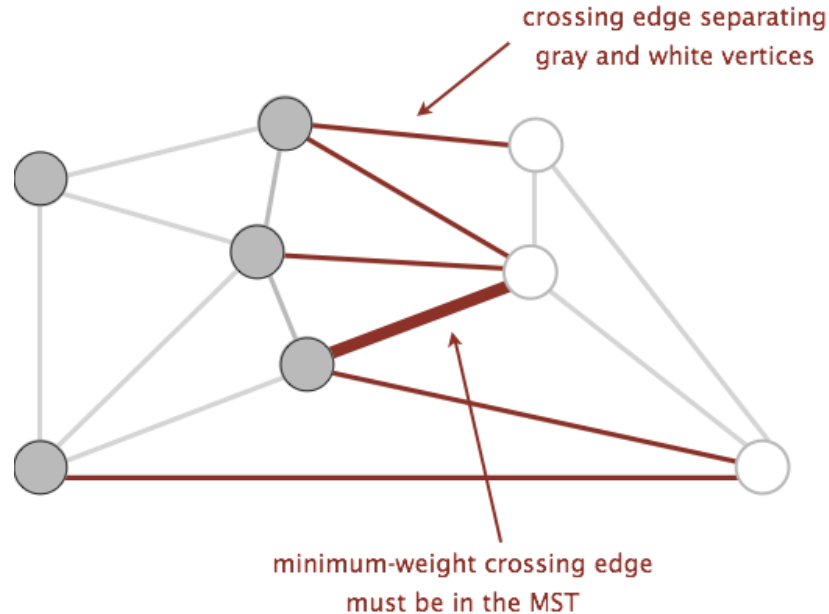


Example SPT from bottom left vertex:



Cut Property

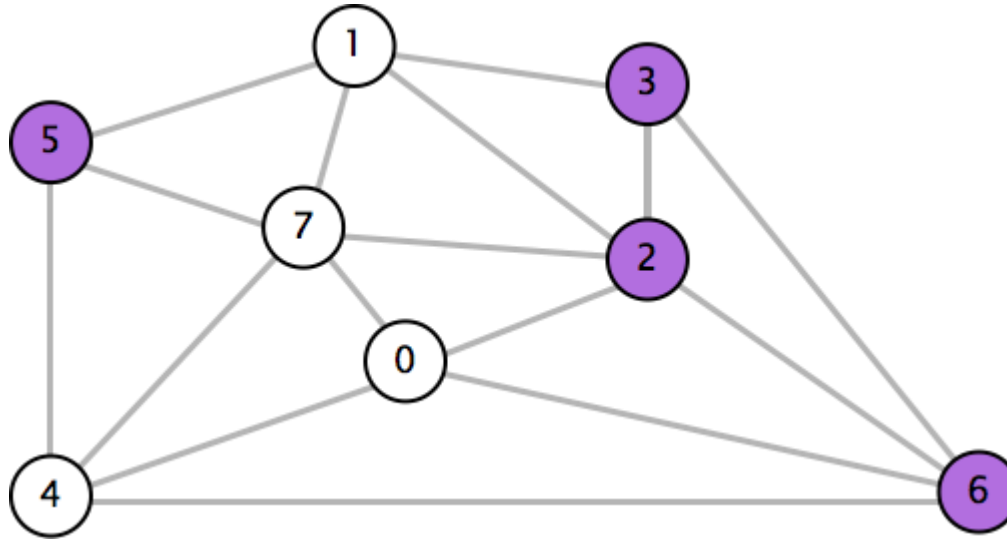
- A **cut** is an assignment of a graph's nodes to two non-empty sets
- A **crossing edge** is an edge which connects a node from one set to a node from the other set



Cut property: Given any cut, minimum weight crossing edge *is in the MST*

Cut Property Example

Which edge is the minimum weight edge crossing the cut $\{2, 3, 5, 6\}$?

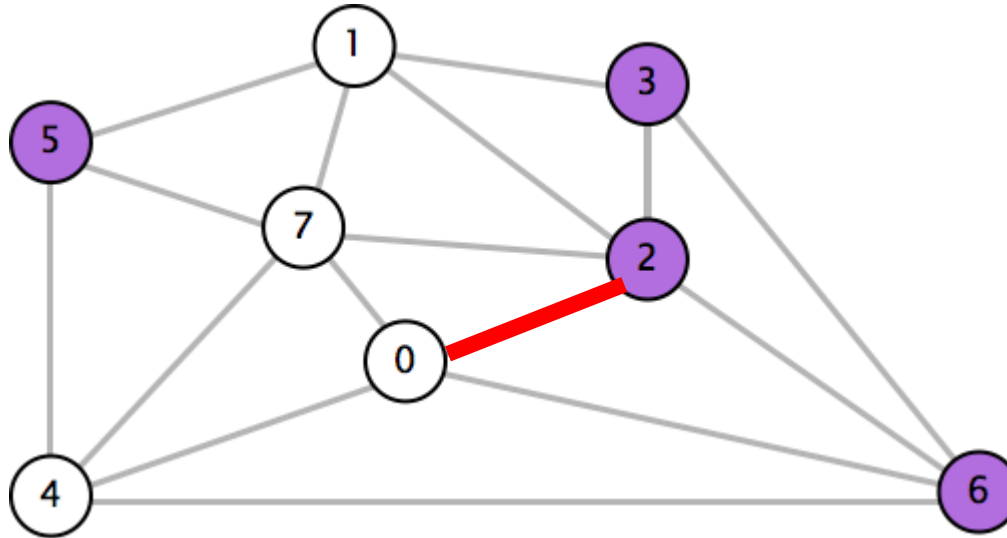


0-7	0.16
2-3	0.17
1-7	0.19
0-2	0.26
5-7	0.28
1-3	0.29
1-5	0.32
2-7	0.34
4-5	0.35
1-2	0.36
4-7	0.37
0-4	0.38
6-2	0.40
3-6	0.52
6-0	0.58
6-4	0.93

Cut Property Example

Which edge is the minimum weight edge crossing the cut $\{2, 3, 5, 6\}$?

- 0-2. Must be part of the MST!

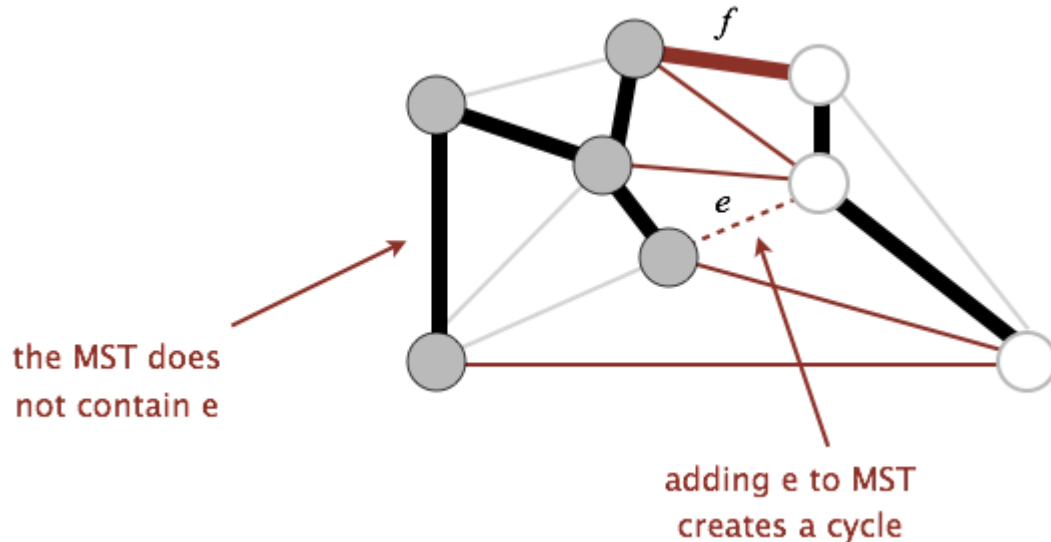


0-7	0.16
2-3	0.17
1-7	0.19
0-2	0.26
5-7	0.28
1-3	0.29
1-5	0.32
2-7	0.34
4-5	0.35
1-2	0.36
4-7	0.37
0-4	0.38
6-2	0.40
3-6	0.52
6-0	0.58
6-4	0.93

Cut Property Proof

Suppose that the minimum crossing edge e were not in the MST

- Adding e to the MST creates a cycle
- Some other edge f must also be a crossing edge
- Removing f and adding e is a lower weight spanning tree
- Contradiction!



Generic MST Finding Algorithm

Start with no edges in the MST

- Find a cut that has no crossing edges in the MST
- Add smallest crossing edge to the MST
- Repeat until $V-1$ edges

This should work, but we need ***some way*** of finding a cut with no crossing edges!

Prim's Algorithm

Start from some arbitrary start node

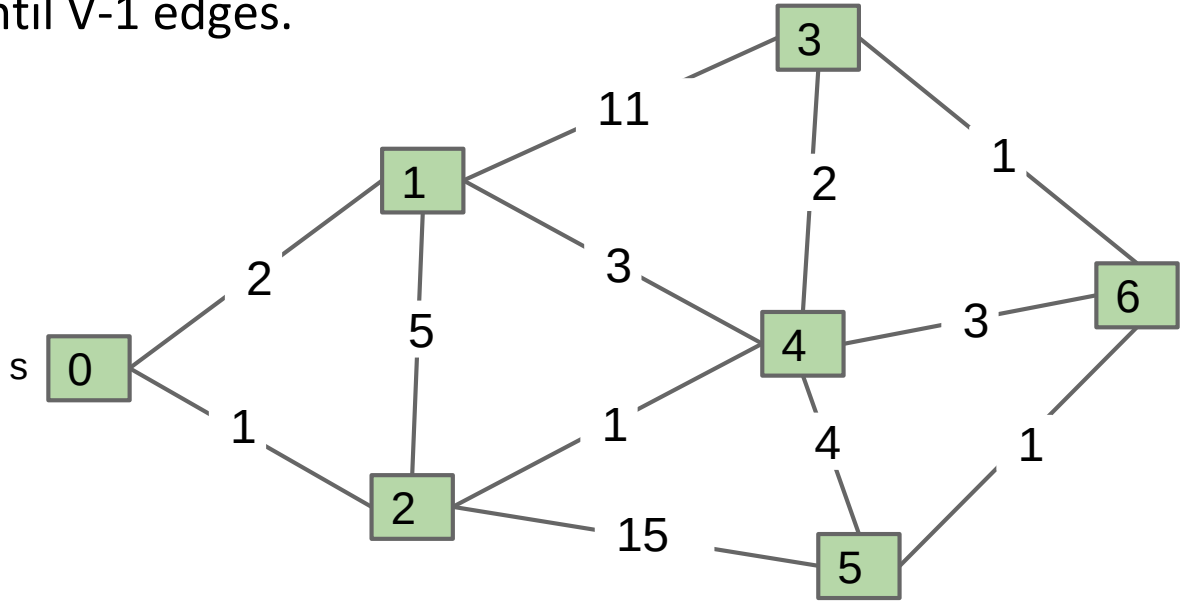
- Repeatedly add shortest edge (mark black) that has one node inside the MST under construction
- Repeat until $V-1$ edges

Prim's Demo

Start from some arbitrary start node.

- Add shortest edge (mark black) that has one node inside the MST under construction. Repeat until $V-1$ edges.

#	edgeTo
0	-
1	-
2	-
3	-
4	-
5	-
6	-

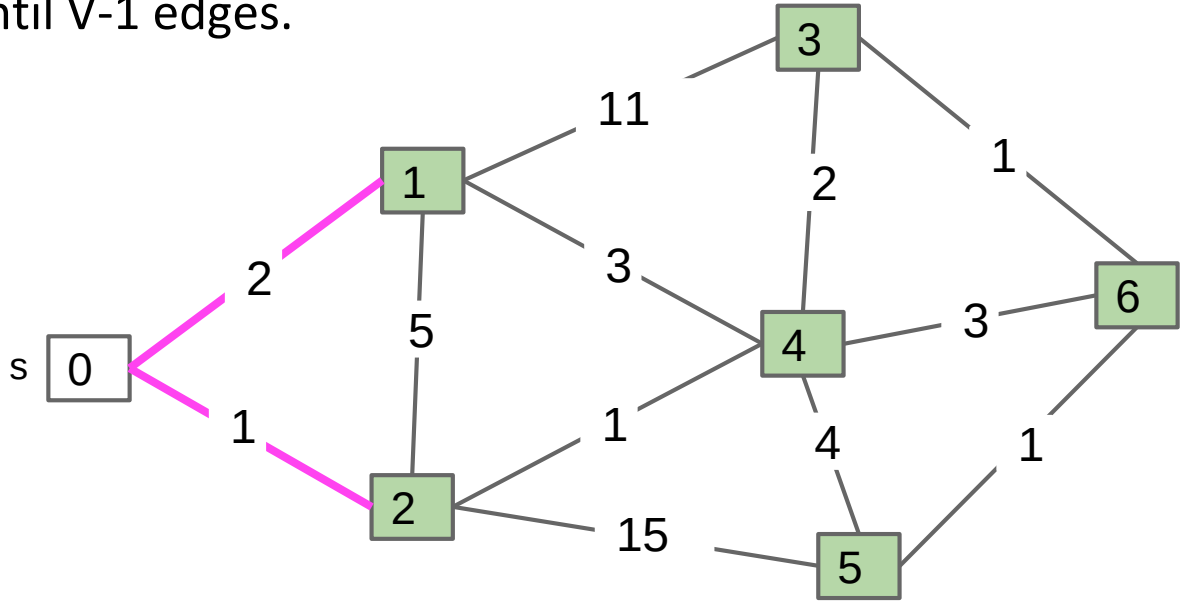


Prim's Demo

Start from some arbitrary start node.

- Add shortest edge (mark black) that has one node inside the MST under construction. Repeat until $V-1$ edges.

#	edgeTo
0	-
1	-
2	-
3	-
4	-
5	-
6	-

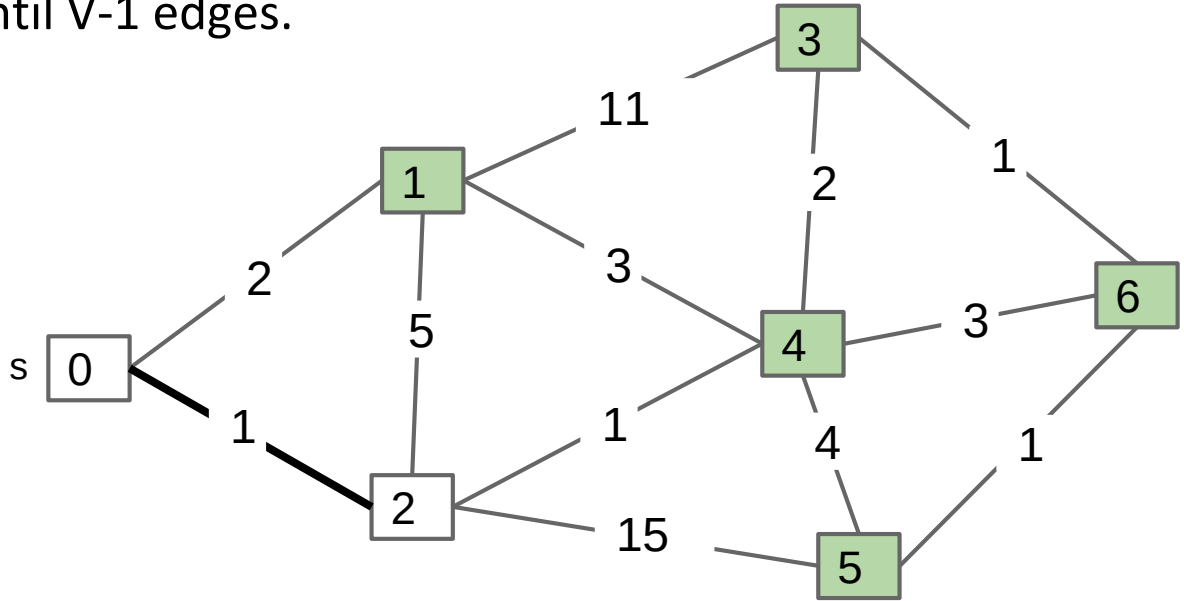


Prim's Demo

Start from some arbitrary start node.

- Add shortest edge (mark black) that has one node inside the MST under construction. Repeat until $V-1$ edges.

#	edgeTo
0	-
1	-
2	0
3	-
4	-
5	-
6	-

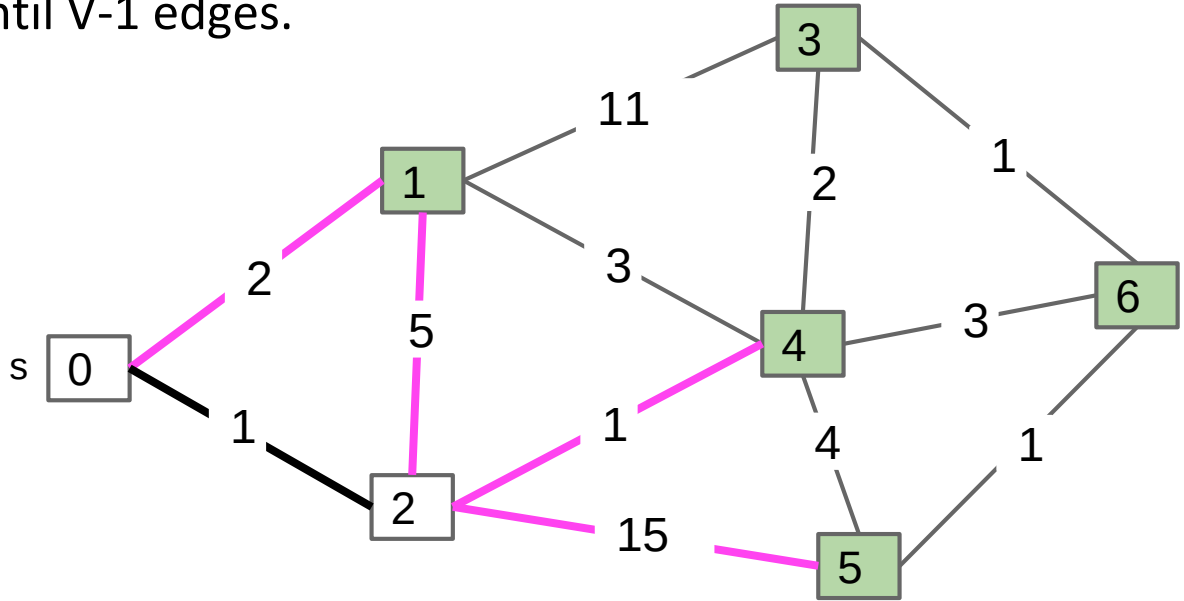


Prim's Demo

Start from some arbitrary start node.

- Add shortest edge (mark black) that has one node inside the MST under construction. Repeat until $V-1$ edges.

#	edgeTo
0	-
1	-
2	0
3	-
4	-
5	-
6	-

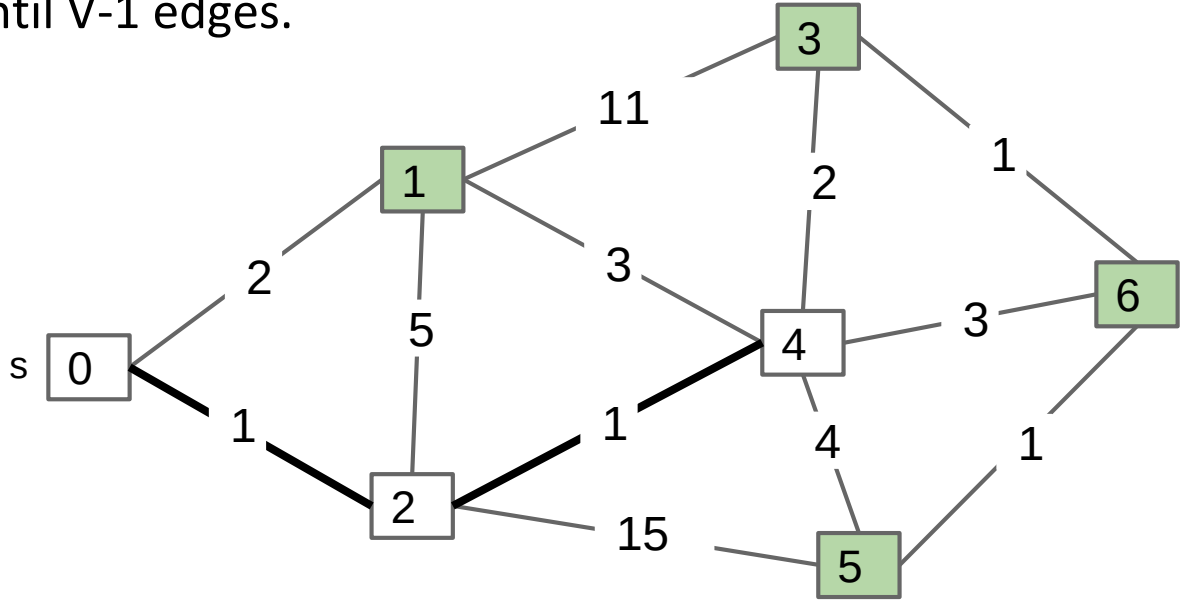


Prim's Demo

Start from some arbitrary start node.

- Add shortest edge (mark black) that has one node inside the MST under construction. Repeat until $V-1$ edges.

#	edgeTo
0	-
1	-
2	0
3	-
4	2
5	-
6	-

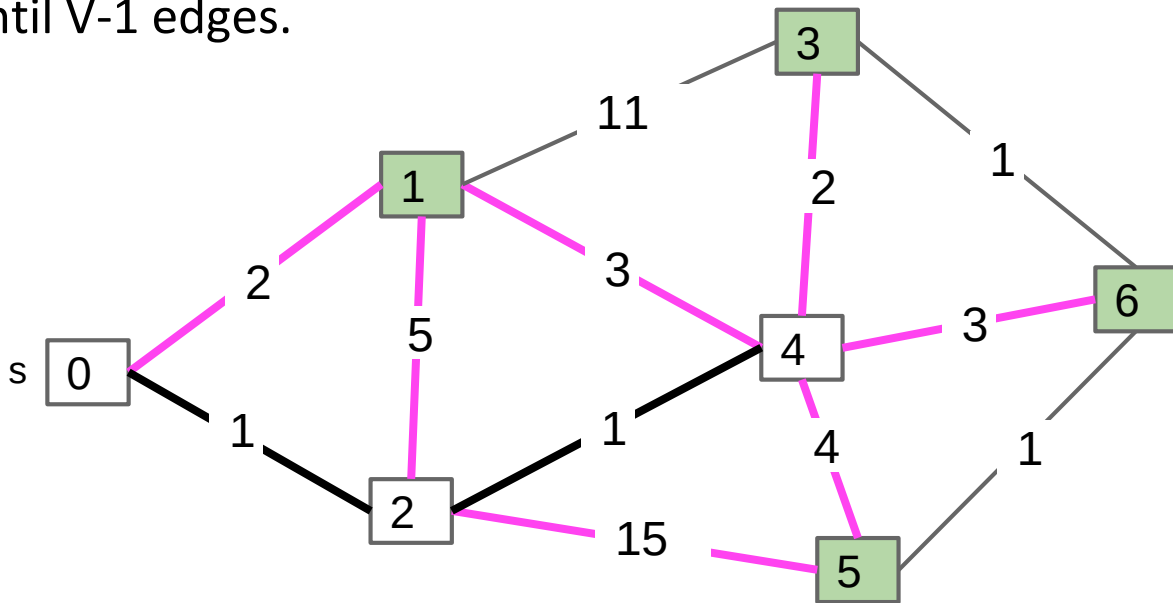


Prim's Demo

Start from some arbitrary start node.

- Add shortest edge (mark black) that has one node inside the MST under construction. Repeat until $V-1$ edges.

#	edgeTo
0	-
1	-
2	0
3	-
4	2
5	-
6	-



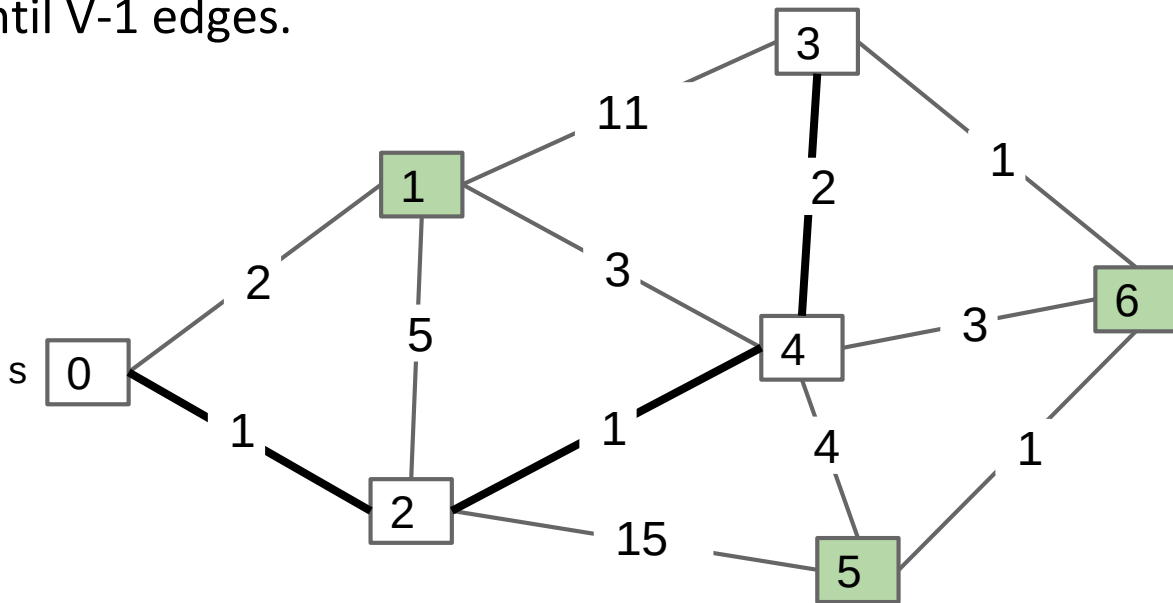
Which edge is added next?

Prim's Demo

Start from some arbitrary start node.

- Add shortest edge (mark black) that has one node inside the MST under construction. Repeat until $V-1$ edges.

#	edgeTo
0	-
1	-
2	0
3	4
4	2
5	-
6	-



Which edge is added next?

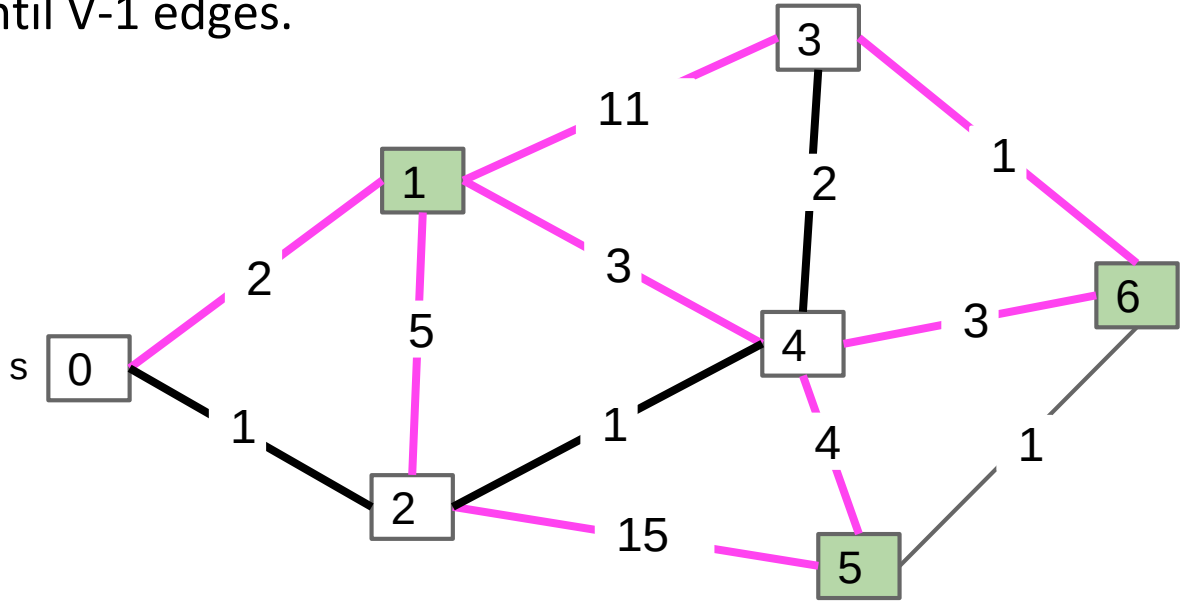
- Either 0-1 or 4-3 are guaranteed to work

Prim's Demo

Start from some arbitrary start node.

- Add shortest edge (mark black) that has one node inside the MST under construction. Repeat until $V-1$ edges.

#	edgeTo
0	-
1	-
2	0
3	4
4	2
5	-
6	-

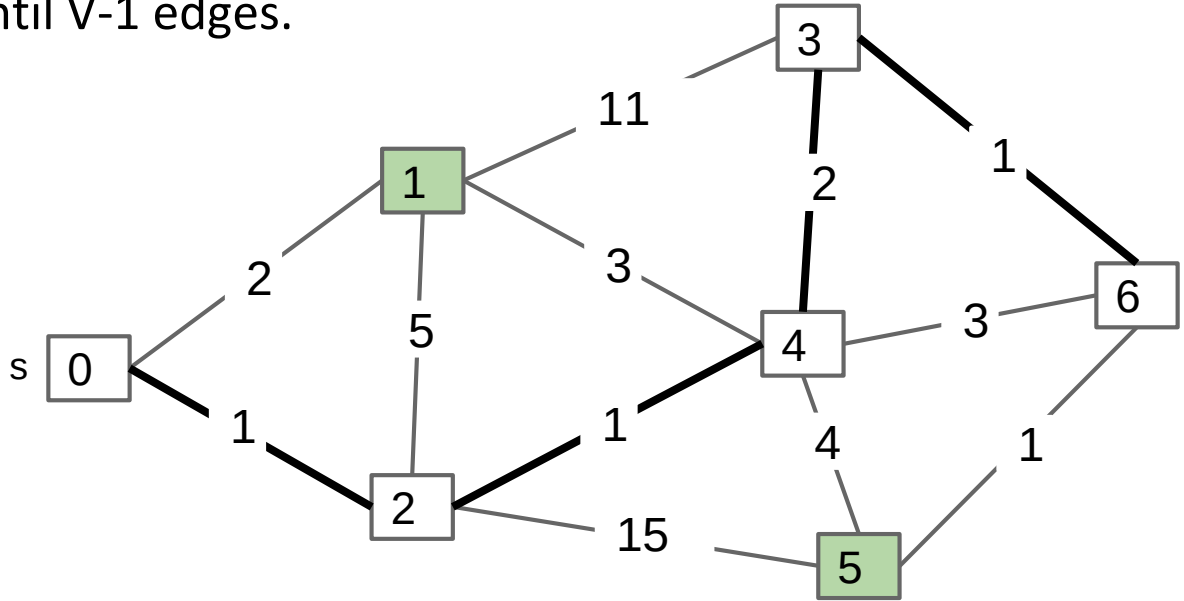


Prim's Demo

Start from some arbitrary start node.

- Add shortest edge (mark black) that has one node inside the MST under construction. Repeat until $V-1$ edges.

#	edgeTo
0	-
1	-
2	0
3	4
4	2
5	-
6	3

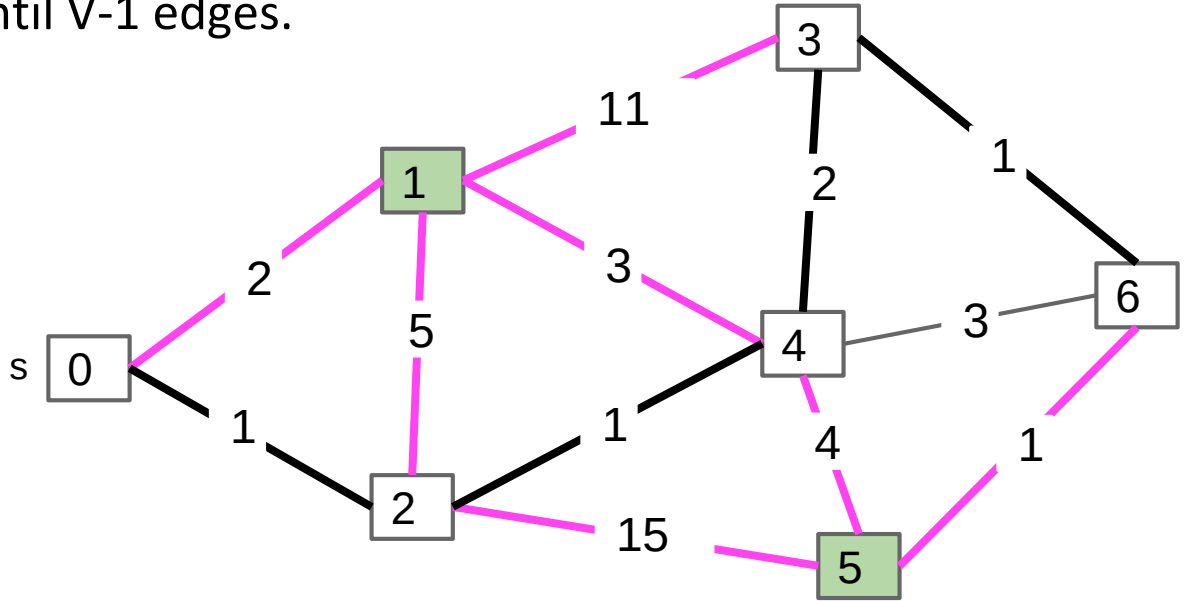


Prim's Demo

Start from some arbitrary start node.

- Add shortest edge (mark black) that has one node inside the MST under construction. Repeat until $V-1$ edges.

#	edgeTo
0	-
1	-
2	0
3	4
4	2
5	-
6	3

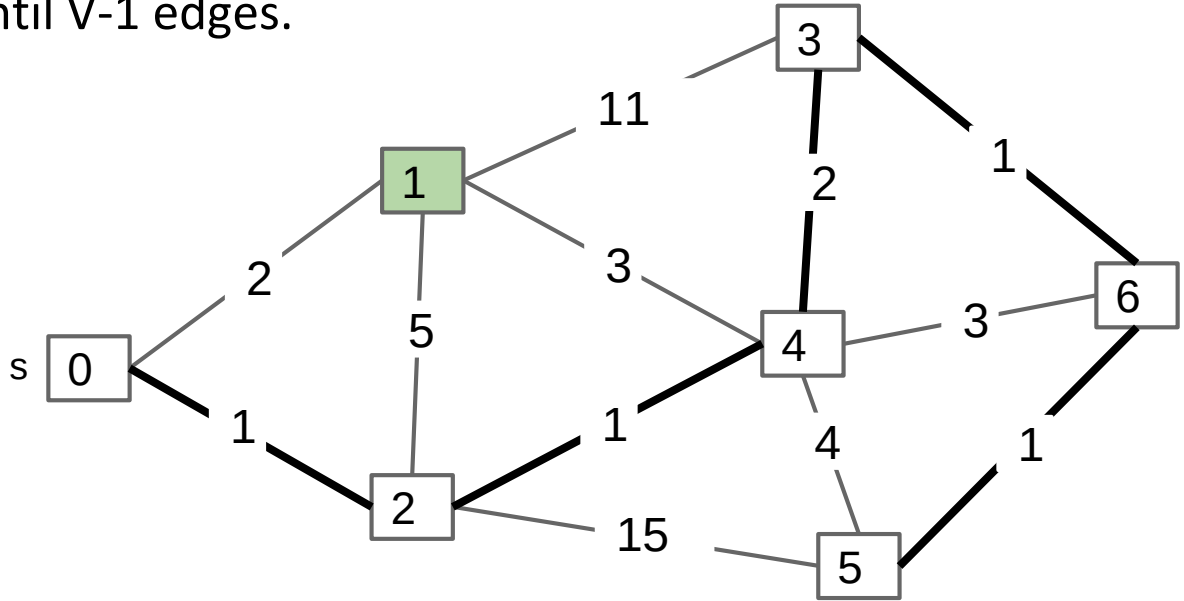


Prim's Demo

Start from some arbitrary start node.

- Add shortest edge (mark black) that has one node inside the MST under construction. Repeat until $V-1$ edges.

#	edgeTo
0	-
1	-
2	0
3	4
4	2
5	6
6	3

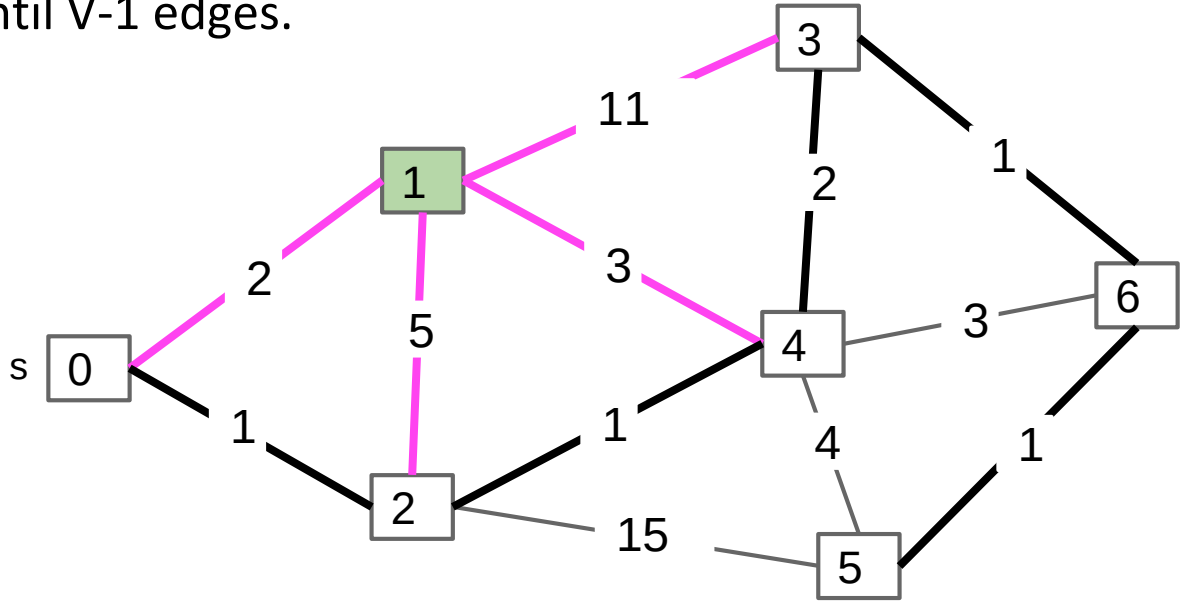


Prim's Demo

Start from some arbitrary start node.

- Add shortest edge (mark black) that has one node inside the MST under construction. Repeat until $V-1$ edges.

#	edgeTo
0	-
1	-
2	0
3	4
4	2
5	6
6	3

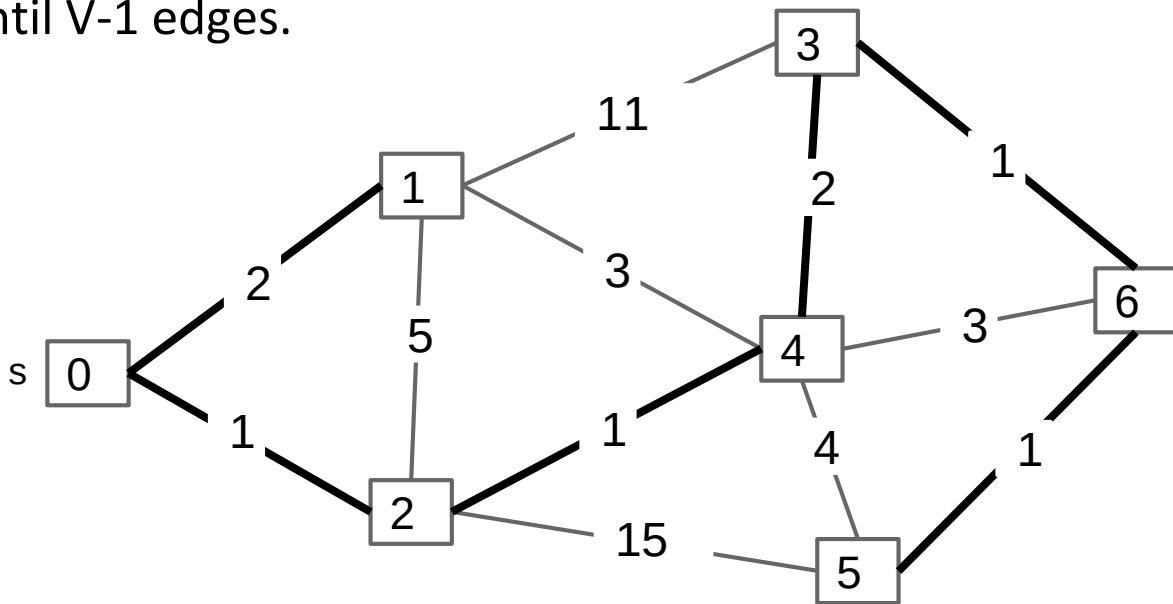


Prim's Demo

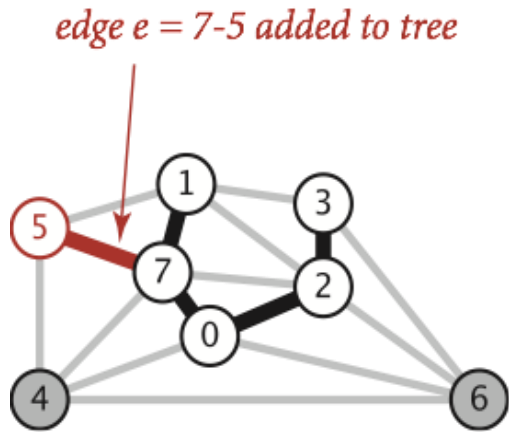
Start from some arbitrary start node.

- Add shortest edge (mark black) that has one node inside the MST under construction. Repeat until $V-1$ edges.

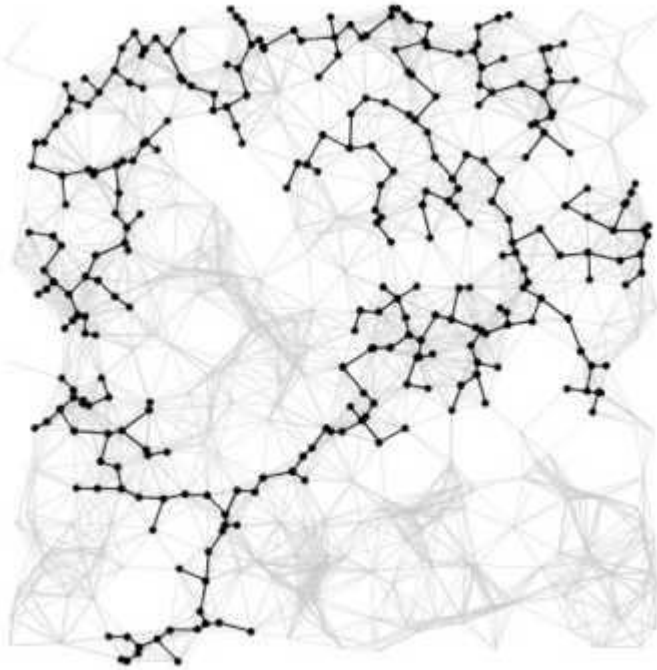
#	edgeTo
0	-
1	0
2	0
3	4
4	2
5	6
6	3



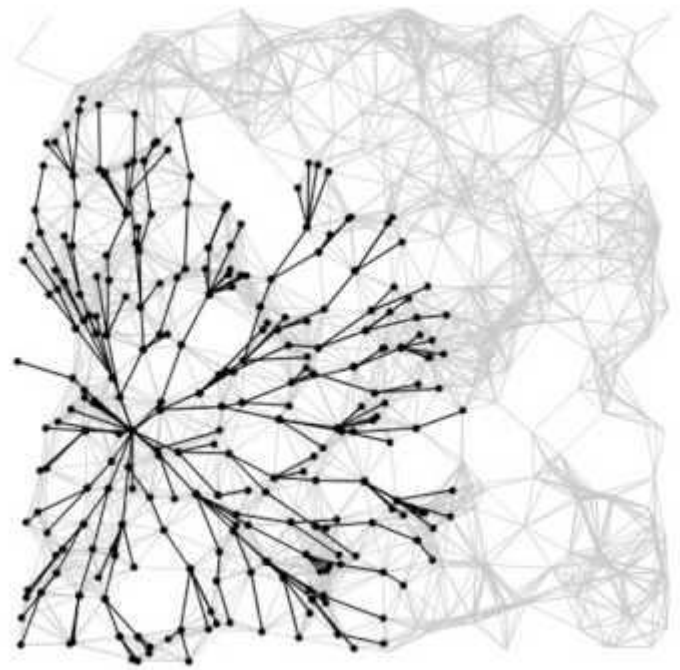
.....



Prim's vs. Dijkstra's



Prim's Algorithm



Dijkstra's Algorithm

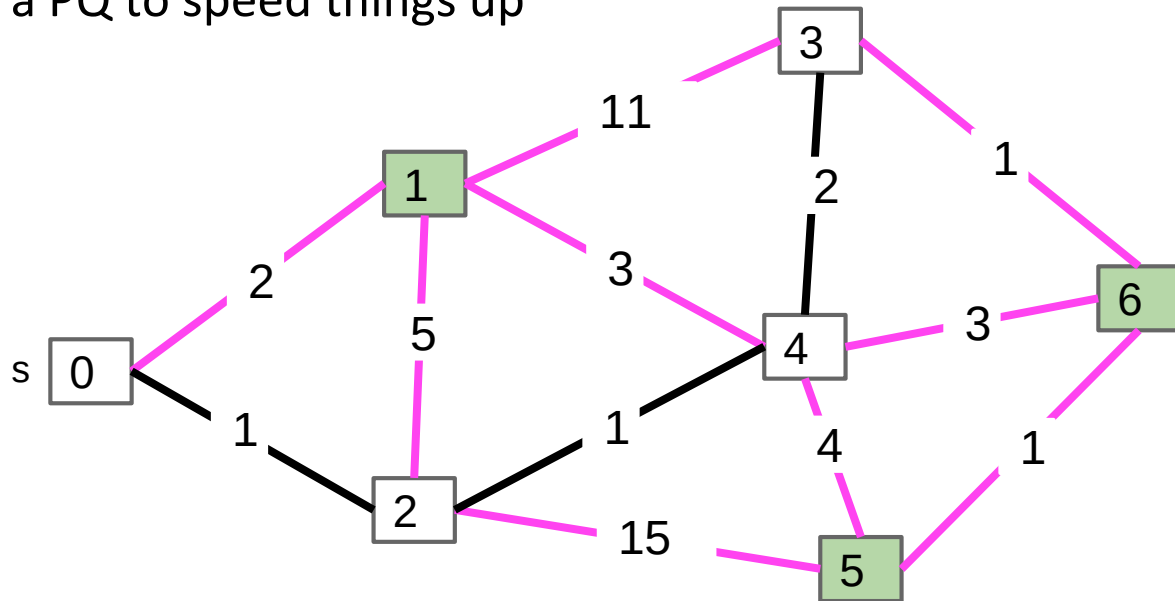
Prim's Algorithm Implementation

We need to find an efficient implementation of Prim's algorithm

- iterating over purple edges shown can be unnecessary and slow

Can use some cleverness and a PQ to speed things up

- very similar to Dijkstra's!

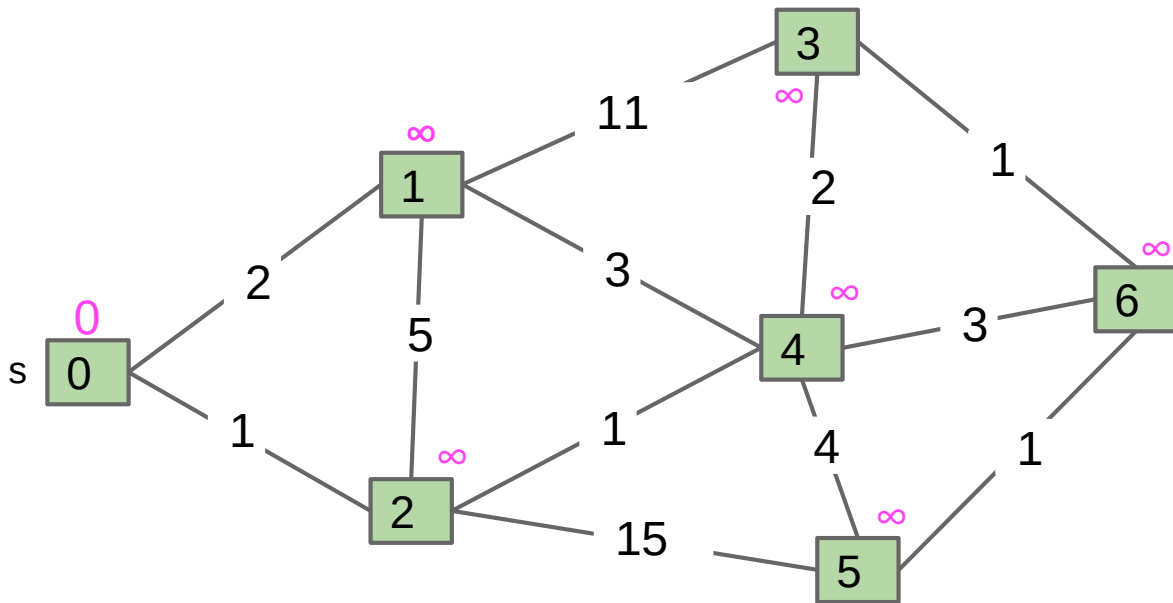


Prim's Implementation Demo

Insert all vertices into PQ, storing vertices in order of distance from tree

Repeat: Remove closest vertex v from PQ, and relax all edges pointing from v

#	distTo	edgeTo
0	0	-
1	∞	-
2	∞	-
3	∞	-
4	∞	-
5	∞	-
6	∞	-



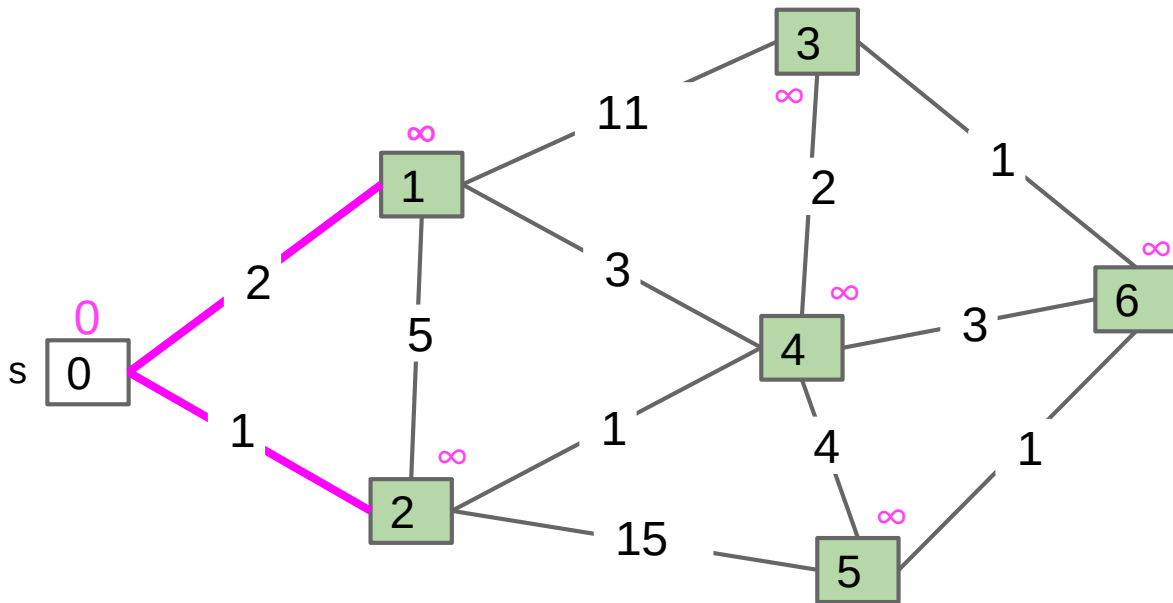
PQ: [(0: 0), (1: ∞), (2: ∞), (3: ∞), (4: ∞), (5: ∞), (6: ∞)]

Prim's Implementation Demo

Insert all vertices into PQ, storing vertices in order of distance from tree

Repeat: Remove closest vertex v from PQ, and relax all edges pointing from v

#	distTo	edgeTo
0	0	-
1	∞	-
2	∞	-
3	∞	-
4	∞	-
5	∞	-
6	∞	-



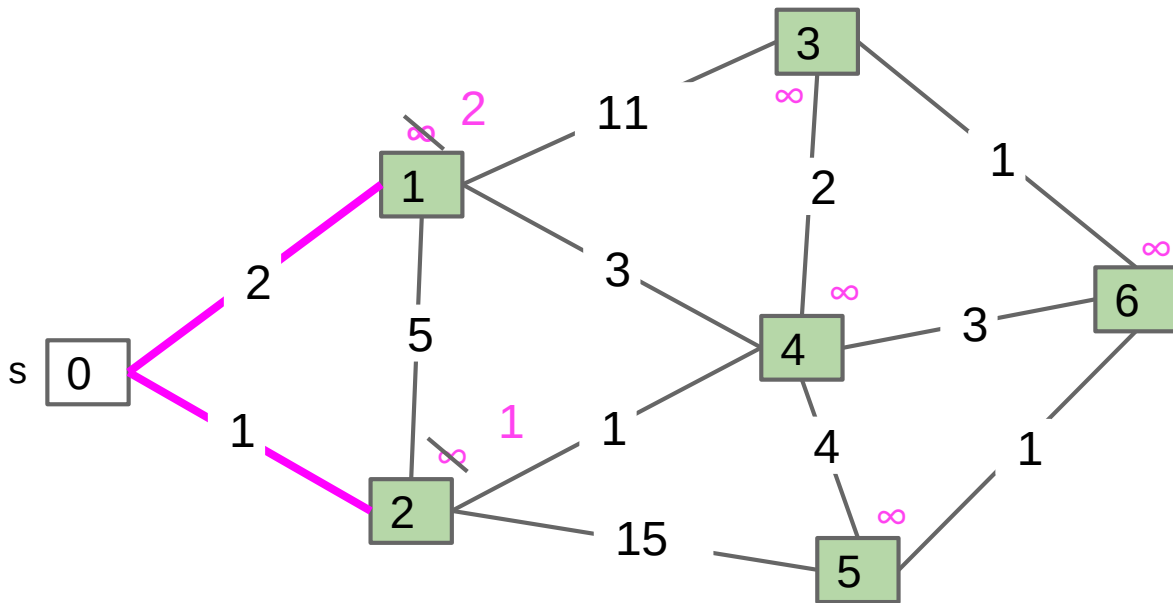
PQ: [(1: ∞), (2: ∞), (3: ∞), (4: ∞), (5: ∞), (6: ∞)]

Prim's Implementation Demo

Insert all vertices into PQ, storing vertices in order of distance from tree

Repeat: Remove closest vertex v from PQ, and relax all edges pointing from v

#	distTo	edgeTo
0		-
1	2	0
2	1	0
3	∞	-
4	∞	-
5	∞	-
6	∞	-



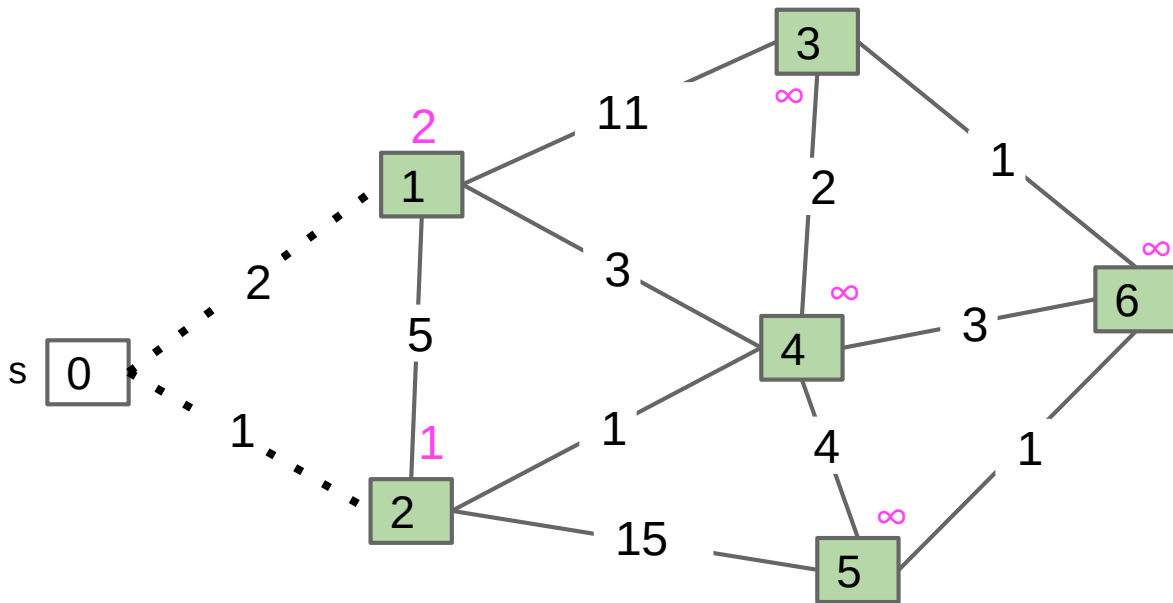
PQ: [(2: 1), (1: 2), (3: ∞), (4: ∞), (5: ∞), (6: ∞)]

Prim's Implementation Demo

Insert all vertices into PQ, storing vertices in order of distance from tree

Repeat: Remove closest vertex v from PQ, and relax all edges pointing from v

#	distTo	edgeTo
0	0	-
1	2	0
2	1	0
3	∞	-
4	∞	-
5	∞	-
6	∞	-



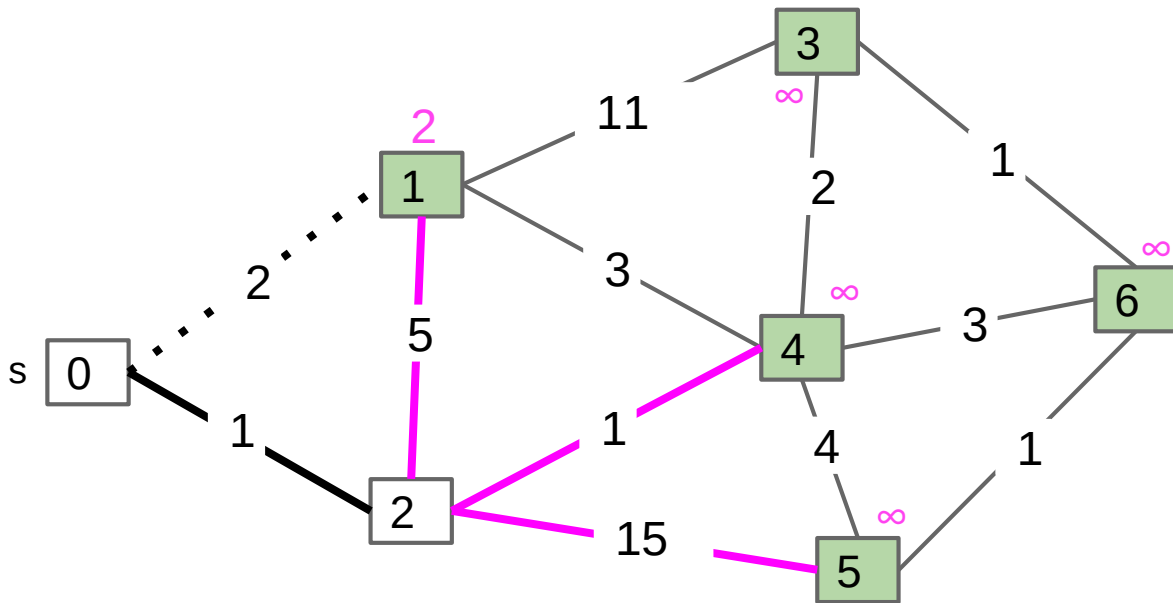
PQ: [(2: 1), (1: 2), (3: ∞), (4: ∞), (5: ∞), (6: ∞)]

Prim's Implementation Demo

Insert all vertices into PQ, storing vertices in order of distance from tree

Repeat: Remove closest vertex v from PQ, and relax all edges pointing from v

#	distTo	edgeTo
0		-
1	2	0
2		0
3	∞	-
4	∞	-
5	∞	-
6	∞	-



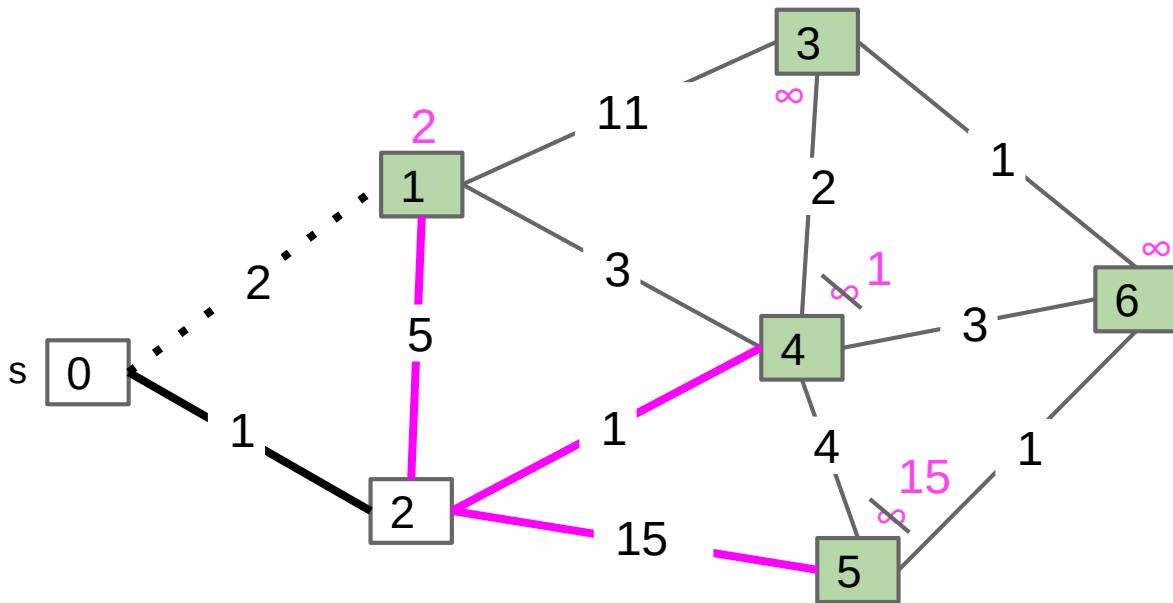
PQ: [(1: 2), (3: ∞), (4: ∞), (5: ∞), (6: ∞)]

Prim's Implementation Demo

Insert all vertices into PQ, storing vertices in order of distance from tree

Repeat: Remove closest vertex v from PQ, and relax all edges pointing from v

#	distTo	edgeTo
0		-
1	2	0
2	0	0
3	∞	-
4	1	2
5	15	2
6	∞	-



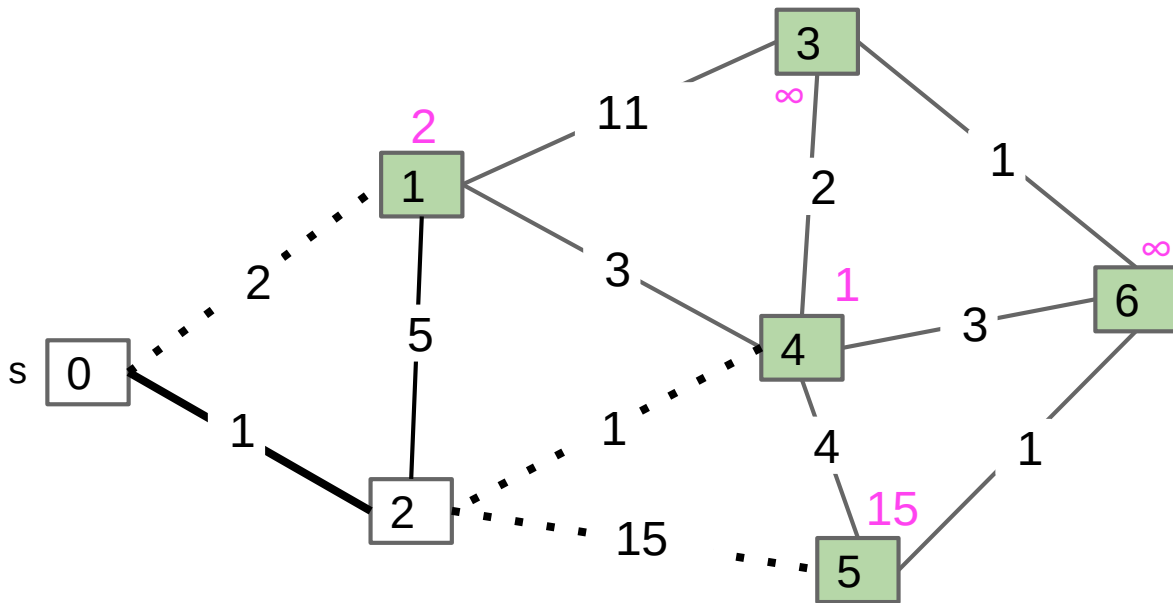
PQ: [(4: 1), (1: 2), (5: 15), (3: ∞), (6: ∞)]

Prim's Implementation Demo

Insert all vertices into PQ, storing vertices in order of distance from tree

Repeat: Remove closest vertex v from PQ, and relax all edges pointing from v

#	distTo	edgeTo
0		-
1	2	0
2		0
3	∞	-
4	1	2
5	15	2
6	∞	-



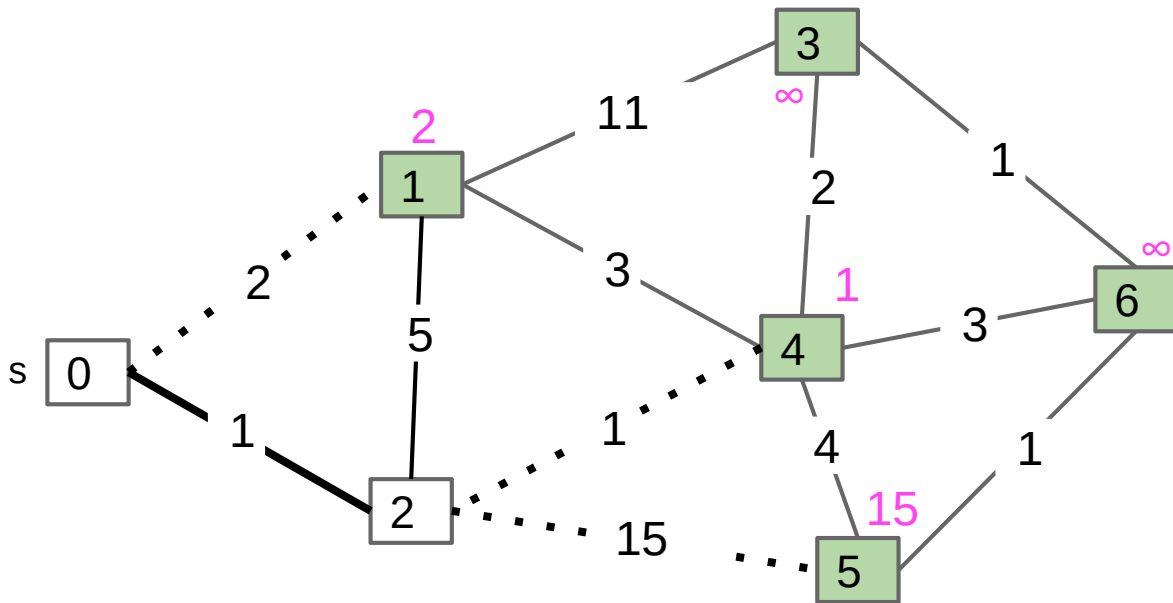
PQ: [(4: 1), (1: 2), (5: 15), (3: ∞), (6: ∞)]

Prim's Implementation Demo

Insert all vertices into PQ, storing vertices in order of distance from tree

Repeat: Remove closest vertex v from PQ, and relax all edges pointing from v

#	distTo	edgeTo
0		-
1	2	0
2		0
3	∞	-
4	1	2
5	15	2
6	∞	-



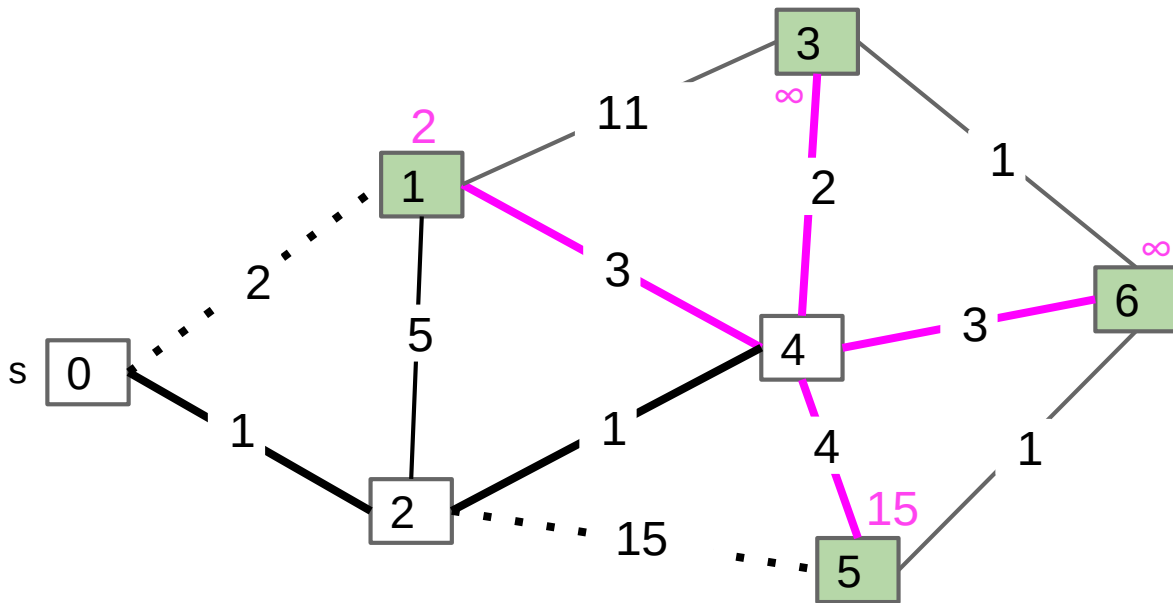
PQ: [(4: 1), (1: 2), (5: 15), (3: ∞), (6: ∞)]

Prim's Implementation Demo

Insert all vertices into PQ, storing vertices in order of distance from tree

Repeat: Remove closest vertex v from PQ, and relax all edges pointing from v

#	distTo	edgeTo
0		-
1	2	0
2		0
3	∞	-
4		2
5	15	2
6	∞	-



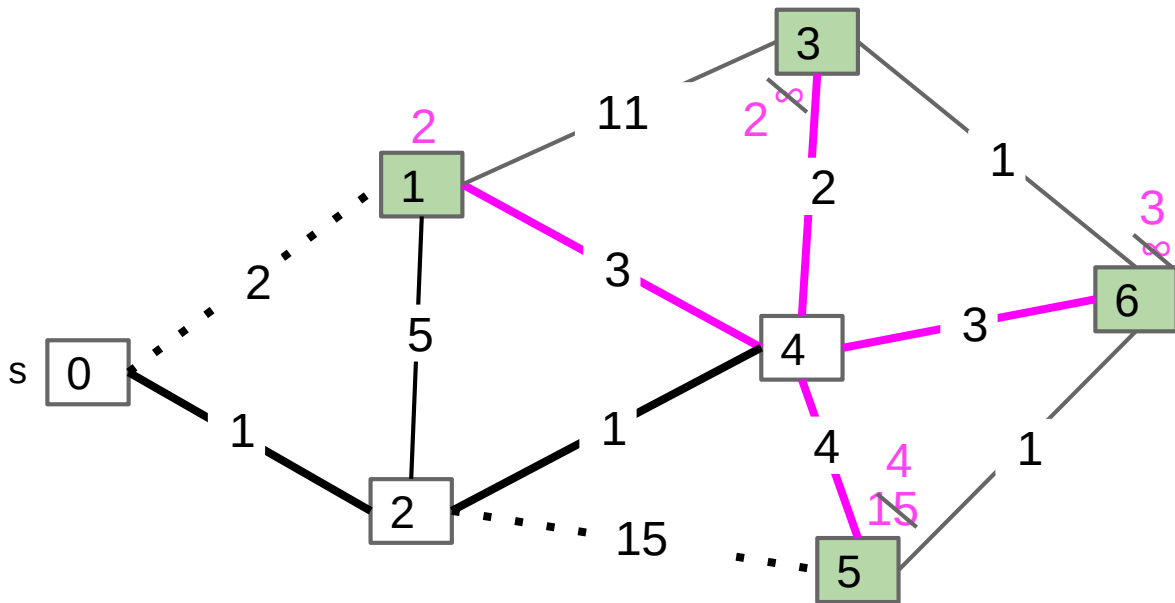
PQ: [(1: 2), (5: 15), (3: ∞), (6: ∞)]

Prim's Implementation Demo

Insert all vertices into PQ, storing vertices in order of distance from tree

Repeat: Remove closest vertex v from PQ, and relax all edges pointing from v

#	distTo	edgeTo
0	-	-
1	2	0
2	0	0
3	2	4
4	2	2
5	4	4
6	3	4



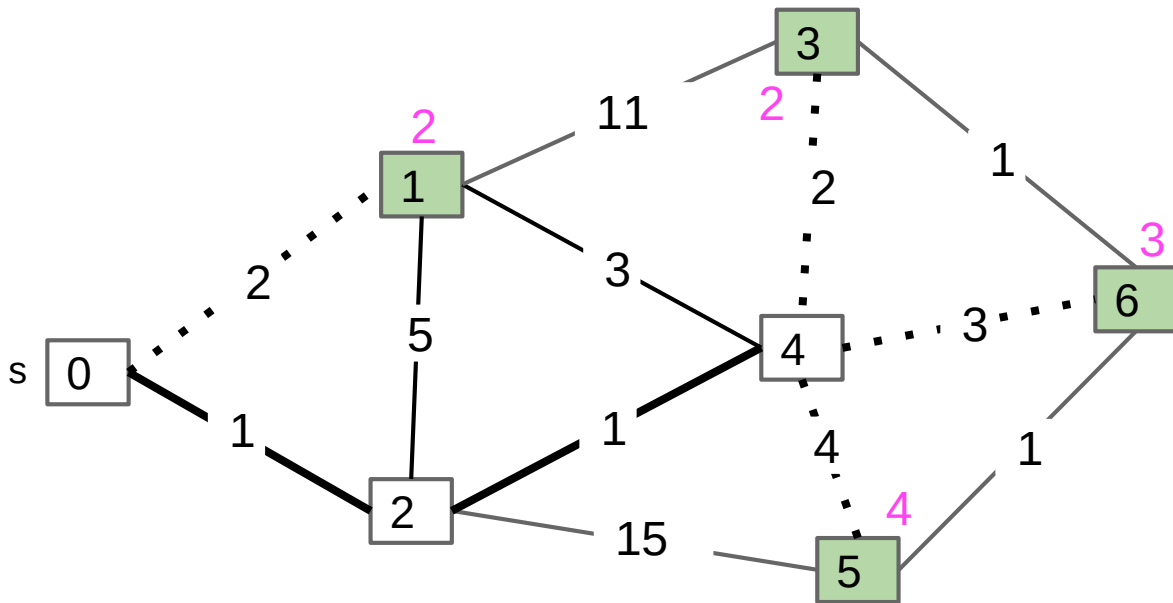
PQ: [(1: 2), (3: 2), (6: 3), (5: 4)]

Prim's Implementation Demo

Insert all vertices into PQ, storing vertices in order of distance from tree

Repeat: Remove closest vertex v from PQ, and relax all edges pointing from v

#	distTo	edgeTo
0		-
1	2	0
2	0	0
3	2	4
4	2	2
5	4	4
6	3	4



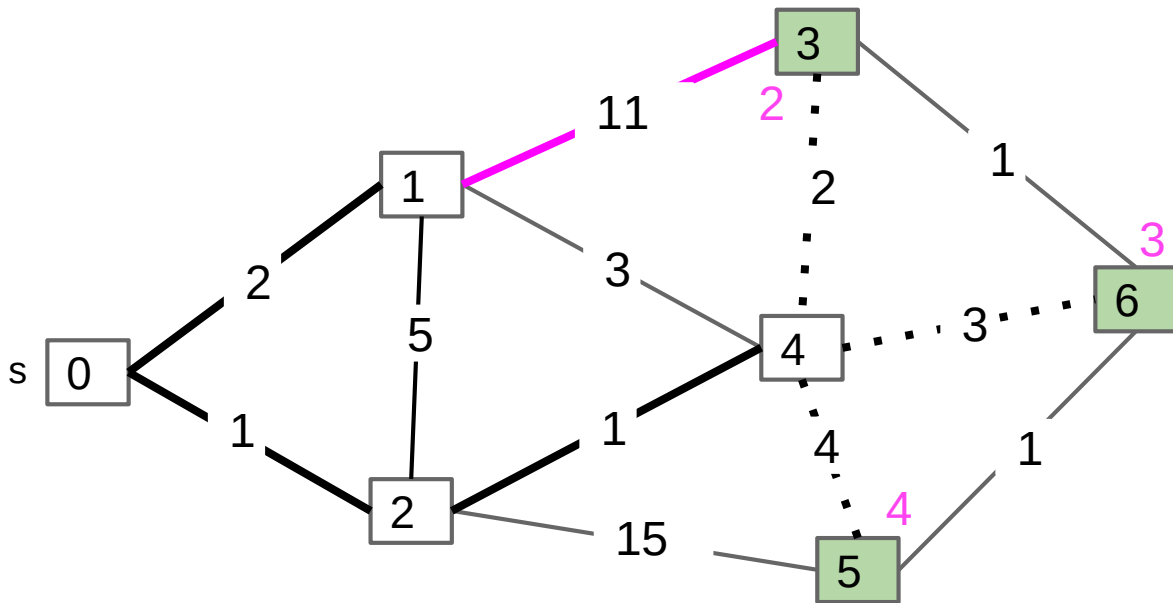
PQ: [(1: 2), (3: 2), (6: 3), (5: 4)]

Prim's Implementation Demo

Insert all vertices into PQ, storing vertices in order of distance from tree

Repeat: Remove closest vertex v from PQ, and relax all edges pointing from v

#	distTo	edgeTo
0	-	-
1	0	0
2	0	0
3	2	4
4	2	2
5	4	4
6	3	4



PQ: [(3: 2), (6: 3), (5: 4)]

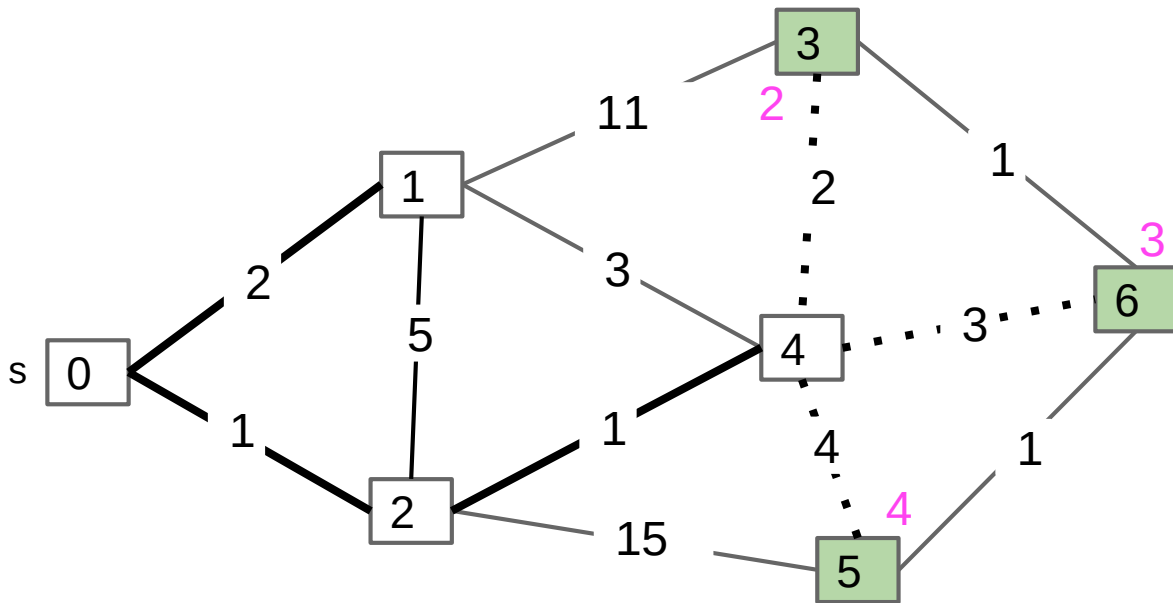
No need to consider edges with weight 5 and 3 since other side is already marked!

Prim's Implementation Demo

Insert all vertices into PQ, storing vertices in order of distance from tree

Repeat: Remove closest vertex v from PQ, and relax all edges pointing from v

#	distTo	edgeTo
0	-	-
1	0	0
2	0	0
3	2	4
4	2	2
5	4	4
6	3	4



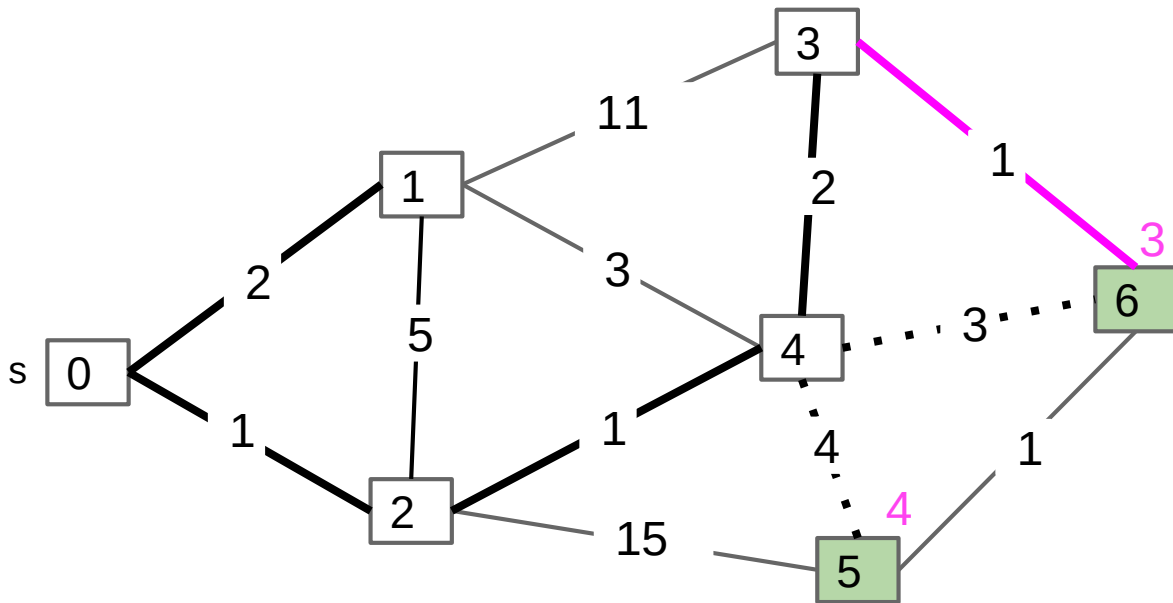
PQ: [(3: 2), (6: 3), (5: 4)]

Prim's Implementation Demo

Insert all vertices into PQ, storing vertices in order of distance from tree

Repeat: Remove closest vertex v from PQ, and relax all edges pointing from v

#	distTo	edgeTo
0		-
1		0
2		0
3		4
4		2
5	4	4
6	3	4



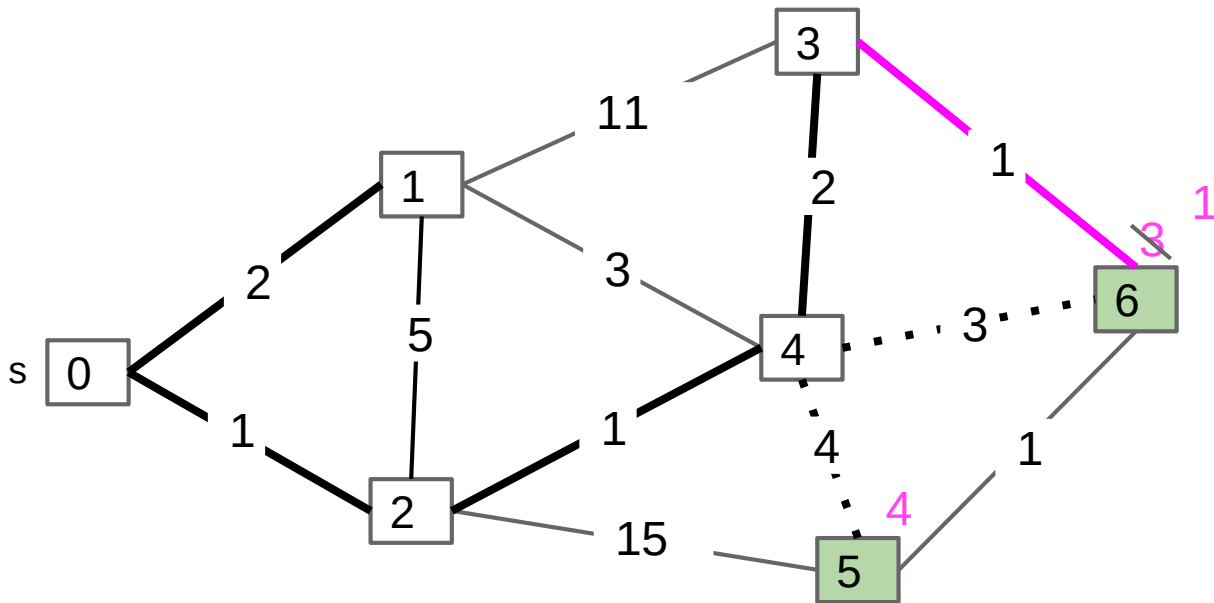
PQ: [(6: 3), (5: 4)]

Prim's Implementation Demo

Insert all vertices into PQ, storing vertices in order of distance from tree

Repeat: Remove closest vertex v from PQ, and relax all edges pointing from v

#	distTo	edgeTo
0		-
1		0
2		0
3		4
4		2
5	4	4
6	1	3



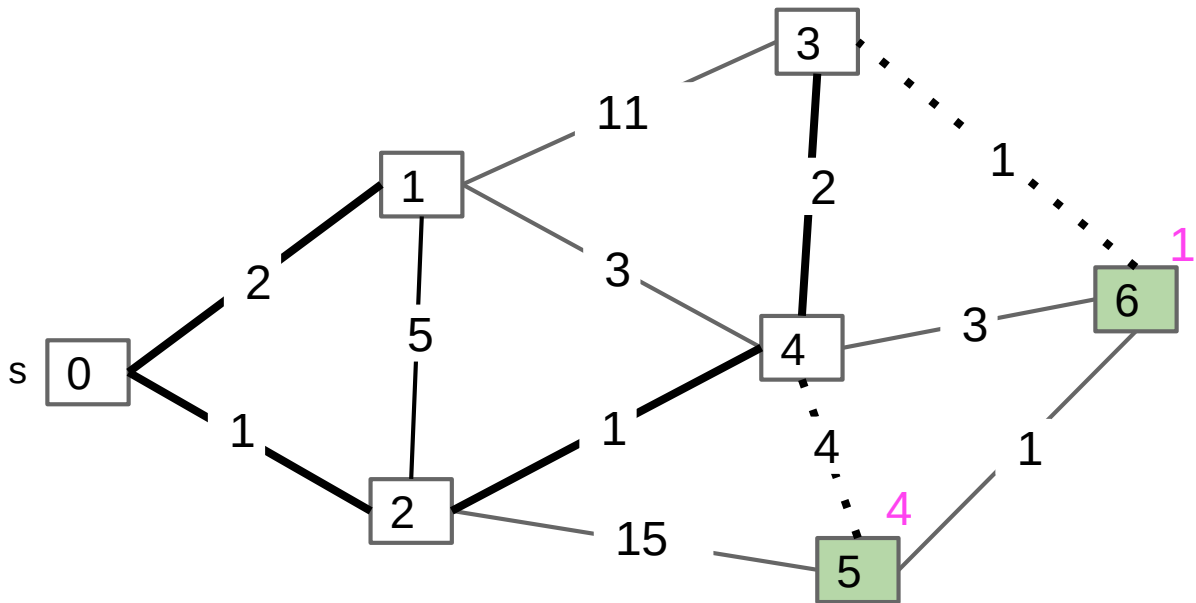
PQ: [(6: 1), (5: 4)]

Prim's Implementation Demo

Insert all vertices into PQ, storing vertices in order of distance from tree

Repeat: Remove closest vertex v from PQ, and relax all edges pointing from v

#	distTo	edgeTo
0		-
1		0
2		0
3		4
4		2
5	4	4
6	1	3



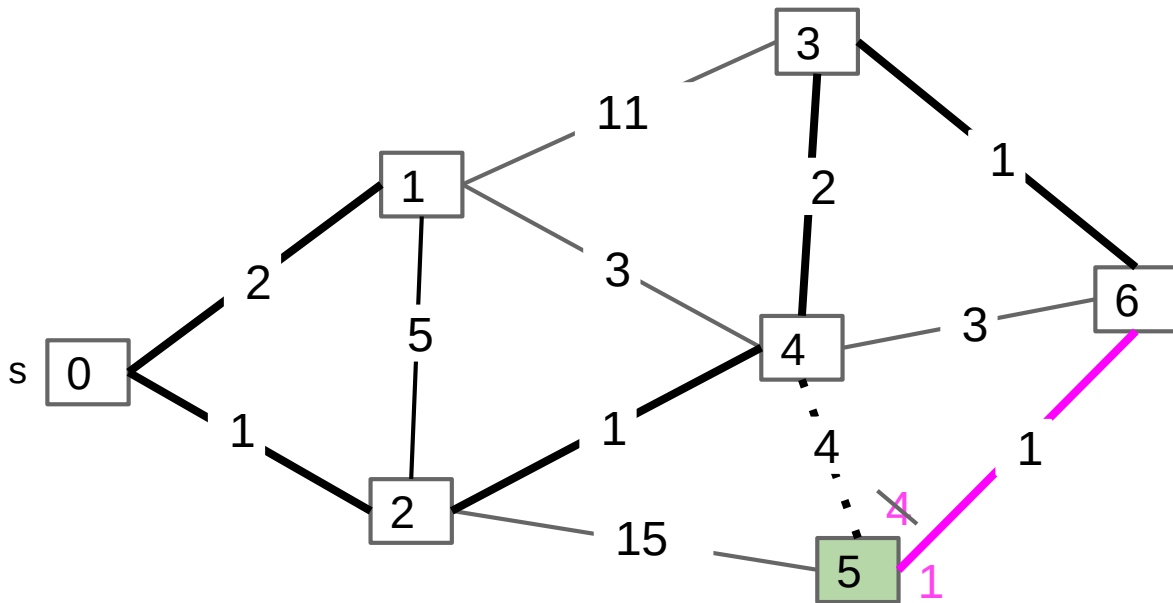
PQ: [(6: 1), (5: 4)]

Prim's Implementation Demo

Insert all vertices into PQ, storing vertices in order of distance from tree

Repeat: Remove closest vertex v from PQ, and relax all edges pointing from v

#	distTo	edgeTo
0		-
1		0
2		0
3		4
4		2
5	1	6
6		3



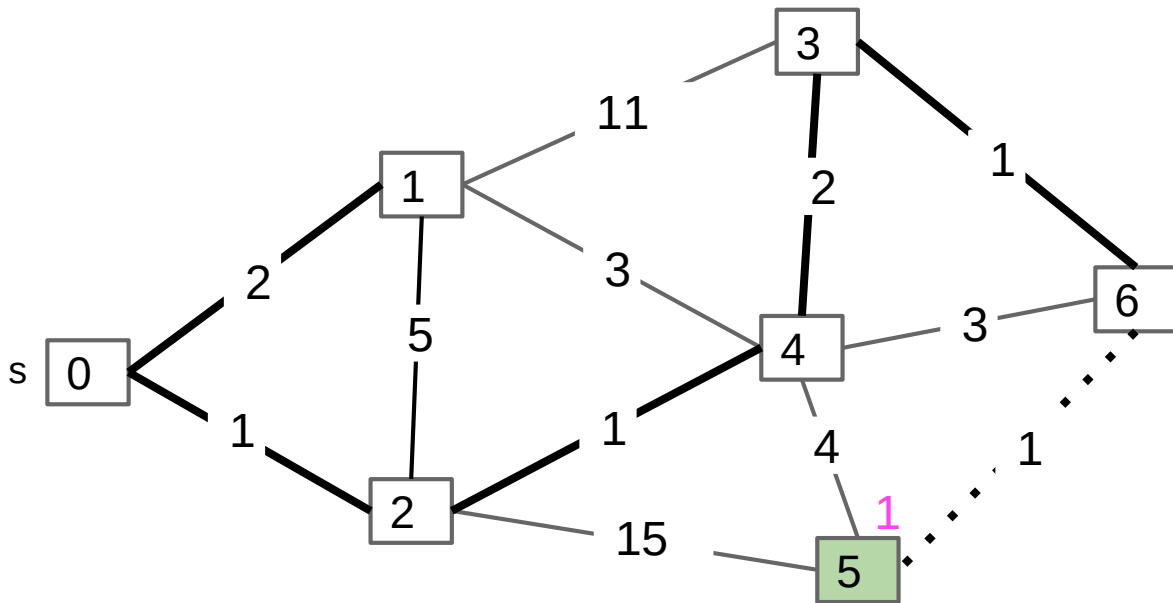
PQ: [(5: 1)]

Prim's Implementation Demo

Insert all vertices into PQ, storing vertices in order of distance from tree

Repeat: Remove closest vertex v from PQ, and relax all edges pointing from v

#	distTo	edgeTo
0		-
1		0
2		0
3		4
4		2
5	1	6
6		3



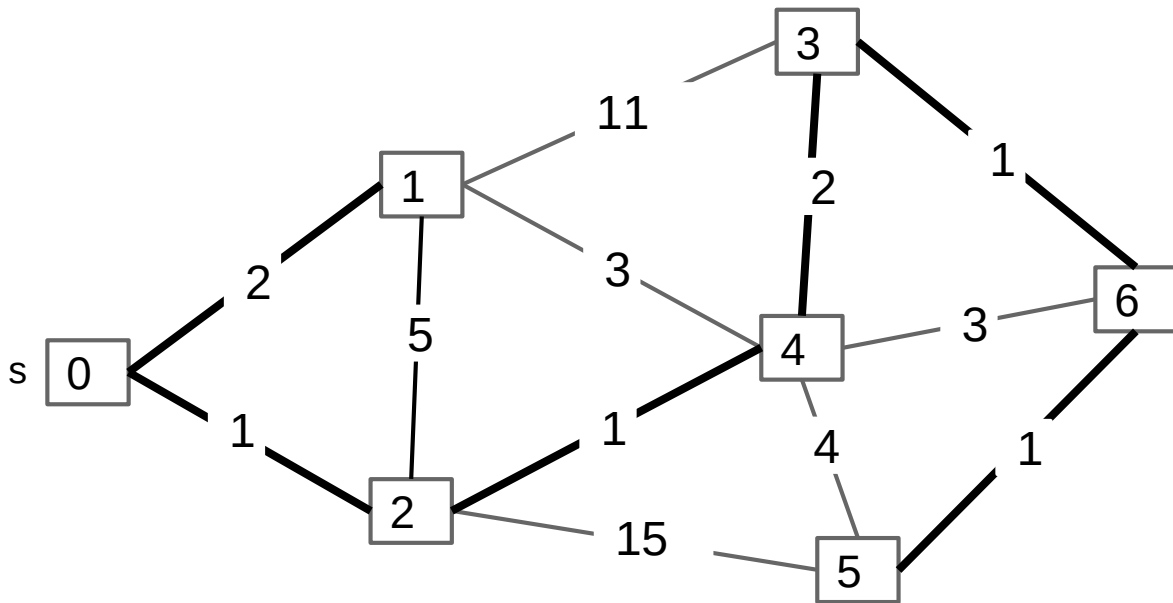
PQ: [(5: 1)]

Prim's Implementation Demo

Insert all vertices into PQ, storing vertices in order of distance from tree

Repeat: Remove closest vertex v from PQ, and relax all edges pointing from v

#	distTo	edgeTo
0	-	-
1	0	
2	0	
3	4	
4	2	
5	6	
6	3	



PQ: []

Prim's vs. Dijkstra's

Prim's and Dijkstra's algorithms are exactly the same, except Dijkstra's considers “distance from the source”, and Prim's considers “distance from the tree.”

Visit order:

- Dijkstra's algorithm visits vertices in order of distance from the source.
- Prim's algorithm visits vertices in order of distance from the MST under construction.

Relaxation:

- Relaxation in Dijkstra's considers an edge better based on distance to source.
- Relaxation in Prim's considers an edge better based on distance to tree.

Prim's Implementation 1

```
public class PrimMST {
    public PrimMST(EdgeWeightedGraph G) {
        edgeTo = new Edge[G.V()];
        distTo = new double[G.V()];
        marked = new boolean[G.V()];
        pq = new MinPQ<Double>(G.V());

        distTo[s] = 0.0;
        setDistancesToInfinityExceptS(s);
        insertAllVertices(pq);

        /* Get vertices in order of distance from tree */
        while (!pq.isEmpty()) {
            int v = pq.delMin();
            scan(G, v);
        }
    }
    ...
}
```

PQ is ordered by distTo
tree, must be an indexed
MinPQ with decreaseKey

get vertex closest to tree
that is unvisited

scan means to consider
all of a vertices outgoing
edges (next slide)

Prim's Implementation 2

```
while (!pq.isEmpty()) {  
    int v = pq.delMin();  
    scan(G, v);  
}
```

```
private void scan(EdgeWeightedGraph G, int v) {  
    marked[v] = true;  
    for (Edge e : G.adj(v)) {  
        int w = e.other(v);  
        if (marked[w]) { continue; }  
        if (e.weight() < distTo[w]) {  
            distTo[w] = e.weight();  
            edgeTo[w] = e;  
            pq.decreaseKey(w, distTo[w]);  
        }  
    }  
}
```

vertex v is closest, so add to MST

already in MST, so go to next edge
better path to a particular vertex
found, so update current best known
for that vertex

Prim's Runtime

```
while (!pq.isEmpty()) {  
    int v = pq.delMin();  
    scan(G, v);  
}
```

```
private void scan(EdgeWeightedGraph G, int v) {  
    marked[v] = true;  
    for (Edge e : G.adj(v)) {  
        int w = e.other(v);  
        if (marked[w]) { continue; }  
        if (e.weight() < distTo[w]) {  
            distTo[w] = e.weight();  
            edgeTo[w] = e;  
            pq.decreaseKey(w, distTo[w]);  
        }  
    }  
}
```

What is the runtime of Prim's algorithm?

- Assume all PQ operations take $O(\log(V))$ time
- Give your answer in Big O notation

Prim's Algorithm Runtime

Priority Queue operation count, assuming binary heap based PQ:

- Addition: V , each costing $O(\log V)$ time
- Delete min: V , each costing $O(\log V)$ time
- Decrease key: $O(E)$, each costing $O(\log V)$ time

Overall runtime: $O(V \cdot \log(V) + V \cdot \log(V) + E \cdot \log V)$

- Assuming $E > V$, this is just **$O(E \log V)$**

Operation	# Operations	Cost per operation	Total cost
PQ add	V	$O(\log V)$	$O(V \log V)$
PQ delMin	V	$O(\log V)$	$O(V \log V)$
PQ decreaseKey	$O(E)$	$O(\log V)$	$O(E \log V)$

Kruskal's Algorithm

Initially mark all edges gray

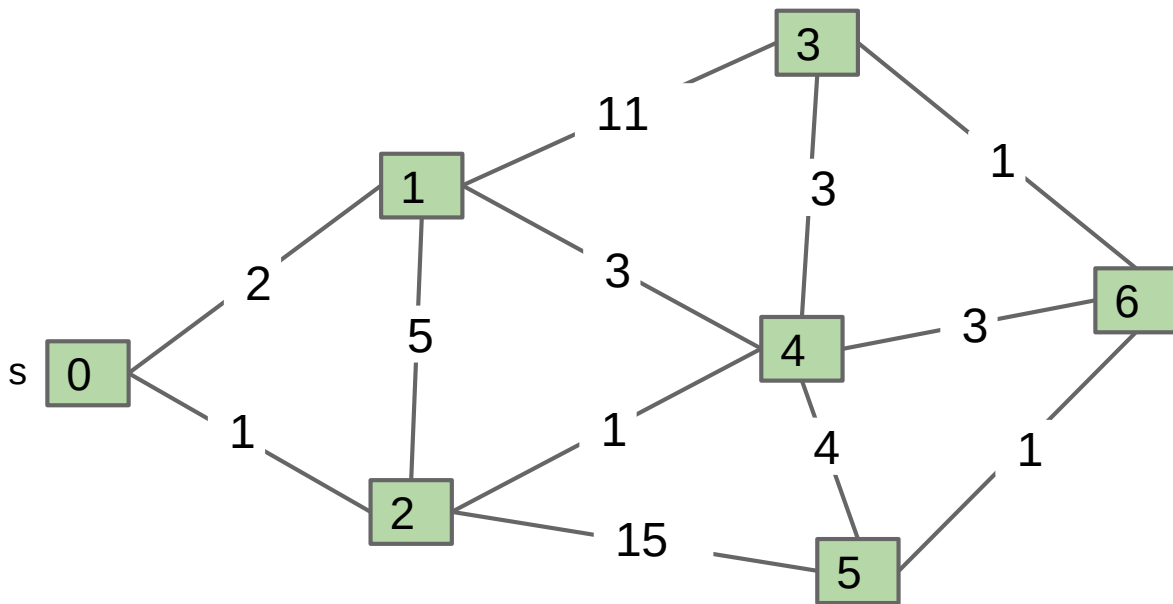
- Consider edges in increasing order of weight
- Add edge to MST (mark black) unless doing so creates a cycle
- Repeat until $V-1$ edges

Kruskal's Demo

Consider edges in order of increasing weight. Add to MST unless a cycle is created.

- Repeat until $V-1$ edges

0-2	1
2-4	1
3-6	1
5-6	1
0-1	2
4-1	3
3-4	3
6-4	3
4-5	4
1-2	5
1-3	11
2-5	15



MST: []

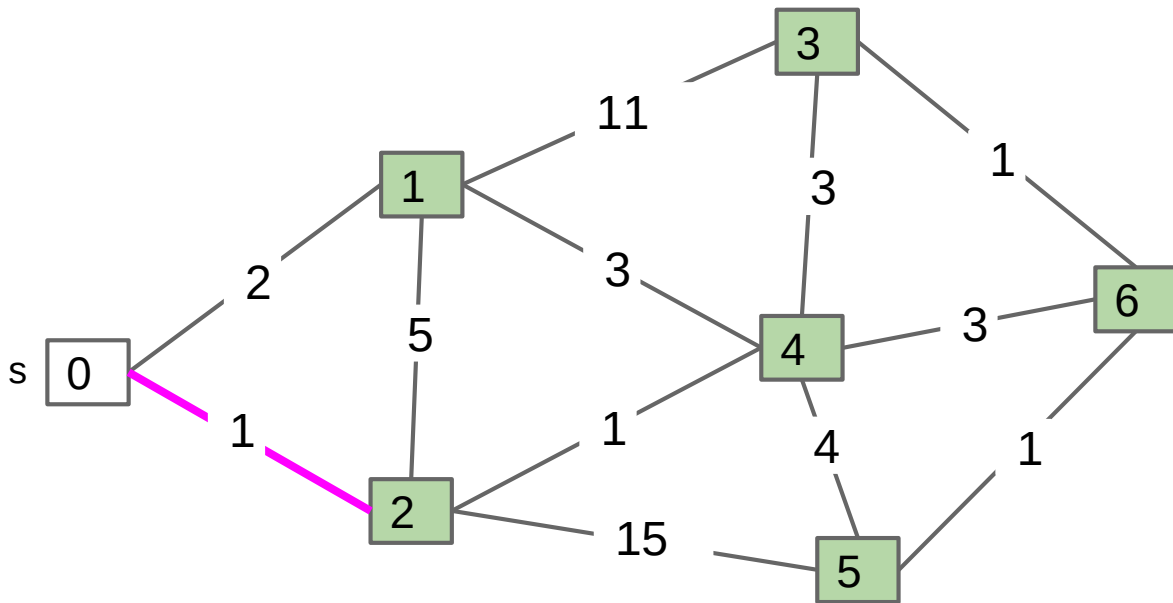
Kruskal's Demo

Consider edges in order of increasing weight. Add to MST unless a cycle is created.

- Repeat until $V-1$ edges

0-2	1
2-4	1
3-6	1
5-6	1
0-1	2
4-1	3
3-4	3
6-4	3
4-5	4
1-2	5
1-3	11
2-5	15

Cycle?



White and green colorings for vertices show cut being implicitly utilized by Kruskal's algorithm

MST: []

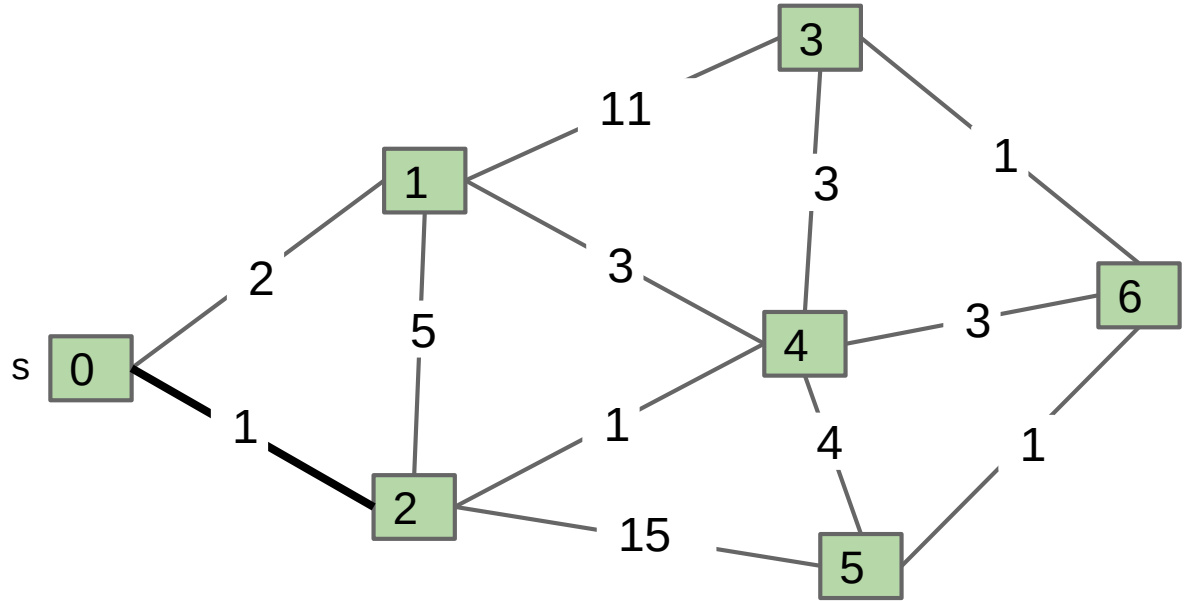
Kruskal's Demo

Consider edges in order of increasing weight. Add to MST unless a cycle is created.

- Repeat until $V-1$ edges

0-2	1
2-4	1
3-6	1
5-6	1
0-1	2
4-1	3
3-4	3
6-4	3
4-5	4
1-2	5
1-3	11
2-5	15

Cycle? No.



MST: [0-2]

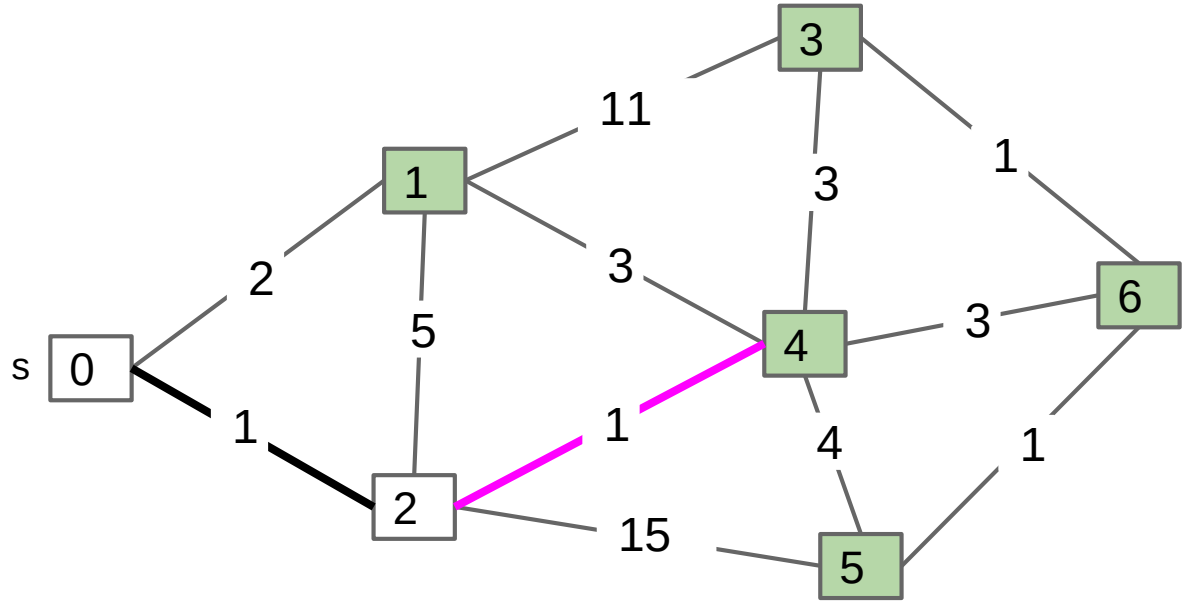
Kruskal's Demo

Consider edges in order of increasing weight. Add to MST unless a cycle is created.

- Repeat until $V-1$ edges

0-2	1
2-4	1
3-6	1
5-6	1
0-1	2
4-1	3
3-4	3
6-4	3
4-5	4
1-2	5
1-3	11
2-5	15

Cycle? No.



MST: [0-2]

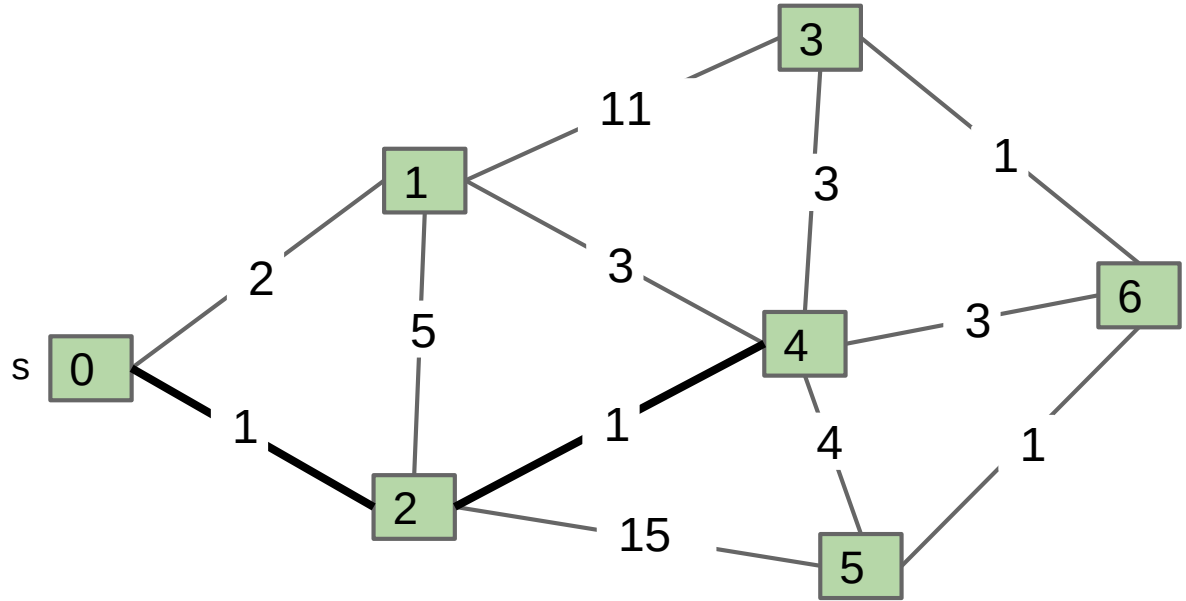
Kruskal's Demo

Consider edges in order of increasing weight. Add to MST unless a cycle is created.

- Repeat until $V-1$ edges

0-2	1
2-4	1
3-6	1
5-6	1
0-1	2
4-1	3
3-4	3
6-4	3
4-5	4
1-2	5
1-3	11
2-5	15

Cycle? No.



MST: [0-2, 2-4]

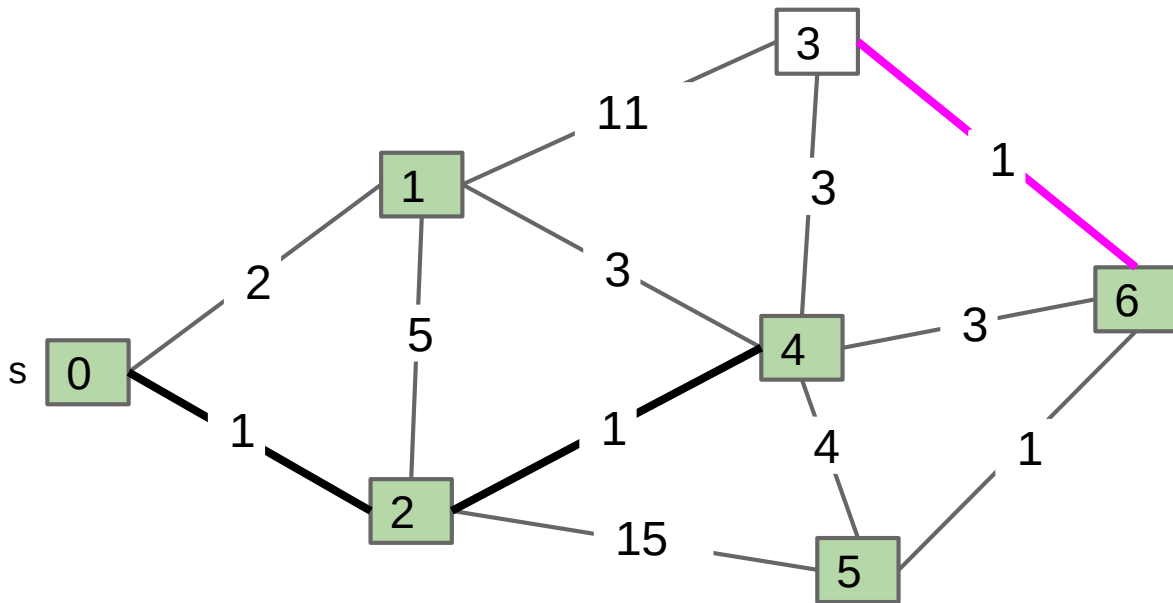
Kruskal's Demo

Consider edges in order of increasing weight. Add to MST unless a cycle is created.

- Repeat until $V-1$ edges

0-2	1
2-4	1
3-6	1
5-6	1
0-1	2
4-1	3
3-4	3
6-4	3
4-5	4
1-2	5
1-3	11
2-5	15

Cycle?



MST: [0-2, 2-4]

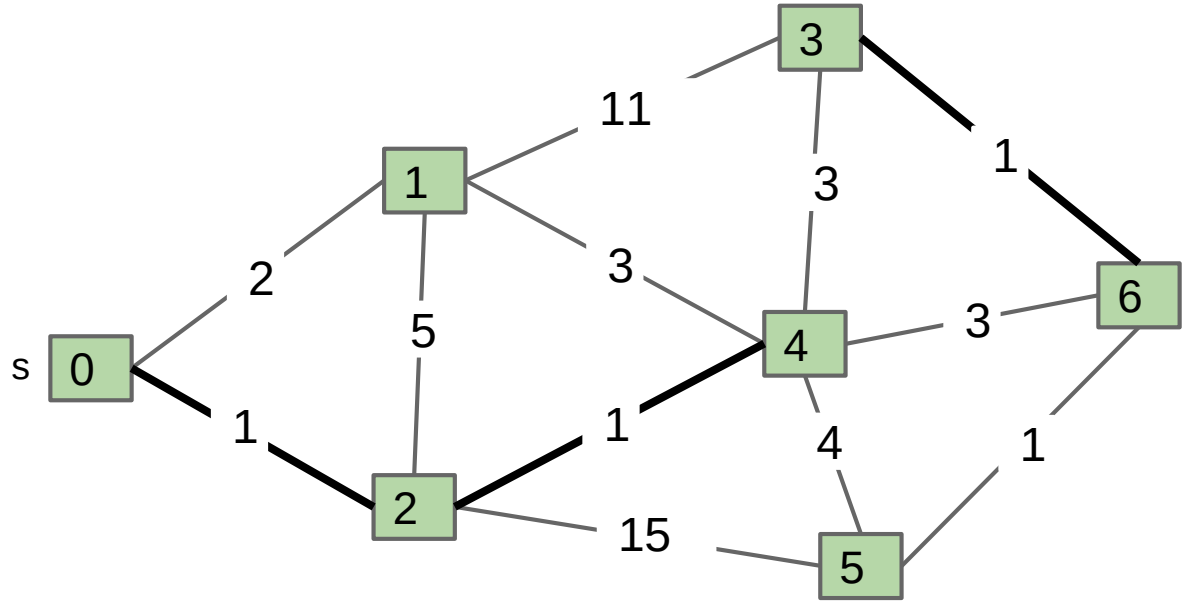
Kruskal's Demo

Consider edges in order of increasing weight. Add to MST unless a cycle is created.

- Repeat until $V-1$ edges

0-2	1
2-4	1
3-6	1
5-6	1
0-1	2
4-1	3
3-4	3
6-4	3
4-5	4
1-2	5
1-3	11
2-5	15

Cycle? No.



MST: [0-2, 2-4, 3-6]

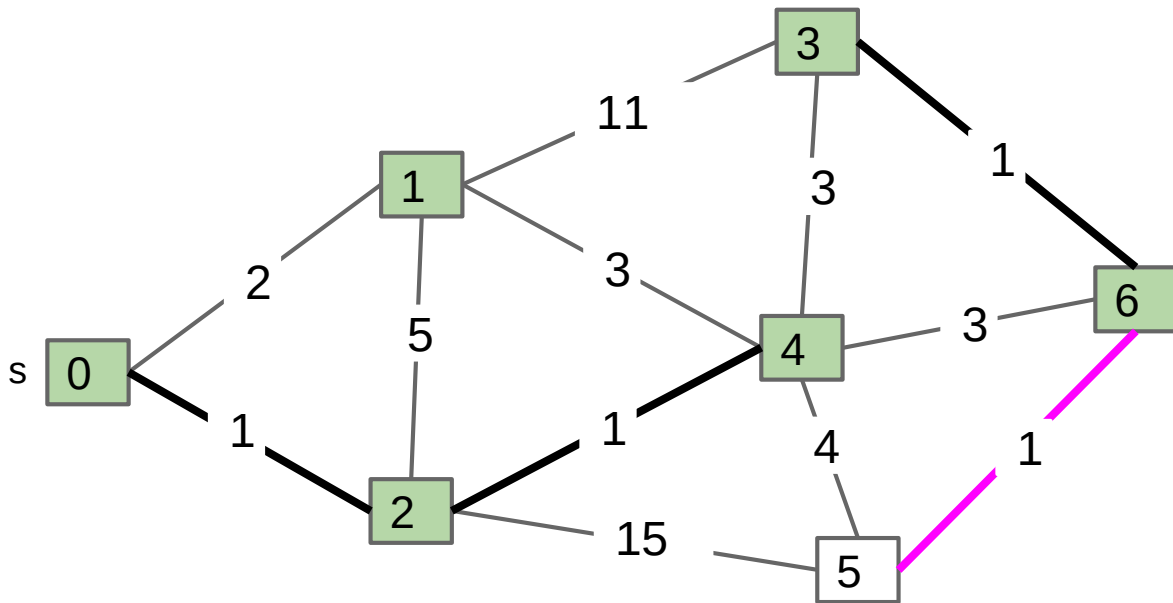
Kruskal's Demo

Consider edges in order of increasing weight. Add to MST unless a cycle is created.

- Repeat until $V-1$ edges

0-2	1
2-4	1
3-6	1
5-6	1
0-1	2
4-1	3
3-4	3
6-4	3
4-5	4
1-2	5
1-3	11
2-5	15

Cycle?



MST: [0-2, 2-4, 3-6]

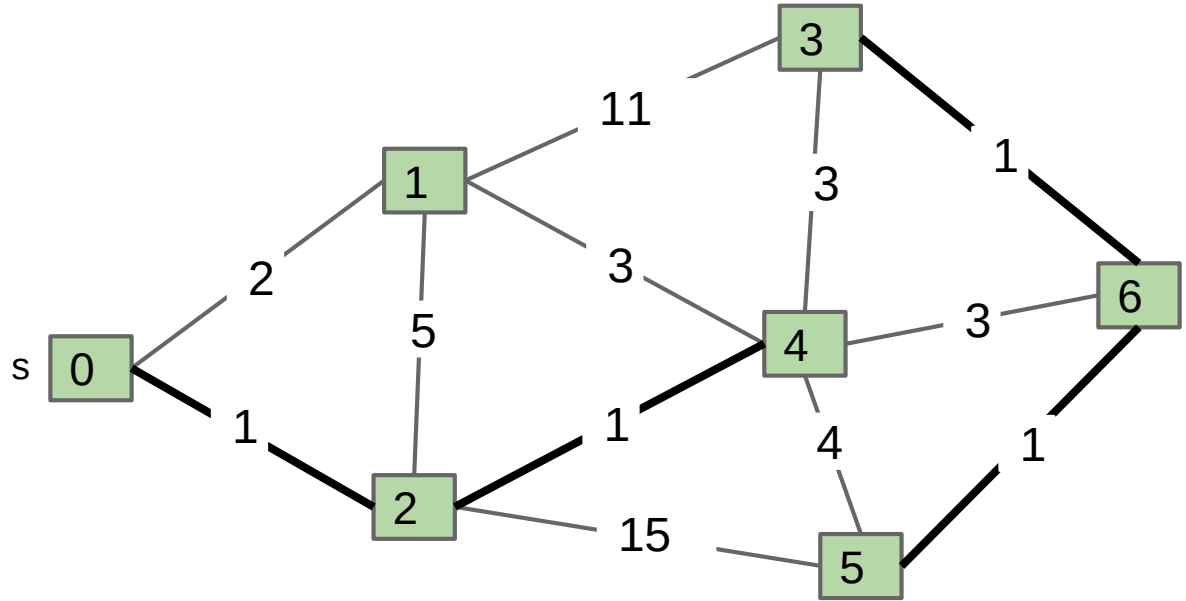
Kruskal's Demo

Consider edges in order of increasing weight. Add to MST unless a cycle is created.

- Repeat until $V-1$ edges

0-2	1
2-4	1
3-6	1
5-6	1
0-1	2
4-1	3
3-4	3
6-4	3
4-5	4
1-2	5
1-3	11
2-5	15

Cycle? No.



MST: [0-2, 2-4, 3-6, 5-6]

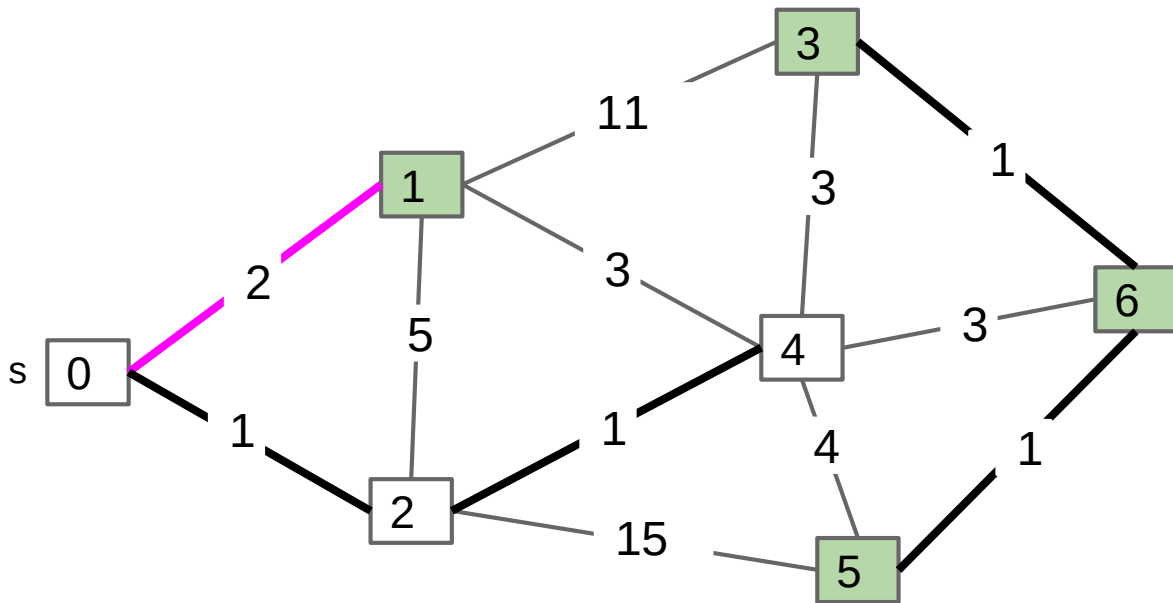
Kruskal's Demo

Consider edges in order of increasing weight. Add to MST unless a cycle is created.

- Repeat until $V-1$ edges

0-2	1
2-4	1
3-6	1
5-6	1
0-1	2
4-1	3
3-4	3
6-4	3
4-5	4
1-2	5
1-3	11
2-5	15

Cycle?



MST: [0-2, 2-4, 3-6, 5-6]

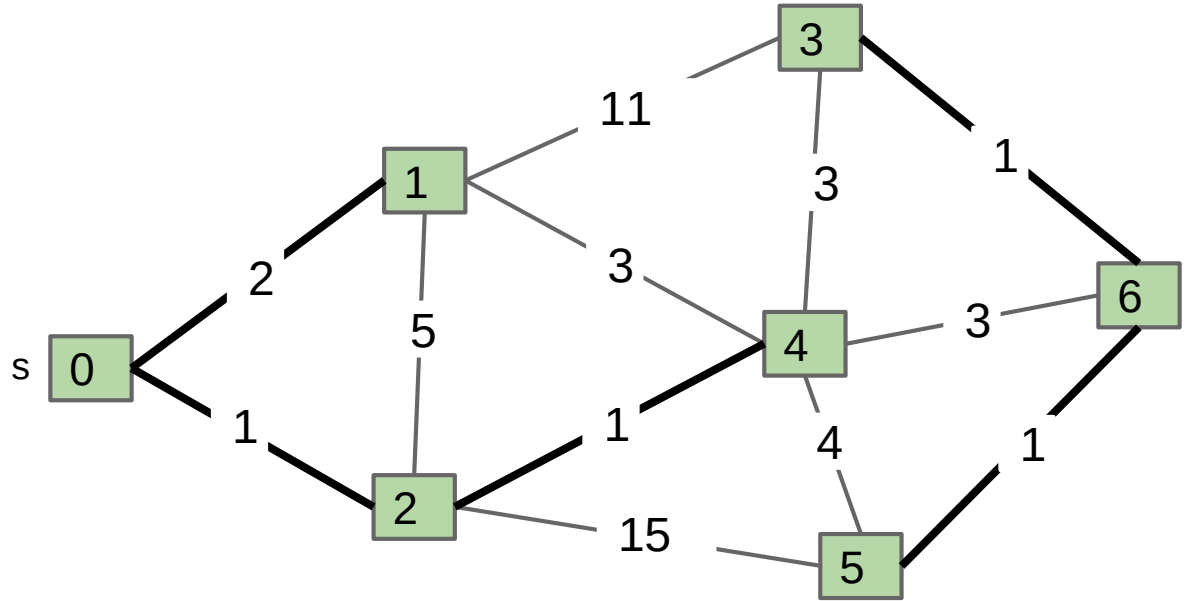
Kruskal's Demo

Consider edges in order of increasing weight. Add to MST unless a cycle is created.

- Repeat until $V-1$ edges

0-2	1
2-4	1
3-6	1
5-6	1
0-1	2
4-1	3
3-4	3
6-4	3
4-5	4
1-2	5
1-3	11
2-5	15

Cycle? No.



MST: [0-2, 2-4, 3-6, 5-6, 0-1]

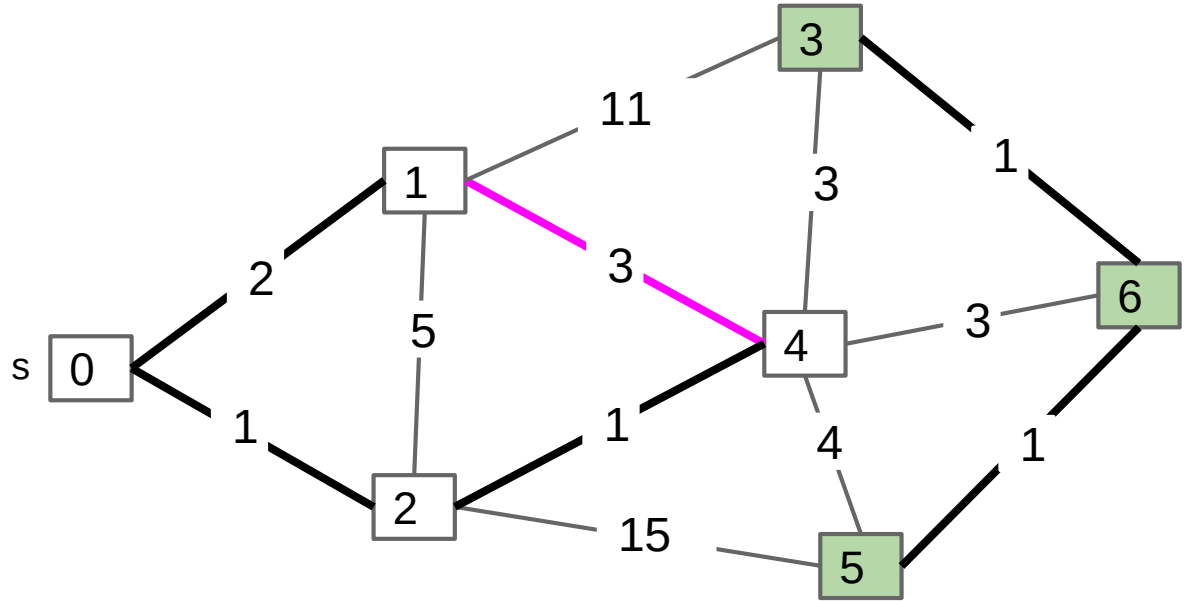
Kruskal's Demo

Consider edges in order of increasing weight. Add to MST unless a cycle is created.

- Repeat until $V-1$ edges

0-2	1
2-4	1
3-6	1
5-6	1
0-1	2
4-1	3
3-4	3
6-4	3
4-5	4
1-2	5
1-3	11
2-5	15

Cycle?



MST: [0-2, 2-4, 3-6, 5-6, 0-1]

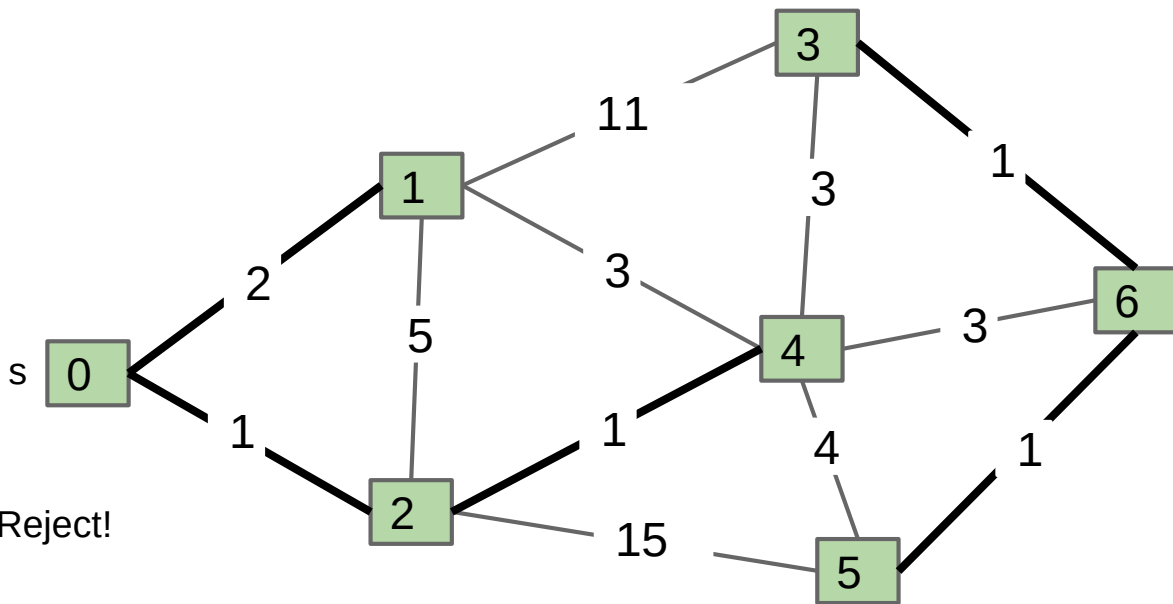
Kruskal's Demo

Consider edges in order of increasing weight. Add to MST unless a cycle is created.

- Repeat until $V-1$ edges

0-2	1
2-4	1
3-6	1
5-6	1
0-1	2
4-1	3
3-4	3
6-4	3
4-5	4
1-2	5
1-3	11
2-5	15

Cycle? Yes. Reject!



MST: [0-2, 2-4, 3-6, 5-6, 0-1]

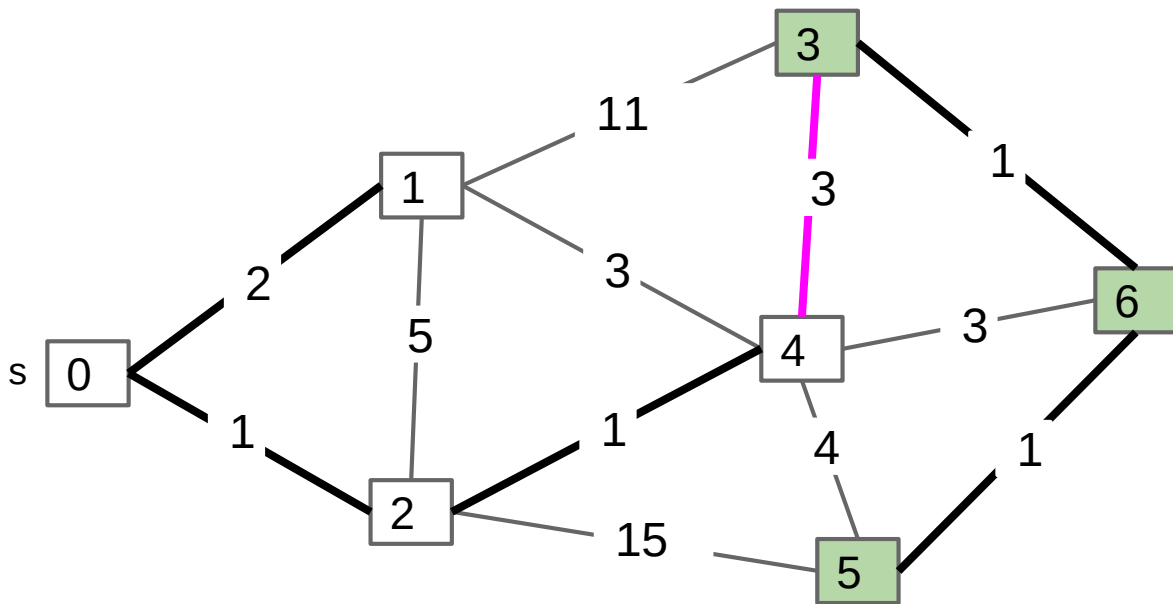
Kruskal's Demo

Consider edges in order of increasing weight. Add to MST unless a cycle is created.

- Repeat until $V-1$ edges

0-2	1
2-4	1
3-6	1
5-6	1
0-1	2
4-1	3
3-4	3
6-4	3
4-5	4
1-2	5
1-3	11
2-5	15

Cycle?



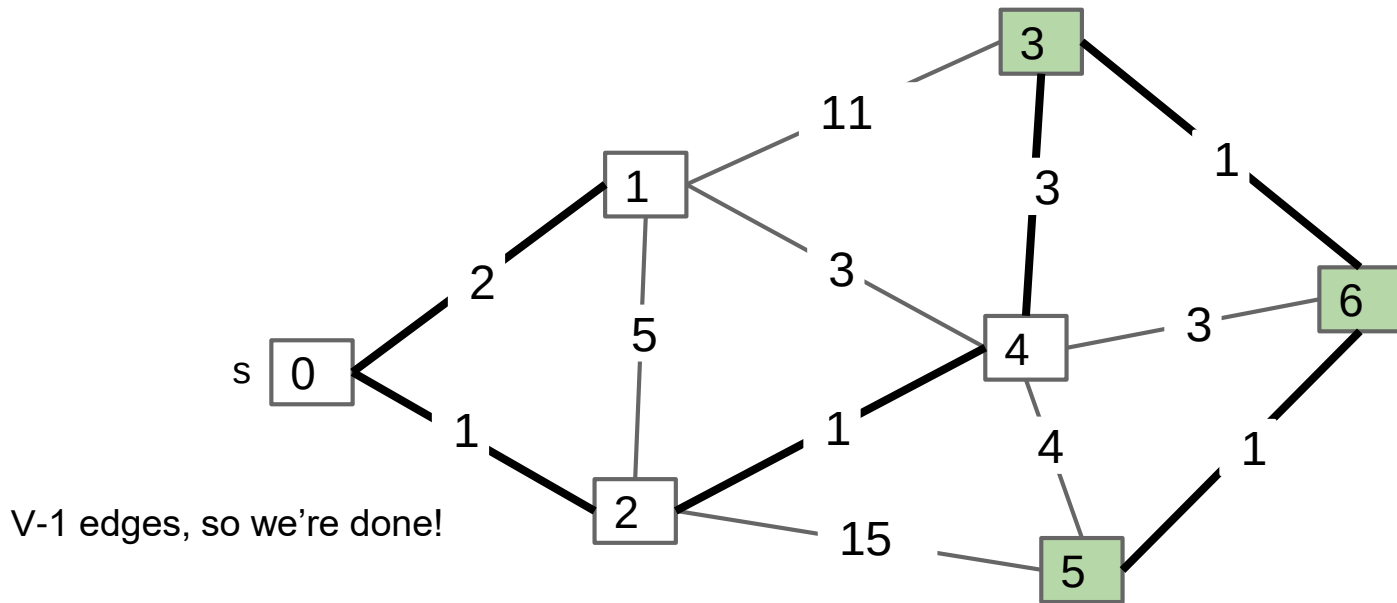
MST: [0-2, 2-4, 3-6, 5-6, 0-1]

Kruskal's Demo

Consider edges in order of increasing weight. Add to MST unless a cycle is created.

- Repeat until $V-1$ edges

0-2	1
2-4	1
3-6	1
5-6	1
0-1	2
4-1	3
3-4	3
6-4	3
4-5	4
1-2	5
1-3	11
2-5	15

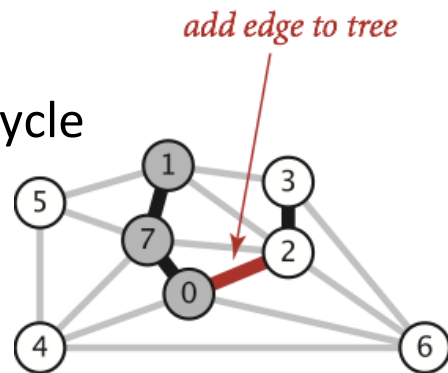


MST: [0-2, 2-4, 3-6, 5-6, 0-1, 3-4]

Kruskal's Algorithm Correctness

Initially mark all edges gray

- Consider edges in increasing order of weight
- Add edge to MST (mark black) unless doing so creates a cycle
- Repeat until $V-1$ edges



Why does Kruskal's work? Special case of generic MST algorithm

- Suppose we add edge $e = v \rightarrow w$
- Side 1 of cut is all vertices connected to v , side 2 is everything else
- No crossing edge is black (since we don't allow cycles)
- No crossing edge has lower weight (consider in increasing order)

how do we **implement** Kruskal's Algorithm?

Kruskal's Algorithm Implementation

Initially mark all edges gray

- Consider edges in increasing order of weight
- Add edge to MST (mark black) unless doing so creates a cycle
- Repeat until $V-1$ edges

Implement Kruskal's with:

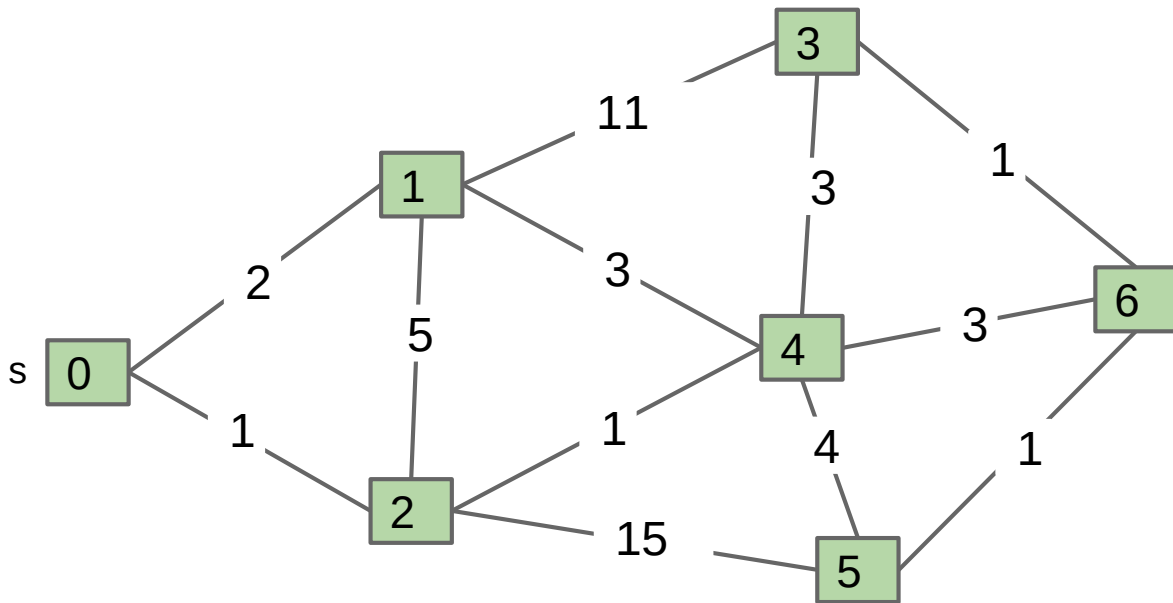
- **MinPQ** using **Binary Heap**, and
- **Union Find** using **Weighted Quick Union with Path Compression** data structures

Kruskal's Implementation Demo

Insert all edges into PQ.

Repeat: Remove smallest weight edge. Add to MST if no cycle created.

PQ: (0-2: 1), (2-4: 1),
(3-6: 1), (5-6: 1),
(0-1: 2), (4-1: 3),
(3-4: 3), (6-4: 3),
(4-5: 4), (1-2: 5),
(1-3: 11), (2-5: 15)



UF: []

MST: []

Kruskal's Implementation Demo

Insert all edges into PQ.

Repeat: Remove smallest weight edge. Add to MST if no cycle created.

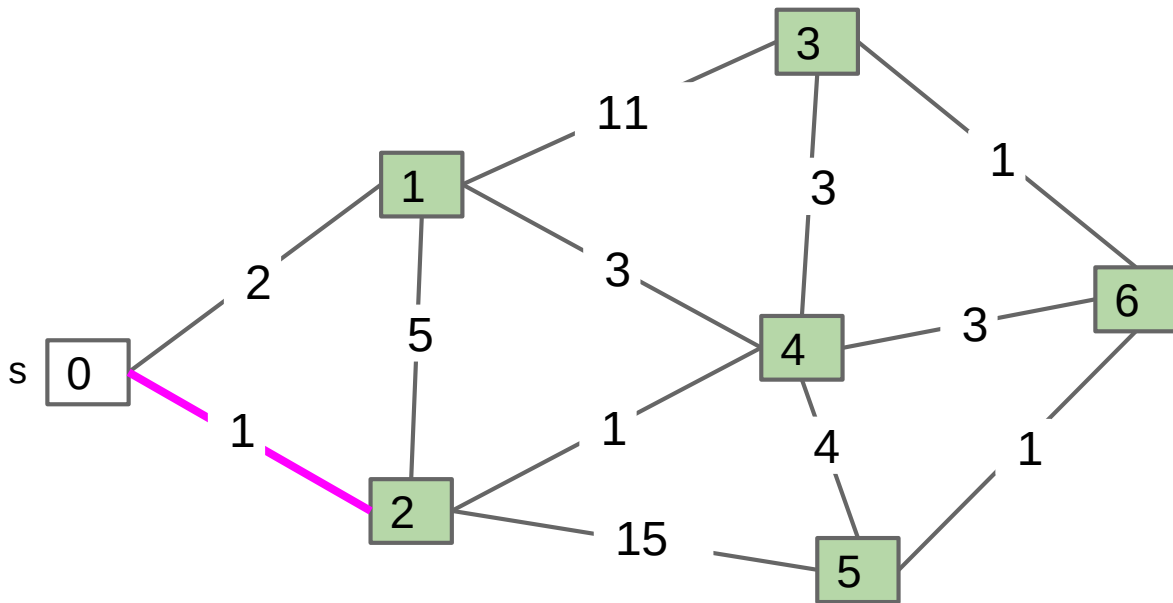
PQ: (2-4: 1),
(3-6: 1), (5-6: 1),
(0-1: 2), (4-1: 3),
(3-4: 3), (6-4: 3),
(4-5: 4), (1-2: 5),
(1-3: 11), (2-5: 15)

Removed edge: e=0-2

Cycle? isSameGroup(0, 2)

UF: []

MST: []

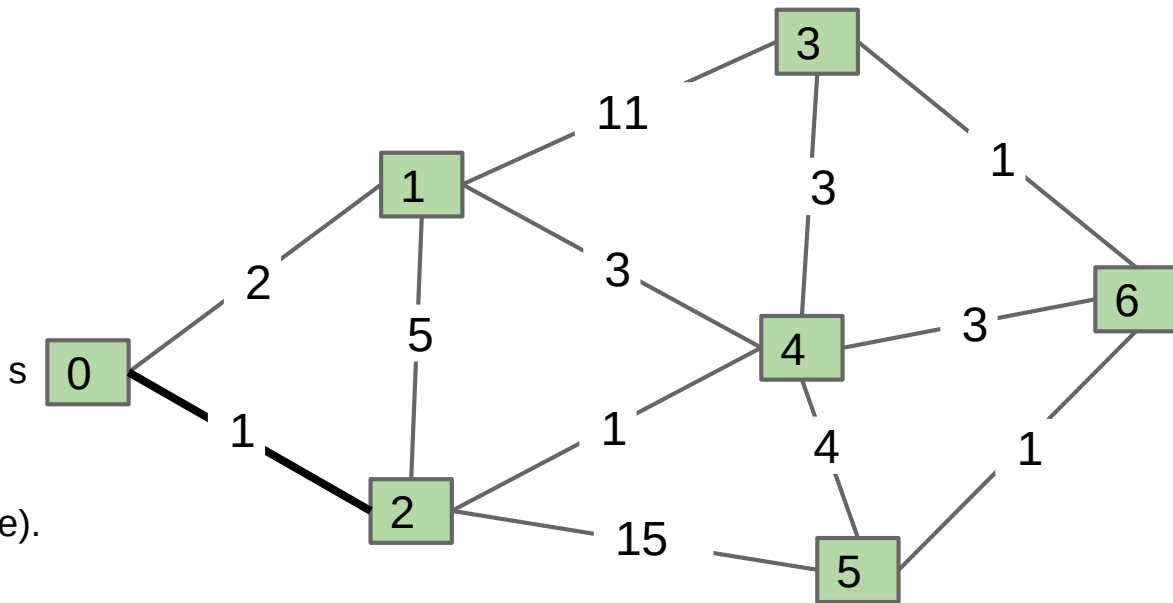


Kruskal's Implementation Demo

Insert all edges into PQ.

Repeat: Remove smallest weight edge. Add to MST if no cycle created.

PQ: (2-4: 1),
(3-6: 1), (5-6: 1),
(0-1: 2), (4-1: 3),
(3-4: 3), (6-4: 3),
(4-5: 4), (1-2: 5),
(1-3: 11), (2-5: 15)



Removed edge: $e=0-2$

Cycle? No. $\text{union}(0, 2)$. $\text{add}(e)$.

UF: [0-2]

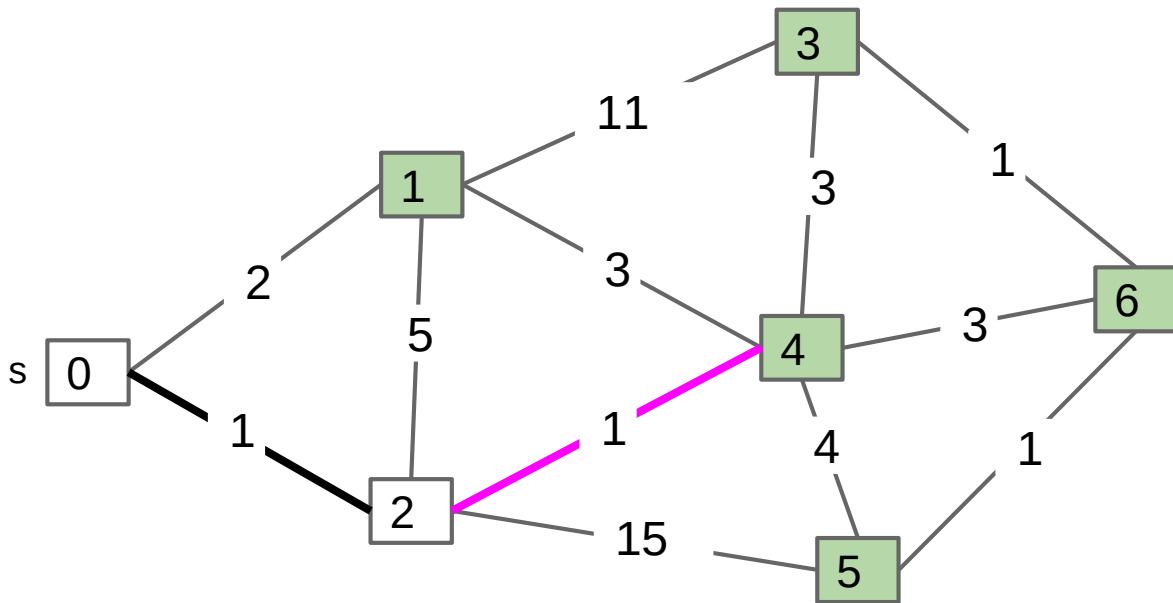
MST: [0-2]

Kruskal's Implementation Demo

Insert all edges into PQ.

Repeat: Remove smallest weight edge. Add to MST if no cycle created.

PQ: (3-6: 1), (5-6: 1),
(0-1: 2), (4-1: 3),
(3-4: 3), (6-4: 3),
(4-5: 4), (1-2: 5),
(1-3: 11), (2-5: 15)



Removed edge: $e=2-4$

Cycle? `isSameGroup(2, 4)`

UF: [0-2]

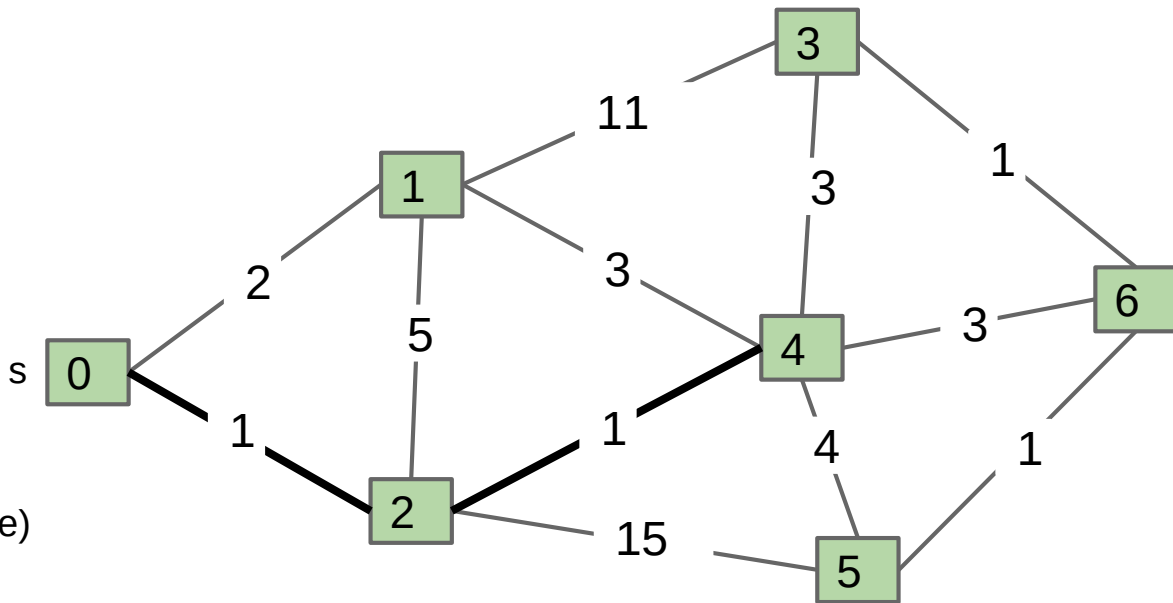
MST: [0-2]

Kruskal's Implementation Demo

Insert all edges into PQ.

Repeat: Remove smallest weight edge. Add to MST if no cycle created.

PQ: (3-6: 1), (5-6: 1),
(0-1: 2), (4-1: 3),
(3-4: 3), (6-4: 3),
(4-5: 4), (1-2: 5),
(1-3: 11), (2-5: 15)



Removed edge: $e=2-4$

Cycle? No. $\text{union}(2, 4)$. $\text{add}(e)$

UF: [0-2-4]

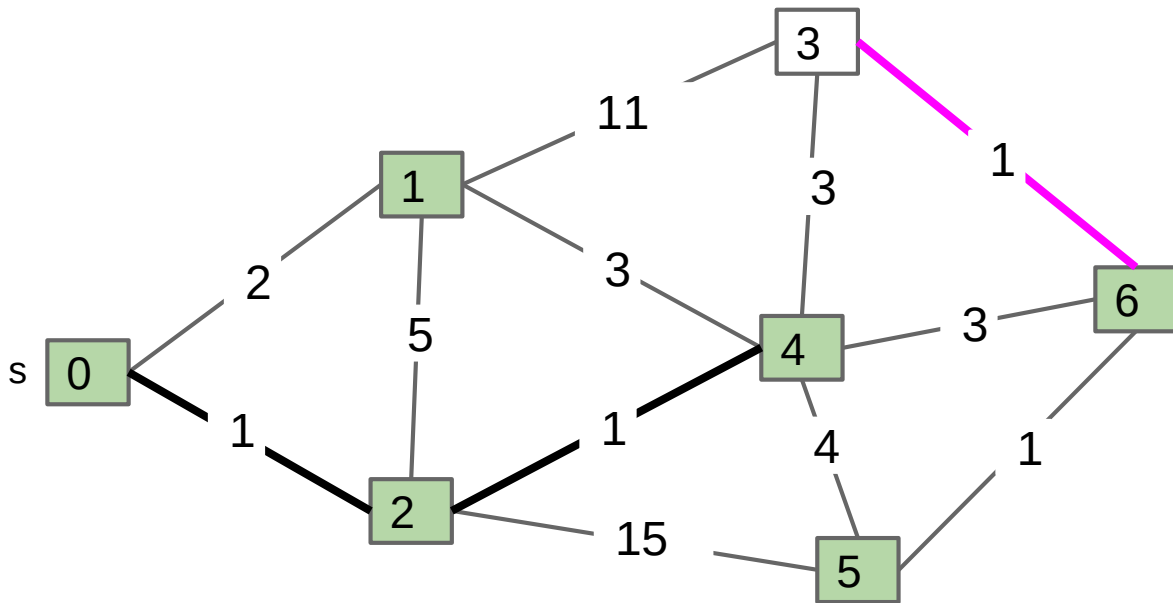
MST: [0-2, 2-4]

Kruskal's Implementation Demo

Insert all edges into PQ.

Repeat: Remove smallest weight edge. Add to MST if no cycle created.

PQ: (5-6: 1),
(0-1: 2), (4-1: 3),
(3-4: 3), (6-4: 3),
(4-5: 4), (1-2: 5),
(1-3: 11), (2-5: 15)



Removed edge: 3-6

Cycle? isSameGroup(3, 6)

UF: [0-2-4]

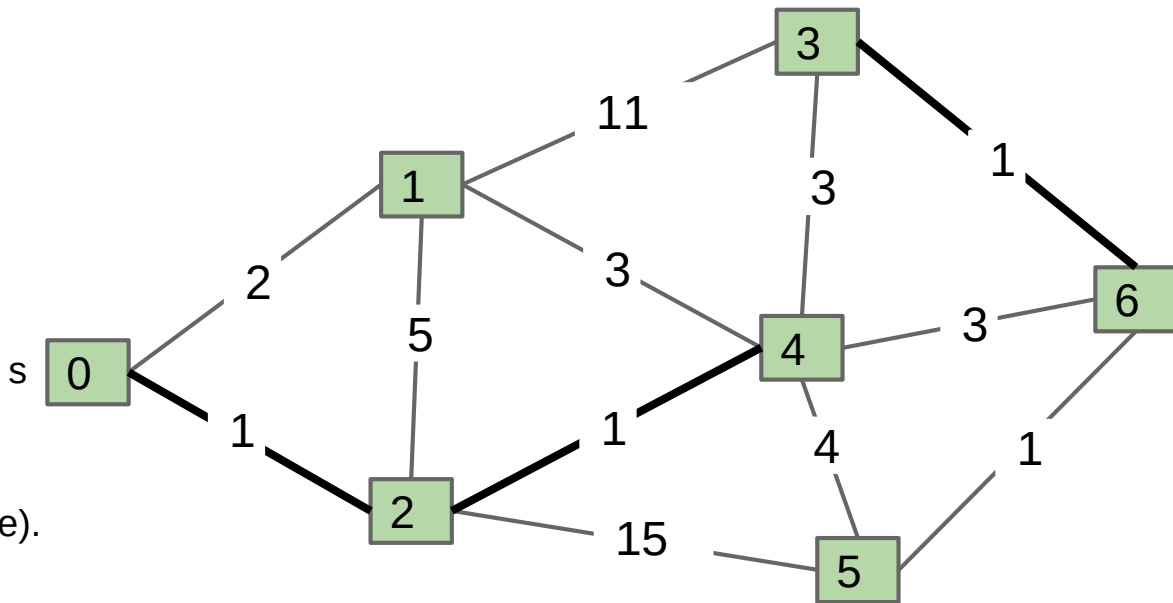
MST: [0-2, 2-4]

Kruskal's Implementation Demo

Insert all edges into PQ.

Repeat: Remove smallest weight edge. Add to MST if no cycle created.

PQ: (5-6: 1),
(0-1: 2), (4-1: 3),
(3-4: 3), (6-4: 3),
(4-5: 4), (1-2: 5),
(1-3: 11), (2-5: 15)



Removed edge: $e=3-6$

Cycle? No. $\text{union}(3, 6)$. $\text{add}(e)$.

UF: [0-2-4, 3-6]

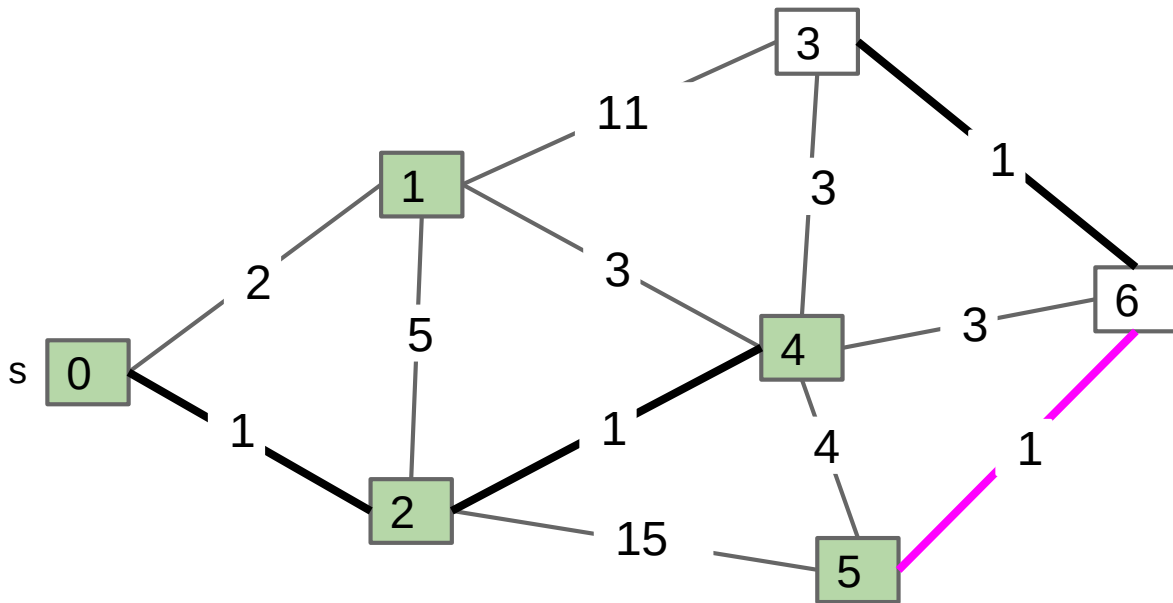
MST: [0-2, 2-4, 3-6]

Kruskal's Implementation Demo

Insert all edges into PQ.

Repeat: Remove smallest weight edge. Add to MST if no cycle created.

PQ: (0-1: 2), (4-1: 3),
(3-4: 3), (6-4: 3),
(4-5: 4), (1-2: 5),
(1-3: 11), (2-5: 15)



Removed edge: e=5-6

Cycle? isSameGroup(5, 6)

UF: [0-2-4, 3-6]

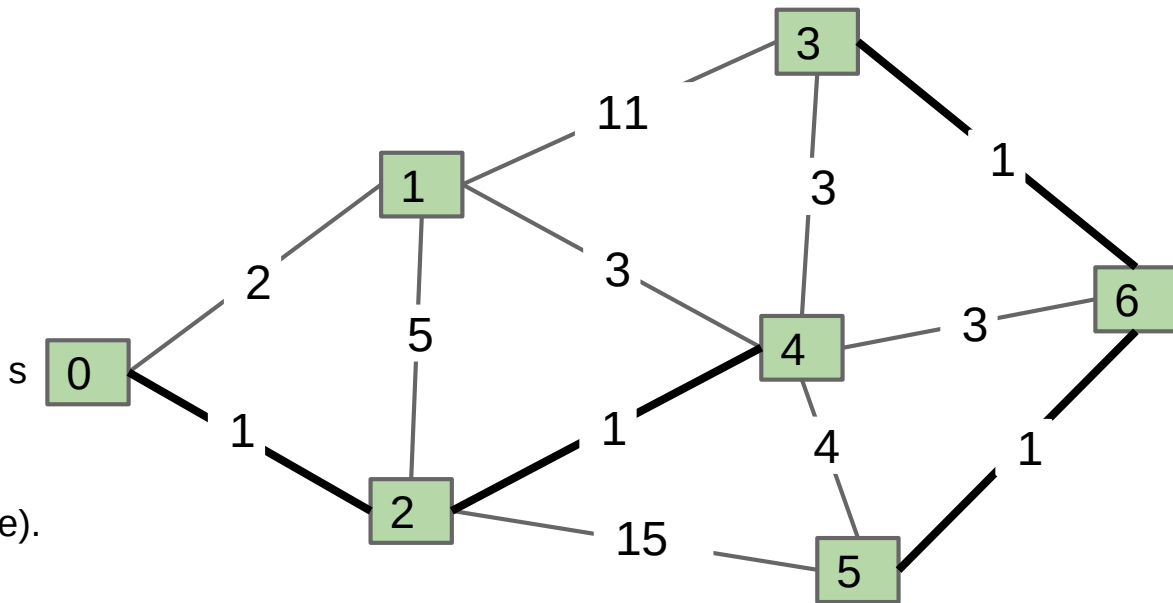
MST: [0-2, 2-4, 3-6]

Kruskal's Implementation Demo

Insert all edges into PQ.

Repeat: Remove smallest weight edge. Add to MST if no cycle created.

PQ: (0-1: 2), (4-1: 3),
(3-4: 3), (6-4: 3),
(4-5: 4), (1-2: 5),
(1-3: 11), (2-5: 15)



Removed edge: $e=5-6$

Cycle? No. union(5, 6). add(e).

UF: [0-2-4, 3-6-5]

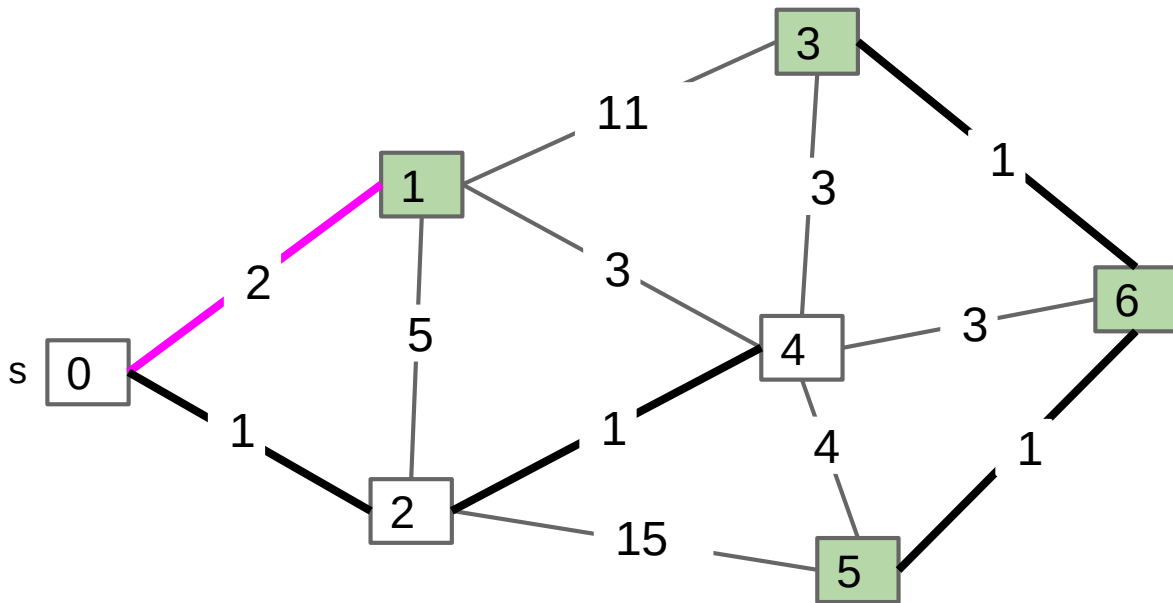
MST: [0-2, 2-4, 3-6, 5-6]

Kruskal's Implementation Demo

Insert all edges into PQ.

Repeat: Remove smallest weight edge. Add to MST if no cycle created.

PQ: (4-1: 3),
(3-4: 3), (6-4: 3),
(4-5: 4), (1-2: 5),
(1-3: 11), (2-5: 15)



Removed edge: $e=0-1$

Cycle? `isSameGroup(0, 1)`

UF: [0-2-4, 3-6-5]

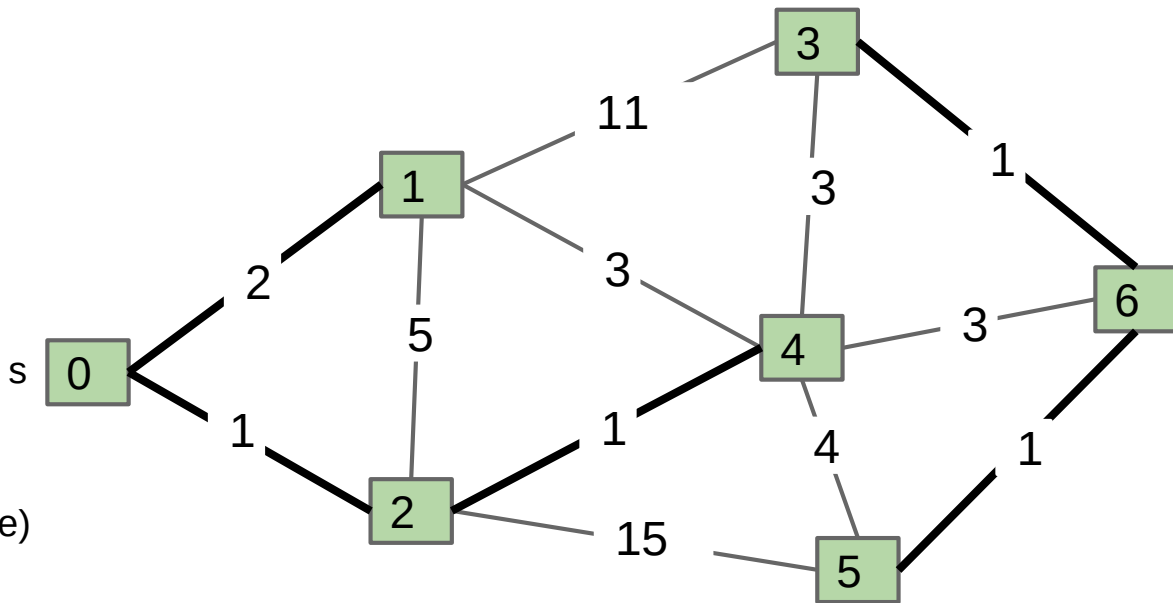
MST: [0-2, 2-4, 3-6, 5-6]

Kruskal's Implementation Demo

Insert all edges into PQ.

Repeat: Remove smallest weight edge. Add to MST if no cycle created.

PQ: (4-1: 3),
(3-4: 3), (6-4: 3),
(4-5: 4), (1-2: 5),
(1-3: 11), (2-5: 15)



Removed edge: $e=0-1$

Cycle? No. $\text{union}(0, 1)$. $\text{add}(e)$

UF: [0-2-4-1, 3-6-5]

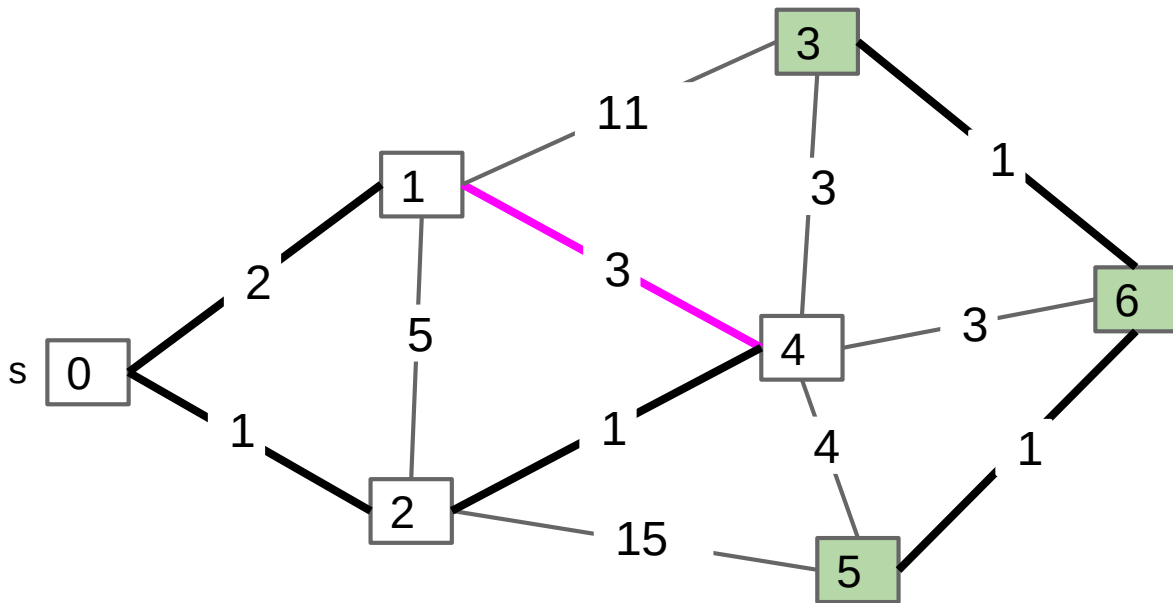
MST: [0-2, 2-4, 3-6, 5-6, 0-1]

Kruskal's Implementation Demo

Insert all edges into PQ.

Repeat: Remove smallest weight edge. Add to MST if no cycle created.

PQ: (3-4: 3), (6-4: 3),
(4-5: 4), (1-2: 5),
(1-3: 11), (2-5: 15)



Removed edge: $e=1-4$

Cycle? `isSameGroup(1, 4)`

UF: `[0-2-4-1, 3-6-5]`

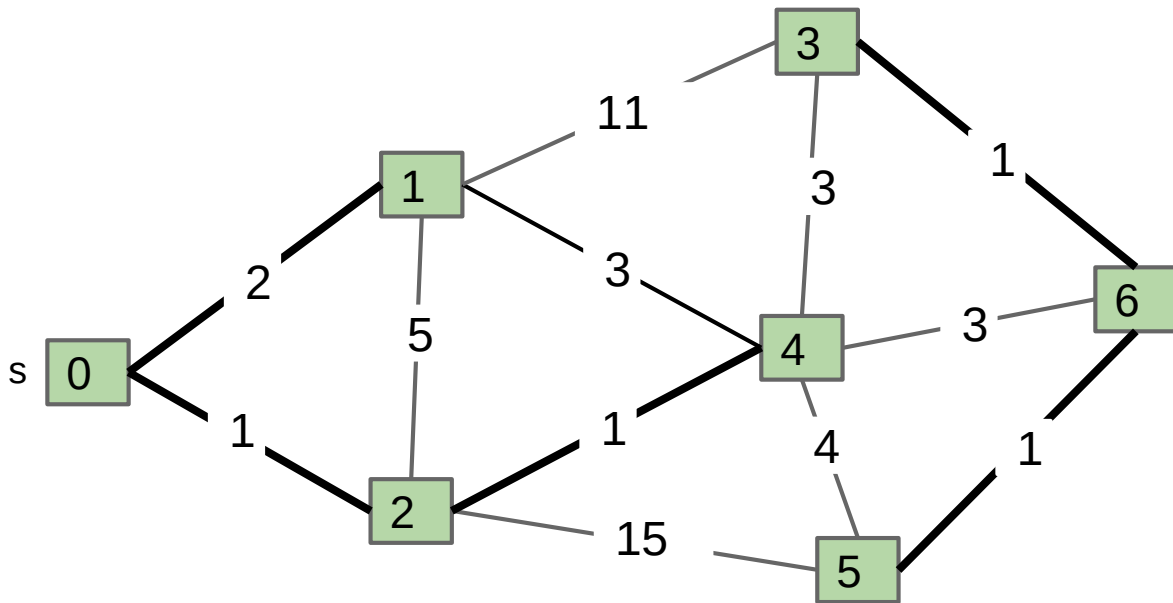
MST: `[0-2, 2-4, 3-6, 5-6, 0-1]`

Kruskal's Implementation Demo

Insert all edges into PQ.

Repeat: Remove smallest weight edge. Add to MST if no cycle created.

PQ: (3-4: 3), (6-4: 3),
(4-5: 4), (1-2: 5),
(1-3: 11), (2-5: 15)



Removed edge: $e=1-4$

Cycle? Yes. Do nothing.

UF: [0-2-4-1, 3-6-5]

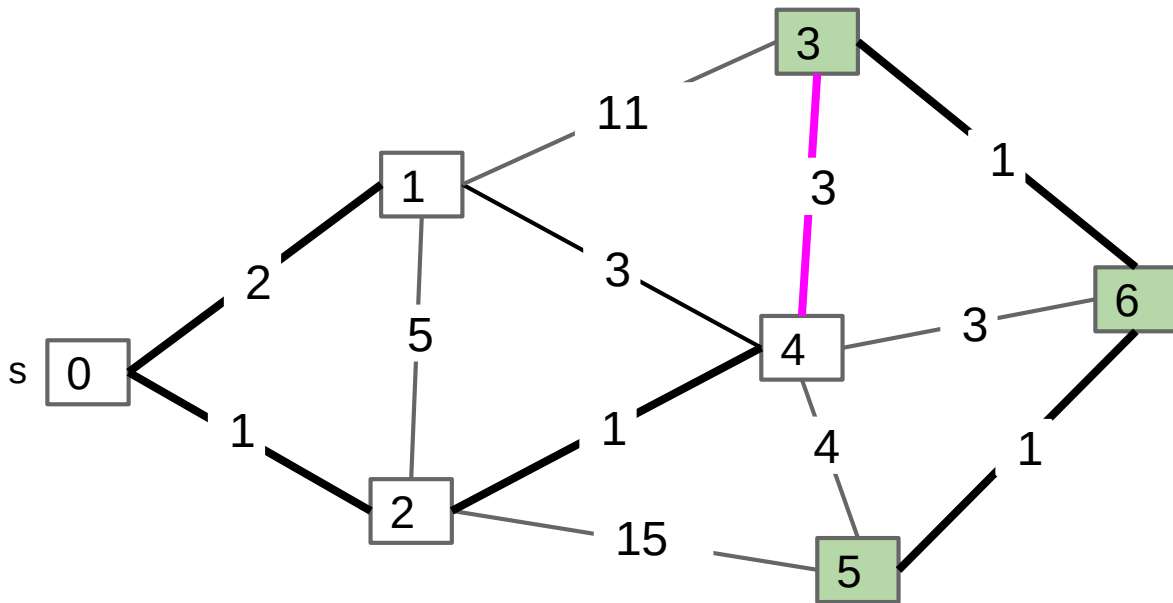
MST: [0-2, 2-4, 3-6, 5-6, 0-1]

Kruskal's Implementation Demo

Insert all edges into PQ.

Repeat: Remove smallest weight edge. Add to MST if no cycle created.

PQ: (6-4: 3),
(4-5: 4), (1-2: 5),
(1-3: 11), (2-5: 15)



Removed edge: $e=3-4$

Cycle? `isSameGroup(3, 4)`

UF: [0-2-4-1, 3-6-5]

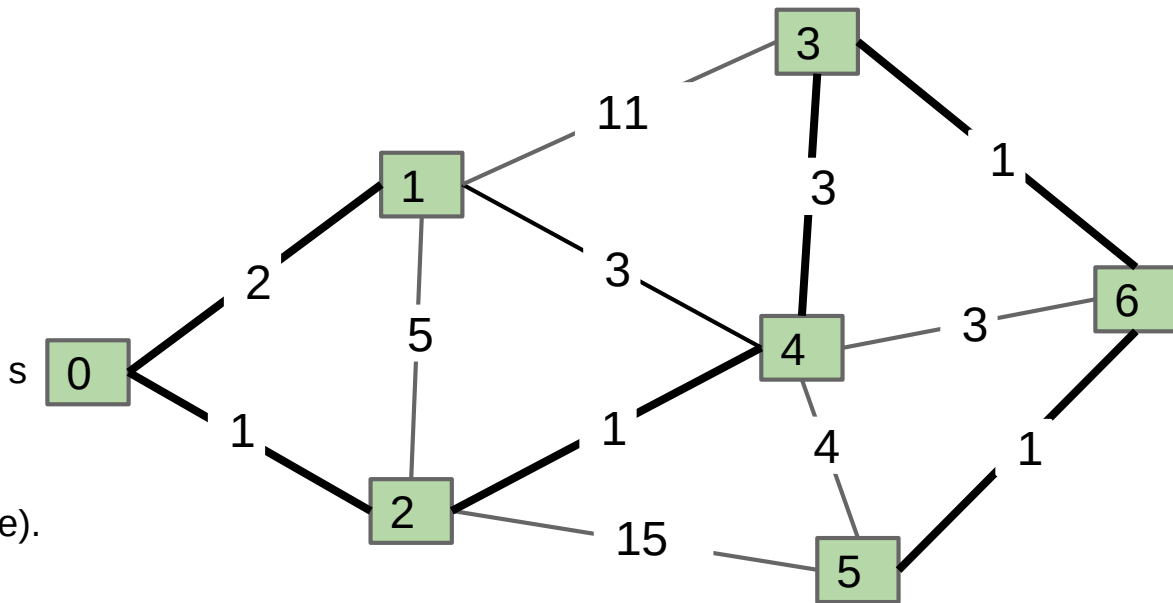
MST: [0-2, 2-4, 3-6, 5-6, 0-1]

Kruskal's Implementation Demo

Insert all edges into PQ.

Repeat: Remove smallest weight edge. Add to MST if no cycle created.

PQ: (6-4: 3),
(4-5: 4), (1-2: 5),
(1-3: 11), (2-5: 15)



Removed edge: $e=3-4$

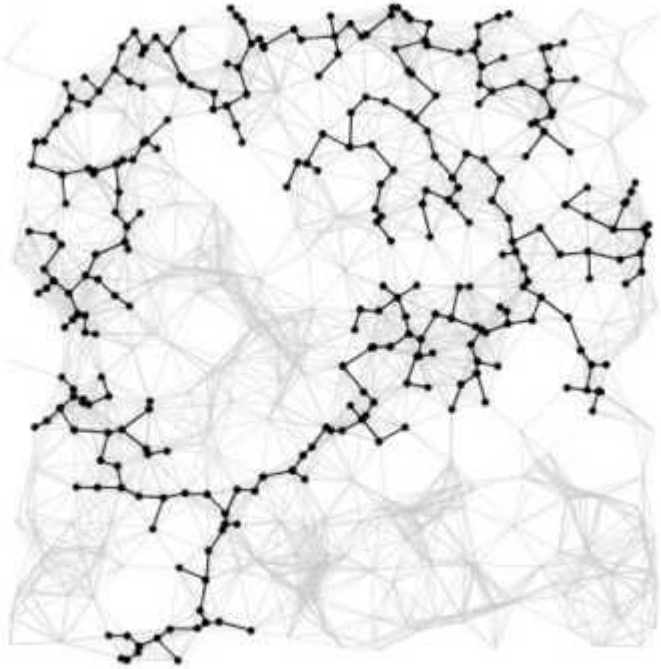
Cycle? No. $\text{union}(3, 4)$. $\text{add}(e)$.

UF: $[\emptyset-2-4-\boxed{1-3}-6-5]$

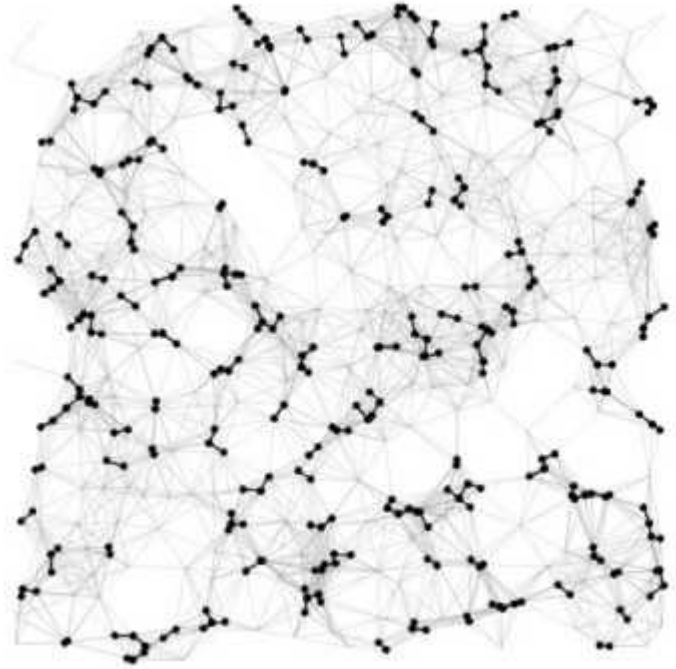
MST: $[\emptyset-2, 2-4, 3-6, 5-6, \emptyset-1, \boxed{3-4}]$

V-1 edges, so done.

Prim's vs. Kruskal's



Prim's Algorithm



Kruskal's Algorithm

Kruskal's Implementation

```
public class KruskalMST {
    private List<Edge> mst = new ArrayList<Edge>();

    public KruskalMST(EdgeWeightedGraph G) {
        MinPQ<Edge> pq = new MinPQ<Edge>();
        for (Edge e : G.edges()) {
            pq.add(e);
        }
        WeightedQuickUnionPC uf =
            new WeightedQuickUnionPC(G.V());
        while (!pq.isEmpty() && mst.size() < G.V()-1) {
            Edge e = pq.delMin();
            int v = e.from();
            int w = e.to();
            if (!uf.isSameGroup(v, w)) {
                uf.union(v, w);
                mst.add(e);
            }
        }
    }
}
```

Kruskal's Runtime

```
public class KruskalMST {
    private List<Edge> mst = new ArrayList<Edge>();

    public KruskalMST(EdgeWeightedGraph G) {
        MinPQ<Edge> pq = new MinPQ<Edge>();
        for (Edge e : G.edges()) {
            pq.add(e);
        }
        WeightedQuickUnionPC uf =
            new WeightedQuickUnionPC(G.V());
        while (!pq.isEmpty() && mst.size() < G.V()-1) {
            Edge e = pq.delMin();
            int v = e.from();
            int w = e.to();
            if (!uf.isSameGroup(v, w)) {
                uf.union(v, w);
                mst.add(e);
            }
        }
    }
}
```

What is the runtime of Kruskal's algorithm?

- Assume all PQ operations (*Binary Heap MinPQ – Lecture 8*) take $O(\log(V))$ time.
- Assume all UF-WQUPC (*Union Find-Weighted Quick Union with Path Compression – Lecture 8 + Exercise 8*) operations take $O(\log^* V)$ time
- Give your answer in Big O notation

Kruskal's Runtime

Kruskal's algorithm running time is **$O(E \log E)$**

Operation	Number of Times	Time per Operation	Total Time
add	E	$O(\log E)$	$O(E \log E)$
delMin	$O(E)$	$O(\log E)$	$O(E \log E)$
union	$O(V)$	$O(\log^* V)$	$O(V \log^* V)$
isSameGroup	$O(E)$	$O(\log^* V)$	$O(E \log^* V)$

If edges are already sorted, the running time is **$O(E \log^* V)$**

Question: Prim's runtime is $O(E \log V)$ and Kruskal's is $O(E \log E)$
Can we do better than them?

State-of-the-Art MST Algorithms

Year	Worst case	Discovered by
1975	$E \log \log V$	Yao
1984	$E \log^* V$	Fredman-Tarjan
1986	$E \log (\log^* V)$	Gabow-Galil-Spencer-Tarjan
1997	$E \alpha(V) \log \alpha(V)$	Chazelle
2000	$E \alpha(V)$	Chazelle
2002	<i>optimal</i>	Pettie-Ramachandran
?	E	?

Shortest Paths and MST Algorithms Summary

Problem	Algorithm	Runtime (if $E > V$)	Notes
Shortest Paths	Dijkstra's	$O(E \log V)$	Fails for negative weight edges
MST	Prim's	$O(E \log V)$	Analogous to Dijkstra's
MST	Kruskal's	$O(E \log E)$	Uses WQUPC

Thank you for your attention !

- **In this lecture, you have learned:**
 - to represent shortest path as a shortest path tree with $V-1$ edges
 - to find a shortest path tree of single source, multiple targets using Dijkstra's algorithm
 - to find a minimum spanning tree using Prim's algorithm and Kruskal's algorithms
 - to implement and analyze them
- Please continue to work on **Exercise 10**